

A Systematic Literature Review on Large Language Models for Automated Program Repair

QUANJUN ZHANG, School of Computer Science and Engineering, Nanjing University of Science and Technology, China

CHUNRONG FANG, YANG XIE, and YUXIANG MA, State Key Laboratory for Novel Software Technology, Nanjing University, China

WEISONG SUN, School of Computer Science and Engineering, Nanyang Technological University, Singapore

YUN YANG, Department of Computing Technologies, Swinburne University of Technology, Australia

ZHENYU CHEN, State Key Laboratory for Novel Software Technology, Nanjing University, China

Automated Program Repair (APR) attempts to patch software bugs and reduce manual debugging efforts. Very recently, with the advances in Large Language Models (LLMs), a rapidly increasing number of APR techniques have been proposed, significantly facilitating software development and maintenance and demonstrating remarkable performance. However, due to ongoing explorations in the LLM-based APR field, it is challenging for researchers to understand the current achievements, challenges, and potential opportunities. This work provides the first systematic literature review to summarize the applications of LLMs in APR between 2020 and 2025. We analyze 189 relevant papers from LLMs, APR and their integration perspectives. First, we categorize existing popular LLMs that are applied to support APR and outline four types of utilization strategies for their deployment. Besides, we detail some specific repair scenarios that benefit from LLMs, e.g., semantic bugs and security vulnerabilities. Furthermore, we discuss several critical aspects of integrating LLMs into APR research, e.g., input forms and open science. Finally, we highlight a set of challenges remaining to be investigated and the potential guidelines for future research. Overall, our paper provides a systematic overview of the research landscape to the APR community, helping researchers gain a comprehensive understanding of achievements and promote future research. Our artifacts are publicly available at the GitHub repository: <https://github.com/iSEngLab/AwesomeLLM4APR>.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**.

Additional Key Words and Phrases: Large Language Model, Automated Program Repair, LLM4APR

1 Introduction

Software bugs are recognized as inevitable and destructive [75, 132, 171], posing safety issues for users worldwide and costing billions of dollars in financial losses annually [10, 23, 69, 166]. It is non-trivial and time-consuming for developers to fix detected software bugs manually [8, 21, 221, 243]. **Automated Program Repair (APR)** plays a crucial role in software development and maintenance, aiming to fix software bugs with minimal human

Authors' Contact Information: Quanjun Zhang, quanjunzhang@njust.edu.cn, School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, China; Chunrong Fang, fangchunrong@nju.edu.cn; Yang Xie, serialxy@outlook.com; Yuxiang Ma, 502022320009@smail.nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; Weisong Sun, weisong.sun@ntu.edu.sg, School of Computer Science and Engineering, Nanyang Technological University, 50 Nanyang Avenue, Singapore; Yun Yang, yyang@swin.edu.au, Department of Computing Technologies, Swinburne University of Technology, Melbourne, Australia; Zhenyu Chen, zychen@nju.edu.cn, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/2-ART

<https://doi.org/10.1145/3799693>

intervention. Following the foundational work GenProg [120, 242] in 2009, APR has been extensively investigated over the past decades [66, 163], and researchers have proposed a variety of APR techniques, including heuristic-based [95, 120, 157, 282], constraint-based [45, 158, 261, 264], and pattern-based [115, 140, 141] ones. Recently, inspired by the advances of **Deep Learning (DL)**, an increasing number of learning-based APR techniques have been proposed that utilize neural network models to automatically learn bug-fixing patterns [30, 97, 127, 128, 149, 224, 274–276, 318, 319]. Thanks to the powerful ability of DL models to learn hidden repair patterns from massive code corpora, learning-based APR has achieved remarkable performance in the last couple of years [292], attracting considerable attention from both academia and industry [101, 102, 107].

Very recently, **Large Language Models (LLMs)** have been successfully applied to a broad range of source code-related tasks [231, 294, 308], such as code generation [125, 232, 235, 320], code summarization [210, 211, 230], and test generation [4, 38, 86, 170, 202, 297–299]. Benefiting from massive model parameters and vast training data, LLMs have demonstrated impressive performance and fundamentally revolutionized the research paradigm in the **Software Engineering (SE)** community. In the domain of APR, beginning with pioneering studies, e.g., TFix [15], CIRCLE [281] and AlphaRepair [255], the community has witnessed an explosion of repair studies utilizing LLMs, already achieving considerable advantages and further indicating significant potential for future research. However, the integration of LLMs within APR is a considerably complex undertaking, making it difficult for interested researchers to understand existing work. For example, existing LLM-based APR studies encompass different research perspectives (e.g., empirical [254], technical [255] and benchmark studies [300]), repair phases (e.g., patch generation [296] and correctness assessment [293]), repair scenarios (e.g., static warnings [101] and syntax errors [102]), mode architectures (e.g., encoder-only [295] and decoder-only [159]) and model utilization paradigms (e.g., fine-tuning [281], few-shot [170] and zero-shot [296]). Despite ongoing explorations in the field, the literature currently lacks a detailed and systematic review of the applications of LLMs in APR, making it challenging for researchers to understand the multitudinous design choices of existing work and conduct follow-up research.

This Paper. To bridge this gap, our work provides the first systematic literature review on the deployment of rapidly emerging LLM-based APR studies. Based on this, the community can gain a comprehensive understanding of the strengths, weaknesses, and gaps in existing LLM-based APR techniques. We discuss what LLMs are widely adopted in state-of-the-art APR research and how they are integrated into the repair workflow. We collect 189 relevant papers and perform a systematic analysis from LLMs, APR, and integration perspectives. From our analysis, we reveal the current challenges and point out possible future directions for LLM-based APR research. Overall, this work offers a thorough overview of the ongoing progress within the LLM-based APR community, aiding researchers in navigating this burgeoning field and advancing toward innovative practices.

Contributions. To sum up, this work makes the following contributions:

- *Survey Methodology.* We conduct the first systematic literature review with 189 high-quality APR papers that utilize recent LLMs to address repair challenges from 2020 to September 2025.
- *Trend Analysis.* We perform a detailed analysis of selected APR studies in terms of publication trends, distribution of publication venues, and types of contributions.
- *LLMs Perspective.* We summarize 78 LLMs utilized to support program repair and provide a summary of the typical usage and trends of different LLM categories in the APR domain.
- *APR Perspective.* We describe common repair scenarios that LLMs are applied to, encompassing 20 bug types, such as security vulnerabilities and programming problems.
- *Integration Perspective.* We discuss some key factors, including datasets, input representations and open science, that impact the performance of integrating LLMs into APR.
- *Challenges and Opportunities.* We summarize some crucial challenges of applying LLMs in the APR field, and pinpoint some potential guidelines for future LLM-based APR research.

Paper Organization. Section 2 introduces some basic concepts about APR and LLMs, and Section 3 discusses the Then, according to the contributions listed above, Section 4 lists our research questions (RQs) and the research methodology to collect papers related to our work. Section 5 investigates the trend and distribution of LLM-based APR studies. Section 6 summarizes LLMs that are used by existing APR studies. Section 7 illustrates the primary repair scenarios that LLMs are applied to and provides a brief description of each work. Section 8 discusses some crucial factors during the integration of LLMs and APR, including datasets, input representation, patch correctness, and open science. Section 9 discusses some challenges and practical guidelines. Section 11 draws the conclusions.

2 Background

In this section, we introduce the core concepts relevant to this work, including APR and LLMs. Section 2.1 outlines the principles and evolution of APR, and Section 2.2 summarizes recent advances in LLM classifications and representative models.

2.1 Automated Program Repair

2.1.1 Problem Description. In the software development and maintenance process, developers usually attempt to implement a designed functionality according to requirement specifications and write test suites to validate its correctness. If all test cases pass, the functionality is deemed correctly implemented; otherwise, developers must analyze the symptoms of failures and make necessary modifications to the buggy code snippets to ensure all test cases pass. Formally, given a buggy program P and its corresponding specification S that P fails to satisfy, APR is defined to find a minimal transformation P' of P that satisfies S . In practice, test cases are usually utilized as the specification, and in such a typical test-driven repair scenario, APR aims to find a minimal program variant that passes the available test suite, which contains at least one test case that the original buggy program P fails to satisfy.

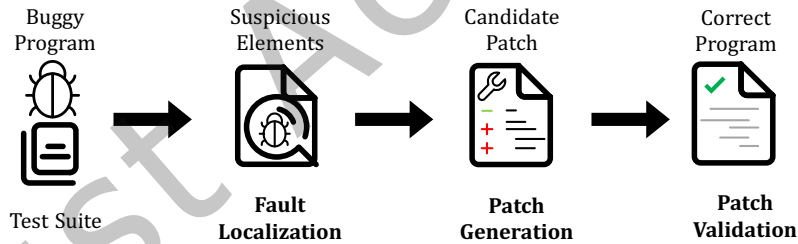


Fig. 1. The basic pipeline of automated program repair.

2.1.2 Repair Workflow. Fig. 1 illustrates the typical *generate-and-validate* workflow of APR, which is usually composed of three parts. Specifically, for a detected bug¹, (1) the **fault localization phase** identifies suspicious code elements that need to be fixed based on off-the-shelf localization techniques [246]; (2) the **patch generation phase** generates program variants (i.e., candidate patches) via transformation rules [292]; (3) the **patch validation phase** utilizes available test suites as the oracle to identify correct patches by dynamic execution [293]. It is important to note that a candidate patch that successfully passes the available test suite is termed a plausible patch. However, if such a plausible patch fails to generalize to additional test cases, it is considered an overfitting

¹In practice, APR is generally triggered after a bug has been detected through testing techniques, such as static analysis and fuzzing tools.

patch. Conversely, a plausible patch that remains effective across broader test cases is recognized as a correct patch, one that is semantically in alignment with patches written manually by developers.

2.1.3 Repair Techniques. The literature has introduced a multitude of APR techniques to generate correct patches from various perspectives. Such APR techniques can be categorized into four categories: (1) **heuristic-based APR** utilizes genetic programming to explore the search space of the correct patch, such as GenProg [120], Astor [157], ARJA [282], and SimFix [95]; (2) **constraint-based APR** usually focuses on the condition synthesis problem by treating program repair as a constraint-solving task, such as Nopol [264], Cardumen [158], ACS [261], and Dynamoth [45]; (3) **pattern-based APR** utilizes pre-defined repair templates that are usually hand-crafted by experts to transform buggy code snippets into correct ones, such as TBar [141], FixMiner [115] and Avatar [140]; (4) **learning-based APR** treats patch generation as a neural machine translation task with the advance of DL models, such as CURE [97], Recoder [318], DEAR [128] and SelfAPR [274].

Among the four types of APR techniques, learning-based APR has achieved remarkable performance by learning hidden bug-fixing patterns automatically from extensive source code databases [292]. Recently, inspired by the success of LLMs in NLP and SE tasks, researchers have increasingly been utilizing advanced LLMs to address software bugs [254, 255, 295, 296]. Compared with learning-based APR, such LLM-based APR techniques have demonstrated significantly better performance and have received growing attention in the community, which is the focus of our work.

2.2 Large Language Models

LLMs refer to advanced Artificial Intelligence (AI) models that undergo extensive pre-training on large text corpora to enhance their capabilities of natural language understanding, generation, and interpretation in a human-like manner [311]. In recent years, their training scope has been extended beyond natural languages to include programming languages, enabling LLMs to reason about and generate source code effectively. The literature has seen a variety of LLMs supporting NLP and SE research [203], which can be categorized into three main categories based on their model architectures: (1) **Encoder-only LLMs**, such as CodeBERT [54], GraphCodeBERT [71], train the encoder part of the Transformer to generate a fixed-dimensional bidirectional representation with Masked Language Modeling (MLM) and Next Sentence Prediction (NSP). MLM aims to predict the original tokens that have been randomly masked out, and NSP predicts whether two given sentences actually follow each other in a text. (2) **Decoder-only LLMs**, such as CodeGPT [147], train the decoder part of the Transformer to support auto-regressive tasks with Causal Language Modeling (CLM), which aims to predict new tokens in a sequence based on previous tokens. (3) **Encoder-decoder LLMs**, such as CodeT5 [238], train both encoder and decoder parts of the Transformer to support sequence-to-sequence generation tasks with denoising objectives. We will summarize existing LLMs and how they are leveraged to support program repair in Section 6.1.

3 Related Work

In this survey, we systematically collect, summarize, and analyze APR studies empowered with advanced LLMs. Although our primary focus is the integration of LLMs with APR, given its growing prominence, there exists some related studies that explore similar aspects of either LLMs or APR. We clarify the fundamental differences between our work and prior work below.

The first group of related surveys attempts to analyze the applications of LLM more generally in SE. Zhang et al. [294] provide a detailed summarization of the LLM-based SE research from two intuitive perspectives: LLMs and their applications in SE. They introduce representative work across 30 LLMs, 15 pre-training tasks, 16 downstream tasks, and 43 specific code-related tasks. Besides, Wang et al. [231] provide a review of LLMs in software testing; Fan et al. [49] discuss the achievements and challenges of LLMs in SE; Hou et al. [83] conduct a

systematic literature review on LLM4SE. However, these works target the whole software engineering/testing workflow rather than our specific program repair task. They only provide a bird’s-eye view of a limited number of LLM-based APR papers, while our work provides an in-depth analysis from various perspectives. Their work indicates that program repair is the most popular scenario in which LLMs are applied within software engineering. Given the complexity and significance of program repair, we are motivated to conduct a more in-depth review of APR achievements involving LLMs.

The second group of related surveys attempts to review the achievements of APR. Gazzola et al. [66] present a survey to organize repair studies, and Monperrus et al. [163] present a bibliography of behavioral and state repair studies. Both works collect publications up to 2017, hence there is no overlap with our research as the first LLM-based APR work is published in 2020. For completeness, we note that an earlier undergraduate thesis [154] also reviews LLMs for program repair; however, our study differs substantially in scope, methodological rigor, and analytical depth. The most highly related work to this paper is Zhang et al. [292], which provides a systematic survey to summarize the state-of-the-art in the learning-based APR community. However, they mainly focus on DL models and devote merely a single section to discussing LLM-based APR studies (only five papers), failing to include the surge of works that have emerged during the last two years. In contrast, our work solely focuses on the technology of LLM applications in the APR community until September 2025, involving the trend of utilized LLMs, optimization techniques, repair scenarios, and input forms.

4 Survey Methodology

In this section, guided primarily by the SEGRESS guidelines [108], we present details of our systematic literature review methodology.

4.1 Research Questions

We attempt to provide a comprehensive overview of recent LLMs’ applications to APR by summarizing the relevant studies and further providing guidelines on the follow-up research. Although the scope of this study is relatively broad, we adopt an SLR approach [109, 190] rather than a systematic mapping study, as our goal is to synthesize detailed evidence, identify methodological patterns, and derive analytical insights rather than merely classify or map existing research topics. To achieve this, this systematic literature review answers the following Research Questions (RQs):

- **RQ1 (Trend Analysis):** What is the trend of APR studies that utilize LLMs?
- **RQ2 (LLM Perspective):** Which popular LLMs have been applied to support APR?
- **RQ3 (APR Perspective):** What repair scenarios have been facilitated by LLMs?
- **RQ4 (Integration Perspective):** What key factors contribute to the integration of LLMs for APR?

4.2 Search Strategy

We search and identify primary papers following the “Quasi-Gold Standard” (QGS) [283] method, which is a common practice to construct a set of known studies for refining search strings by combining manual and automated search processes. Fig. 2 illustrates the general workflow of paper collection and section. We first conduct a manual search to identify a set of relevant studies and derive a search string from them, as detailed in Section 4.3. We then utilize the search string to perform an automated search and employ a series of relatively strict filtering steps to obtain the most relevant studies, as detailed in Section 4.4. We finally use a snowballing search to supplement the search results further, as detailed in Section 4.5. Given the abundance of relevant papers from different research communities, such as SE and AI, this search strategy allows us to capture the most pertinent papers while achieving greater efficiency than a purely manual search with a relatively rigorous process. Particularly, we undertake the following five phases to search for and identify relevant studies.

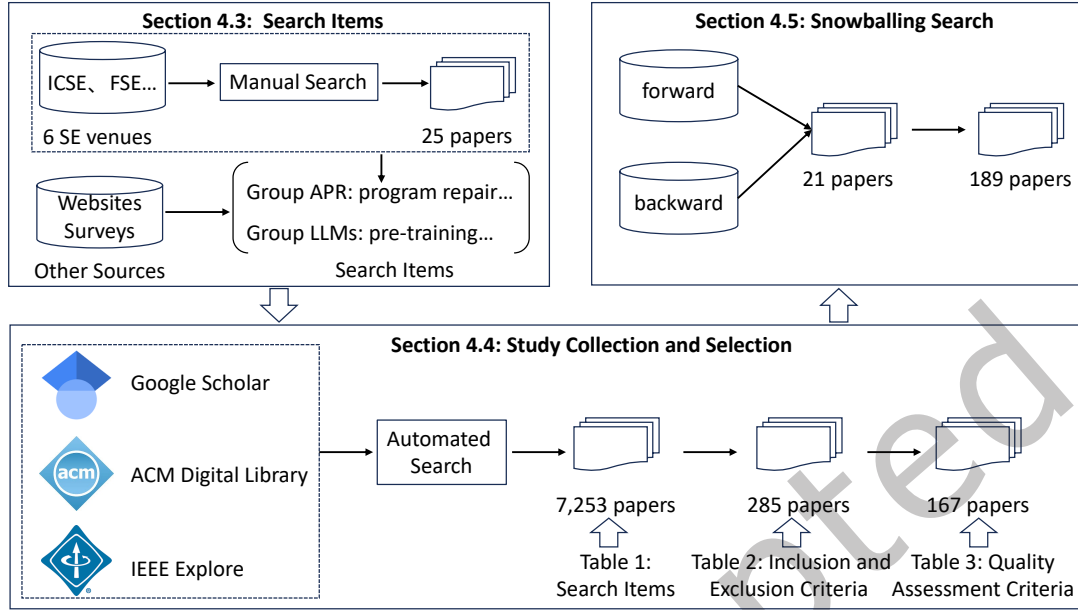


Fig. 2. Process of paper collection and selection.

- (1) **Search Sources.** This phase selects high-quality publication venues for the initial manual search and well-known digital databases for the subsequent automated search.
- (2) **QGS Establishment.** This phase inspects all papers identified in the manual search and filters them by inclusion/exclusion criteria.
- (3) **Search Items.** This phase defines search items with domain knowledge and establishes QGS.
- (4) **Paper Collection and Selection.** This phase conducts an automated search using the above search items and filters the collected papers by inclusion/exclusion criteria.
- (5) **Snowballing Search.** This phase conducts a snowballing search to complement the final search results.

4.3 Search Items

To perform the manual search and establish QGS, we select four top-tier SE conferences (ICSE, ESEC/FSE, ASE, ISSTA) and two journals (TOSEM and TSE) and search for relevant papers involving both program repair and LLMs. We manually inspect and identify 25 papers from 2020 to 2024 that meet our criteria, which then form the basis for constructing QGS. Following prior surveys [233, 272, 292], we divide the search items used for searching papers automatically into two groups: (1) an APR-related group containing some commonly-used keywords related to program repair; and (2) an LLM-related group containing some popular keywords related to large language models or pre-trained models. Besides, as the APR community benefits from well-maintained forums and real-time updated works, we further refine our search items by concluding some frequent keywords from three sources: a community-driven website², a living review of APR by Monperrus [164] and the most recent learning-based APR survey [292]. Finally, we identify a search string that includes several LLM-related

²<http://program-repair.org/bibliography.html>

Table 1. Search items used for retrieving LLM-based APR papers.

("program repair" OR "software repair" OR "automatic repair" OR "code repair" OR "bug repair" OR "bug fix" OR "code fix" OR "automatic fix" OR "patch generation" OR "patch correctness" OR "patch validation" OR "fix generation" OR "code transformation" OR "code edit" OR "fix error") **AND** ("LLM(s)" OR "Large Language Model(s)" OR "Pre-trained" OR "Pretrained" OR "Pre-training" OR "Pretraining" OR "PLM(s)" OR "(Code)BERT" OR "(Code)T5" OR "(Code)GPT" OR "Codex" OR "ChatGPT" OR "(Code)Llama" OR "GPT-*" OR "neural" OR "machine" OR "deep" OR "learning" OR "transformer/transformers" OR "model/models" OR "transfer" OR "supervised")

Table 2. Inclusion criteria and exclusion criteria.

Inclusion criteria	
①	The paper utilizes an LLM in its framework.
②	The paper addresses the task of program repair.
③	The paper is accessible with full text.
④	The paper is only written in English.
Exclusion criteria	
①	The paper is fewer than seven pages.
②	The paper is an old conference version extended to journals with the same authors.
③	The paper uses repair methods to contribute to LLMs.
④	The paper is published as an SLR, review, or survey only mentioning the development of LLMs and APR.
⑤	The paper is a duplicate publication, with the preprint and the published version presenting the same content under distinct titles.
⑥	The paper is published in a workshop or a doctoral symposium.
⑦	The paper is a grey publication, e.g., a technical report or thesis.
⑧	The paper falls into the category of short papers, tool demonstrations, and editorials.

and APR-related keywords frequently appearing in program repair studies that use LLMs. The complete set of search keywords is shown in Table 1.

It is noteworthy that we also include the list of keywords related to machine/deep learning from Zhang et al. [292], so as to avoid missing any papers related to our work as much as possible.

4.4 Study Collection and Selection

To perform the automated search, we collect potentially relevant papers by primarily searching the Google Scholar repository, ACM Digital Library, and IEEE Explorer Digital Library at the end of September 2025. Based on our search strategy, we obtain a total of 7,253 papers. Once we gather the studies based on our automated search strategy, we conduct a filtering phase followed by a quality assessment phase to eliminate papers that do not align with the objectives of this review.

4.4.1 Inclusion and Exclusion Criteria. The search conducted on the selected databases is intentionally broad to ensure the inclusion of as many potentially relevant studies as possible. However, this inclusiveness inevitably resulted in collecting papers outside the scope of our survey. Thus, to refine the selection, we defined a set of specific inclusion and exclusion criteria and applied them to all collected papers, ensuring that every retained study aligned with our research scope. Table 2 presents the designed inclusion and exclusion criteria. For example,

Table 3. Checklist of Quality Assessment Criteria (QAC) for LLM-based APR studies.

ID	Quality Assessment Criteria
QAC1	Is the LLM adopted as the research subject rather than only as a baseline method?
QAC2	Is the impact of the paper on the APR community clearly stated?
QAC3	Is the contribution of the paper clearly stated?
QAC4	Is the paper published in a high-repute venue?
QAC5	Is the relevant artifact made open source, e.g., datasets?
QAC6	Is there a clear motivation for the paper?
QAC7	Is the implementation of the proposed approach described clearly?
QAC8	Is the experimental setup described in detail, e.g., hyper-parameters and environments?
QAC9	Are the findings clearly supported by experimental results?
QAC10	Are the key contributions and limitations of the study discussed?

following Exclusion criterion ②, we filter out papers with fewer than seven pages [292]. Following Exclusion criterion ①, we retain only the extended journal version [90] when a study expanded a previously published conference paper. Following Exclusion criterion ⑤, we identify 21 papers whose published versions have different titles compared to their preprint. In such cases, to avoid duplication, we retain only the published versions. It is worth noting that, given the positioning of this study, we do not adopt a multivocal review[65] and thus exclude broader grey literature (e.g., technical blogs and theses). However, considering the rapid evolution of this field, we incorporate a limited number of *arXiv* papers that have passed our rigorous quality assessment, following recent LLM4SE surveys to ensure the timeliness of our review. During this process, two authors independently inspect the titles, abstracts, keywords, and venues of all remaining papers according to our inclusion/exclusion criteria in Table 2, reducing the number of candidates to 285. Any disagreements between two authors are resolved through discussion, and if a consensus cannot be reached, the third author makes the final decision.

4.4.2 Quality Assessment. To ensure the inclusion of high-quality papers, we conduct a full-text inspection of the remaining papers to assess their relevance and quality within the LLM-based APR field. This step is adopted to mitigate the impact of substantial quality variations observed in the collected studies, particularly among very recent and non-peer-reviewed papers, and is consistent with common practices in most recent LLM-based SE surveys [291, 294]. Following Wang et al. [233], we design ten quality assessment questions, where each paper is independently rated by two authors on a three-tier scale: “yes” (1 point), “partial” (0.5 points), or “no” (0 points). Papers scoring below the threshold of 8 points are excluded from further analysis, resulting in 167 papers being retained. Any disagreement is resolved through discussion, and when necessary, by consultation with the third author. The designed quality assessment criteria (QAC) are presented in Table 3. It is worth noting that, for QAC4, given the emerging nature of this research area and the rapid evolution of LLMs, many recent works have not yet completed the peer-review process. Therefore, we also consider papers from *arXiv* that satisfy our quality assessment criteria. Before inclusion, we carefully check whether each *arXiv* paper had been subsequently published by comparing its title, authorship, and content. If a published version is found, only the peer-reviewed version is retained; otherwise, the *arXiv* version is included and explicitly marked as such in our dataset. This strategy aligns with the common practice in recent LLM-based SE surveys, and ensures our work remains both comprehensive and up to date while maintaining a high standard of quality.

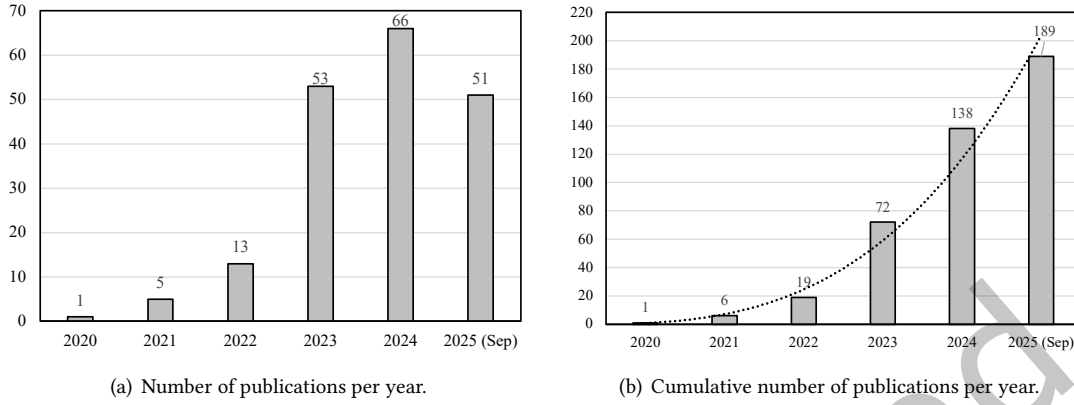


Fig. 3. Publication trends of LLM-based APR studies.

4.5 Snowballing Search

To ensure maximal comprehensiveness of the collected papers, we additionally employ the snowballing search [239] to manually incorporate any potential papers that are overlooked during our previous search process. The snowballing search is a common practice to examine the references (i.e., backward snowballing) or citations (i.e., forward snowballing) within a list of papers to discover additional relevant works. Specifically, we scrutinize every reference and citation within the collected papers to assess their relevance to our research objectives. This meticulous manual analysis of all potential papers through the entire study selection process ultimately leads to the inclusion of 189 papers in our survey. Through this process, we identified 21 additional studies, leading to the inclusion of 189 final papers in our survey. All included papers and their bibliographic information are publicly available in our replication repository for transparency and reproducibility [11].

5 RQ1: What is the trend of APR studies that utilize LLMs?

5.1 What are the trends of publication over time?

We analyze the publication trends of APR studies empowered with LLMs between 2020 and September 2025. Although the Transformer architecture [225] was proposed in 2017, and the groundbreaking LLM, BERT [37], was introduced in 2018, we do not find any studies using LLMs to fix bugs before 2020. Fig. 3(a) illustrates the number of relevant studies, and we find that the number of publications from 2020 to 2024 shows a significant increase, with the number reaching 66 papers in 2024. It should be noted that we collect papers as of September 2025; thus the number of relevant studies (i.e., 51 papers) in 2025 does not fully reflect the overall trend of LLM-based APR for the entire year. We also calculate the cumulative number of publications as shown in Fig. 3(b). To predict the publication trend in the last six years, we fit the cumulative number of publications as a power function, and find that the slope of the curve fitting the distribution increases substantially between 2020 and 2025. Particularly, the coefficient of determination attains the peak value ($R^2 = 0.98833$)³, estimating that there may be over 210 relevant LLM-based APR publications by the end of 2025. Overall, utilizing LLMs to address bug-fixing has

³The publication trend is fitted using a power function $y = a \times x^b$, where $a=0.8451$ and $b=3.0876$. It is worth noting that this trend analysis offers only an indicative projection. Given the rapidly evolving nature of LLM-based research, future publication trends may vary as the field continues to advance.

Table 4. Publication venues with LLM-based APR studies.

Acronym	Venues	Papers	Proportion
ICSE	International Conference on Software Engineering	26	12.94%
arxiv	N.A.	25	12.44%
FSE/ESEC	European Software Engineering Conference and International Symposium on Foundations of Software Engineering	17	8.46%
ASE	International Conference on Automated Software Engineering	13	6.47%
TOSEM	Transactions on Software Engineering Methodology	12	5.97%
ISSTA	International Symposium on Software Testing and Analysis	11	5.47%
ICLR	International Conference on Learning Representations	11	5.47%
TSE	Transactions on Software Engineering	9	4.48%
NeurIPS	Neural Information Processing Systems	7	3.48%
ACL	The Association for Computational Linguistics	6	2.99%
APR	International Automated Program Repair Workshop	6	2.99%
AAAI	AAAI Conference on Artificial Intelligence	3	1.49%
ICSME	The International Conference on Software Maintenance and Evolution	3	1.49%
MSR	International Conference on Mining Software Repositories	3	1.49%
USENIX Security	USENIX Security Symposium	3	1.49%
EMSE	Empirical Software Engineering	2	1.00%
ICST	IEEE International Conference on Software Testing, Verification and Validation	2	1.00%
OOPSLA	Conference on Object-Oriented Programming Systems, Languages, and Applications	2	1.00%
AUSE	Automated Software Engineering	1	0.50%
COMPSAC	International Computer Software and Applications Conference	1	0.50%
DAC	Design Automation Conference	1	0.50%
JSS	Journal of Systems and Software	1	0.50%
NAACL	North American Chapter of the Association for Computational Linguistics	1	0.50%
PLDI	ACM SIGPLAN Conference on Programming Language Design & Implementation	1	0.50%
PMLR	Proceedings of Machine Learning Research	1	0.50%
S&P	IEEE Symposium on Security and Privacy	1	0.50%
SLE	ACM SIGPLAN International Conference on Software Language Engineering	1	0.50%
TDSC	IEEE Transactions on Dependable and Secure Computing	1	0.50%
TIFS	IEEE Transactions on Information Forensics and Security	1	0.50%
TLT	IEEE Transactions on Learning Technologies	1	0.50%
TODAES	ACM Transactions on Design Automation of Electronic Systems	1	0.50%
WWW	Proceedings of the ACM Web Conference	1	0.50%
Others	N.A.	14	6.97%
Total	N.A.	189	94.03%

become a prevalent trend since 2020, and a large number of studies will adopt LLMs to address the challenges of APR in the future.

5.2 What is the distribution of the publication venues?

We analyze 189 studies published in various publication venues. Table 4 lists the number of relevant papers published in each publication venue. We find that 87.56% of papers are published in peer-reviewed venues, and ICSE is the most popular venues that contains 26 papers, followed by ESEC/FSE (17 papers), ASE (13 papers), TOSEM (12 papers), and ISSTA (11 papers). Besides, most top popular venues are top-tier conferences and symposiums, and only three journals have more than one relevant publication, i.e., TOSEM (12 papers), TSE (9 papers), EMSE (2 papers). This finding indicates a preference for presenting the latest work at conferences due to the timeliness of conference proceedings. Although LLM-based APR research primarily appears in top-tier SE venues (e.g., ICSE, FSE, ASE, TOSEM, ISSTA), the field has also gained attention from the broader AI (e.g., NeurIPS, AAAI, and ICLR), Programming Language (e.g., OOPSLA and PLDI), and Security (e.g., S&P, TDSC, and TIFS) communities. This interdisciplinary diffusion reflects the growing influence and popularity of LLM-based techniques in program repair. We also find that the remaining 12.44% of papers are non-peer-reviewed and published on arXiv. This phenomenon can likely be attributed to the rapid emergence of relevant studies in a

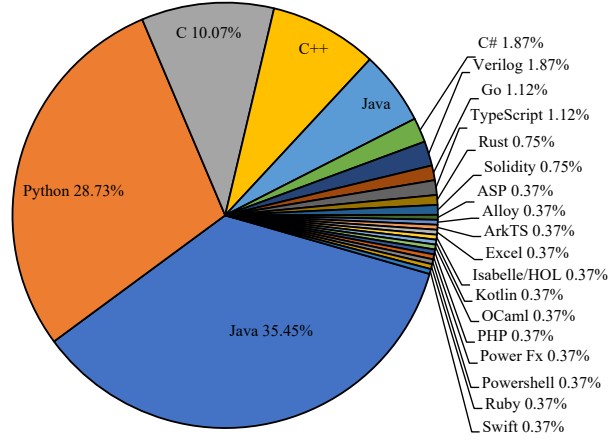


Fig. 4. Distribution of LLM-based studies across different programming languages.

short development time. Considering the varying quality levels of such non-peer-reviewed papers, we conducted a rigorous evaluation to ensure the inclusion of high-quality relevant papers in this work.

5.3 What is the distribution of the programming languages?

Fig. 4 presents the distribution of different programming languages targeted by existing LLM-based APR techniques. We find that Java is the most widely utilized programming language, accounting for 35.45% of all collected papers, followed by Python (28.73%) and C (10.07%). The possible reason lies in the availability of mature datasets for these languages, which serve as recognized benchmarks for evaluating repair techniques, such as Defects4J [103], QuixBugs [134], BFP [224], CVEfixes [16], Big-Vul [50], and TFix [15].

We also find that LLM-based APR encompasses a broader range of programming languages compared to traditional APR. For example, our collected papers involve 24 different programming languages in total, including both general-purpose languages (e.g., Java, Python, C, C++, JavaScript, Rust) and domain-specific languages (e.g., Verilog for hardware description and Solidity for smart contracts), whereas learning-based APR techniques [292] are typically limited to only five common languages (i.e., Java, JavaScript, Python, C, and C++). Importantly, we notice that some rare languages, previously overlooked in the APR community, are now being addressed. This broader language adaptability of LLM-based APR might stem from the inherent capabilities of LLMs to encapsulate general programming knowledge that can be transferred across multiple languages by fine-tuning [281]. In particular, recent LLMs are much larger and are trained on more diverse datasets, allowing them to incorporate knowledge from many different languages [290]. Besides, LLMs' robust natural language understanding capabilities facilitate the few-shot or zero-shot repair settings with limited learning samples, which is a significant advantage over DL models [292] that typically require extensive repair corpora for training. Consequently, LLMs can efficiently handle lesser-known programming languages, like Verilog [1] and Rust [35], which are often underrepresented in previous APR research [164]. This phenomenon highlights the promising prospects and scalability of APR brought about by recent LLMs.

5.4 What are the types of publication contributions?

We categorize collected papers according to their main contributions into four categories: new technique or methodology, empirical study, benchmark, and human study, as illustrated in Table 5. It is worth noting that,

Table 5. Categories of four main contributions in collected studies.

Category	Description	Papers
New Technique or Methodology	The study proposes a novel approach to address specific challenges in the APR community, empowered by LLMs.	118
Empirical Study	The study explores the performance of LLMs in fixing software bugs with a quantitative and qualitative analysis.	56
Benchmark	The paper introduces a new benchmark specifically designed to evaluate the fix capabilities of LLMs for various bug types	15
Human Study	The study conducts a survey to explore practitioners' perspectives or experiences of LLMs when fixing software bugs.	3

although most studies are assigned to a single category, a small number of studies make equally substantial contributions to multiple categories. We find that 118 relevant papers are published, with the aim of proposing a novel repair approach or framework with LLMs to address various issues in the APR community. Besides, 56 papers concentrate on conducting empirical studies to explore the actual benefits of LLMs in fixing various bugs, such as the potential of fine-tuning LLMs in vulnerability repair [67, 295]. We further notice 15 relevant studies constructing new benchmarks to evaluate the performance of LLMs and three papers [67, 160] administering a survey to offer insights into how practitioners or developers think about and employ LLMs to fix software bugs in practice.

Summary of Results for RQ1

(1) LLMs have shown a booming trend in fixing software bugs, with 189 papers between 2020 and 2025. (2) The number of conference papers employing LLMs for APR significantly exceeds that of journal papers, with ICSE and TOSEM being the most popular conference and journal venues, respectively. (3) LLM-based APR papers are published in different research fields, including SE, AI, and Security. (4) There are 24 programming languages that LLM-based APR has been applied to, with Java, Python, C, and C++ being the most frequently targeted. (5) LLMs have been applied to some underrepresented programming languages, such as Verilog and Rust. (6) The vast majority of collected studies primarily focus on introducing new techniques and conducting empirical research, while two papers perform user studies to understand practitioners' attitudes and experiences regarding leveraging various LLMs for solving bug-fixing tasks.

6 RQ2: Which popular LLMs have been applied to support APR?

6.1 What is the distribution of involved LLMs?

Plenty of LLMs, with millions to billions (even more) of parameters, have been proposed and adapted to automatically fix software bugs. We analyze all collected papers and identify 78 distinct LLMs that are already utilized in existing APR studies. Table 6 illustrates the variety of different LLMs and presents their usage counts across different years from 2020 to 2025. We only include the top 50 most frequently used models due to page limitations. It can be seen, GPT-3.5 (72)⁴ and GPT-4 (51) from OpenAI are the two most popular LLMs, followed by CodeT5 (31) and CodeLLaMA (28). We categorize these LLMs into three groups according to their architectures: encoder-only, encoder-decoder, and decoder-only LLMs.

⁴We note that several prior LLM4SE studies [83, 231] use the term ChatGPT to refer to the underlying GPT-3.5-Turbo model released in November 2022. Since ChatGPT is technically a service interface rather than a specific model, to avoid ambiguity, we unify the terminology in this paper and refer to it simply as GPT-3.5.

Table 6. Distribution of existing LLMs utilized in APR

Model	2020	2021	2022	2023	2024	2025	Total	Model	2020	2021	2022	2023	2024	2025	Total
GPT-3.5	0	0	1	24	32	15	72	LLaMA	0	0	0	1	2	0	3
GPT-4	0	0	0	9	27	15	51	PaLM	0	0	0	2	1	0	3
CodeT5	0	0	1	16	12	2	31	RoBERTa	0	0	1	1	0	1	3
CodeLLaMA	0	0	0	4	14	10	28	T5	0	1	1	1	0	0	3
Codex	0	0	2	12	6	2	22	TFix	0	0	1	1	1	0	3
GPT-4o	0	0	0	0	3	12	15	BART	0	1	1	0	0	0	2
StarCoder	0	0	0	0	12	3	15	Bard	0	0	0	0	2	0	2
CodeBERT	0	1	2	7	2	1	13	Claude	0	0	0	1	1	0	2
CodeGen	0	0	0	3	7	2	12	CodeGen-2	0	0	0	1	1	0	2
InCoder	0	0	0	7	4	0	11	CodeQwen1.5	0	0	0	0	1	1	2
PLBART	0	0	0	7	3	0	10	DeepSeekCoder-V2	0	0	0	0	1	1	2
GPT-4o-mini	0	0	0	0	0	9	9	Deepseek-v2.5	0	0	0	0	0	2	2
DeepseekCoder	0	0	0	0	4	4	8	Falcon	0	0	0	1	1	0	2
Claude-3.5	0	0	0	0	1	6	7	GPT-J	0	0	0	2	0	0	2
GraphCodeBERT	0	0	0	4	2	1	7	GPT-Neo	0	0	0	2	0	0	2
Llama-2	0	0	0	2	4	1	7	Gemini-1.5	0	0	0	0	1	1	2
PolyCoder	0	0	1	3	2	0	6	Gemma	0	0	0	0	0	2	2
UniXcoder	0	0	0	4	0	1	5	LLaMA-3.1	0	0	0	0	0	2	2
BERT	1	1	1	1	1	0	5	LLaMA-3.3	0	0	0	0	0	2	2
CodeGPT	0	0	2	1	1	0	4	Phi 3	0	0	0	0	0	2	2
LLaMA-3	0	0	0	0	1	3	4	Qwen-2.5	0	0	0	0	0	2	2
Mistral	0	0	0	0	1	3	4	Qwen2.5-Coder	0	0	0	0	0	2	2
Claude-3	0	0	0	0	1	2	3	SantaCoder	0	0	0	0	2	0	2
CodeGemma	0	0	0	0	1	2	3	StarCoder2	0	0	0	0	1	1	2
CodeT5+	0	0	0	1	2	0	3	WizardCoder	0	0	0	0	1	1	2

6.1.1 Encoder-only LLMs. Encoder-only LLMs denote a category of LLMs that only utilize the encoder stack of the Transformer architecture. Typically, these models are pre-trained on a massive corpus using a Masked Language Modeling (MLM) task, which is used to learn to predict the identity of masked words based on their context. In the APR community, prominent models like CodeBERT (11) [54] and GraphCodeBERT (6) [71] have been investigated in semantic bugs [88, 159, 255, 321] and security vulnerabilities [295]. As such LLMs only contain an encoder component that is capable of generating context-aware representations for inputs, they are particularly suited for code understanding tasks, such as code search, while not directly applicable to code generation tasks, such as program repair. Thus, in the APR community, as Zhang et al. [295] mention, researchers may need to integrate a new decoder that initializes from scratch to the pre-trained encoder to construct an encoder-decoder architecture for patch generation. We also notice such encoder-only LLMs are utilized to identify patch correctness [119, 215, 217, 219, 293, 317], as discussed in Section 8.3.

6.1.2 Encoder-decoder LLMs. Encoder-decoder LLMs denote a category of LLMs that utilize both the encoder and decoder stacks of the Transformer architecture, thus inherently suitable for transforming one sequence into another. Particularly, the encoder takes one sequence as input and encodes it into a fixed-size hidden state, which effectively captures the semantics and meaning of the input sequence. Then the decoder processes the hidden state and produces the corresponding output sequence using attention mechanisms to refer back to parts of the input sequence as needed. Thanks to the encoder-decoder architecture, such LLMs are particularly suited for code generation tasks in a sequence-to-sequence learning setting, such as program repair. In the APR community, a mass of encoder-decoder LLMs have been widely adopted, such as CodeT5 (23) [238], PLBART (10) [2], UniXcoder (4) [70] and T5 (3) [195]. Similar to traditional learning-based APR [292], such studies usually treat APR as a neural machine translation (NMT) task by supervised sequence-to-sequence learning, such as CIRCLE [281], TFix [15], VulRepair [59] and RAP-Gen [236].

6.1.3 Decoder-only LLMs. Decoder-only LLMs denote a category of LLMs that utilize only the decoder stack of the Transformer architecture. Decoder-only LLMs are typically pre-trained using a causal language modeling (CLM) objective, learning to predict the next word in a sentence based on the preceding words. These models are specifically designed for generating sequences of text autoregressively, i.e., producing one token at a time and using what has been generated so far as context for subsequent tokens. In the APR community, decoder-only LLMs are the most popular and widely used group, compared with encoder-only and encoder-decoder LLMs. Notable repair applications of decoder-only LLMs include a variety of open-sourced models (e.g., GPT-1 [193], GPT-2 [194], CodeGPT [147], GPT-Neo [18], GPT-NeoX [17], GPT-J [229], InCoder [57], CodeGen [173], CodeLLaMA [198] and StarCoder [126]), and closed-source models (e.g., GPT3 [22], GPT-3.5 [179], GPT-4 [180] and Codex [27]), categorized according to their accessibility. The emergence of decoder-only LLM-based APR studies is primarily due to three reasons. The first reason is that these models can naturally perform program repair from a few examples or simple instructions without any fine-tuning. The second reason is the recent surge in decoder-only LLMs, marked by the introduction of commercial products by leading Internet companies, such as GPT-3.5 and GPT-4 by OpenAI. The third reason is that decoder-only LLMs primarily rely on unsupervised learning during the pre-training phase, allowing them to exploit large-scale unlabeled text and code corpora for scalable and powerful language modeling, unlike encoder-based models that depend more on labeled data.

6.1.4 Trend Analysis. Table 6 reveals several clear trends in the evolution of LLM utilization for program repair from 2020 to 2025. First, there is a rapid shift toward large-scale decoder-only models, particularly those developed by OpenAI such as GPT-3.5 and GPT-4, which together account for over 120 studies. This dominance highlights the strong influence of general-purpose conversational models on APR research and reflects a growing interest in zero-shot and few-shot prompting. Second, the diversity of LLMs has expanded significantly since 2023, driven by the release of many open-source alternatives (e.g., CodeLLaMA, StarCoder, DeepSeekCoder). This trend indicates the research community's increasing emphasis on transparency, reproducibility, and cost-effective experimentation. Third, encoder-decoder models such as CodeT5, PLBART, and UniXcoder continue to play an important role, particularly in studies that rely on fine-tuning for task-specific adaptation. Although their overall proportion among newly proposed APR approaches has decreased in recent years, they remain foundational for sequence-to-sequence repair tasks requiring explicit supervision. Finally, the emergence of agent-aware and multimodal LLMs (e.g., GPT-4o and Claude-3.5) during 2024–2025 represents a new phase emphasizing interactive, repository-level, and tool-integrated APR. These models are being actively explored for autonomous reasoning and end-to-end repair, signaling a transition toward dynamic and environment-aware repair workflows.

Overall, the trend demonstrates a gradual evolution from specialized fine-tuned encoder-decoder architectures toward general-purpose and prompt-driven LLMs, reflecting the field's continuous pursuit of scalability, flexibility, and autonomy in APR.

6.2 What approaches are employed to optimize LLMs for program repair?

LLMs typically acquire general knowledge from extensive datasets. Thus, when integrating off-the-shelf LLMs into APR, a fundamental research issue arises: *how to adapt general-purpose LLMs to the specific program repair task*. Table 7 displays the prevalence of four common adaptation strategies in LLM-based APR research: fine-tuning, few-shot prompting, zero-shot prompting, and agent. Among them, fine-tuning (55 papers) and zero-shot prompting (54 papers) are the two most prevalent strategies, together accounting for more than two-thirds of all studies. Meanwhile, few-shot prompting is utilized in 24 papers, followed by agent in 15 papers.

6.2.1 Fine-tuning. Fine-tuning refers to a process where LLMs are further trained on a smaller, task-specific dataset. This is an intuitive way to allow LLMs to adjust their weights and biases through supervised learning, enabling them to perform as expected on new tasks that are similar to, but not exactly the same as, those they

Table 7. Breakdown of LLM utilization strategies for APR

Approach	Year	Count	Studies
Fine-tuning (55 papers)	2021	4	[15], [97], [43], [159]
	2022	8	[3], [59], [128], [63], [281], [117], [88], [107]
	2023	17	[58], [60], [90], [101], [248], [236], [295], [321], [165], [252], [81], [96], [113], [187], [305], [137], [302]
	2024	13	[31], [124], [138], [174], [201], [287], [316], [234], [68], [82], [133], [312], [192]
	2025	13	[20], [33], [151], [245], [61], [240], [53], [148], [153], [200], [269], [91], [206]
Few-shot (24 papers)	2023	8	[155], [93], [102], [170], [162], [48], [62], [254]
	2024	10	[52], [72], [77], [309], [135], [143], [285], [256], [257], [278]
	2025	6	[47], [139], [181], [175], [136], [286]
Zero-shot (54 papers)	2022	5	[255], [110], [191], [196], [208]
	2023	17	[44], [51], [55], [188], [222], [247], [313], [169], [296], [197], [114], [241], [146], [304], [218], [250], [300]
	2024	15	[1], [5], [185], [189], [212], [220], [223], [226], [268], [29], [314], [315], [199], [267], [266]
	2025	17	[35], [64], [92], [168], [25], [263], [251], [280], [289], [209], [24], [84], [111], [123], [183], [186], [104]
Agent (15 papers)	2024	5	[9], [26], [270], [306], [144]
	2025	10	[106], [7], [182], [213], [237], [279], [19], [121], [152], [277]

were initially trained on. In the APR community, fine-tuning is widely utilized during the early emergence of LLMs with millions of parameters, such as T5 and CodeT5, as it can significantly improve performance on the target program repair task without the need to train an LLM from scratch.

We summarize the existing APR studies via fine-tuning LLMs into three stages. First, researchers directly regard program repair as a downstream task by specific datasets, such as fine-tuning T5 [15], CodeT5 [59], CodeBERT [159], and GPT-2 [117]. Second, researchers utilize more advanced fine-tuning strategies for better performance. For example, CIRCLE utilizes continual learning to repair multiple languages with a single model, and RAP-Gen [236] utilizes retrieval-augmented generation to guide patch search space. Recently, Zirak et al. [321] empirically explore the domain shift problem in APR with two LLMs, i.e., TFix and CodeBERT, and three fine-tuning methods, i.e., Full-Fine-Tuning, Tuning-With-Light-Weight-Adapter-Layers, and Curriculum-Learning. Third, researchers conduct empirical studies to explore the actual fix capabilities of various LLMs in different repair scenarios. For example, Zhang et al. [295] fine-tune five LLMs to repair C/C++ security vulnerabilities, Wu et al. [248] involve four LLMs in Java vulnerabilities, and Jiang et al. [96] consider four LLMs on Java semantic bugs.

6.2.2 Few-shot Prompting. Few-shot prompting refers to the ability of LLMs to learn or adapt to new tasks with a very limited amount of data—often only a few examples. This is an effective way to use examples to help LLMs understand the targeted task and generate appropriate responses without any explicit retraining or fine-tuning. In the APR community, few-shot prompting is usually utilized and impressive in LLMs, particularly with billions of parameters, as it requires LLMs’ powerful ability to generalize from very limited data. Researchers typically provide LLMs with a small number of repair examples directly in the input prompt and require LLMs to generate correct patches. For example, Nashid et al. [170] construct effective prompts by retrieving similar repair demonstrations for CodeX, and Xia et al. [254] provide LLMs with examples from the same buggy project to learn the coding style.

6.2.3 Zero-shot Prompting. Zero-shot prompting takes the concept of few-shot prompting even further by requiring LLMs to perform program repair without any explicit examples. This is a recently popular way to query LLMs to perform a variety of unseen tasks, where LLMs are given a task description and must use their pre-existing knowledge and understanding to generate a response or solution. In the APR community, zero-shot rapidly emerges following the advent of LLMs with record-breaking parameters, exemplified by ChatGPT, as it requires a powerful foundation model to perform human-like chats.

There are two typical development routes that utilize LLMs for program repair in a zero-shot prompting setting. The first one is **cloze-style repair**, i.e., reframing program repair as a cloze-style task, and then invoking LLMs to predict partially correct code with the help of repair patterns, such as AlphaRepair [255], GAMMA [296], FitRepair [252] and Repilot [241]. The second one is **conversational-based repair**, i.e., constructing complex prompts with various valuable information (e.g., buggy code, failure diagnostics, even execution feedback), and then chatting with LLMs to generate correct patches, such as Pearce et al. [188], TypeFix [189], RustAssistant [35], Zhang et al. [300], Prenner et al. [191], Sobania et al. [208], and Napoli et al. [169]. Such repair routes usually require LLMs capable of processing long-text prompts and human-like conversations, thus predominantly employing powerful LLMs with billion-level parameters, like ChatGPT and GPT-4. Besides, zero-shot gets rid of training datasets, thus generalizing to various repair scenarios where gathering training data is challenging or impossible, such as hardware bugs [1], DL programs [24] and crash bugs [44].

6.2.4 Agent. An Agent refers to an autonomous system capable of perceiving its environment, reasoning about its current state, and taking a sequence of actions to achieve specific goals. Unlike prior prompting-based APR techniques, which hard-code the repair process into a static, pre-defined pipeline, LLM-based agents allow dynamic adaptation based on intermediate outcomes, contextual feedback. In the repair process, an LLM-based agent equips the underlying LLM with a set of external tools (e.g., compilers, debuggers, and test runners), enabling it to interact with and modify its environment. This flexibility allows LLMs to iteratively plan, execute, and refine repair actions, forming a closed-loop repair workflow. In the APR community, LLM-based agent research can be broadly divided into two directions.

The first direction focuses on enhancing the effectiveness of prompting-based APR methods on traditional benchmarks such as Defects4J and QuixBugs. Representative work includes RepairAgent [19], which is the first autonomous LLM-based repair agent. RepairAgent integrates ChatGPT with a state machine to manage iterative debugging actions, enabling it to autonomously plan repair steps, select tools, and refine patches until a correct fix is achieved. Similarly, FixAgent proposes a multi-agent framework for end-to-end unified debugging, which mimics the cognitive debugging process by structuring agent roles across three levels of complexity. Each level activates a subset of agents specialized in tasks such as summarization, slicing, localization, and patch generation.

The second category involves repository-level repair agents, which target real-world software issues, guided by the development of new benchmarks, such as SWE-bench [100]. Unlike traditional approaches [292] that often rely on predefined conditions (e.g., assuming known bug locations, failure-triggering tests, or single-function repair assumptions), these agents require end-to-end bug repair across full repositories, requiring reasoning across multiple files, dependencies, and test suites. Recent studies have proposed a series of SWE-oriented techniques to address the challenges of repository-level repair, including designing specialized agent-computer interfaces (e.g., SWE-Agent [270]), encoding the whole repository via AST representations (e.g., AutoCodeRover [306]), enabling multi-agent collaboration (e.g., MAGIS [213]), and developing generalist and specialist agents (e.g., OpenHands [237]). More recently, unlike most prior agent-based approaches that rely on offline-trained LLMs and fixed prompting strategies, the agent-based paradigm opens new opportunities for incorporating online learning into APR systems [167, 253], enabling agents to continuously refine their behavior during the repair process in evolving software projects.

6.2.5 Trend analysis. As shown in Table 7, the evolution of LLM-based program repair strategies reflects the continuous advancement of LLM capabilities and adaptation techniques. Fine-tuning, which uses supervised learning to adapt LLMs more closely to the specifics of program repair, becomes popular with the emergence of early million-parameter-level models like CodeT5. Few-shot prompting demonstrates LLMs' capability to generalize from a few examples, popular with the emergence of subsequent billion-parameter-level models like Codex. Zero-shot prompting showcases LLMs' capability to tackle the program repair task without any prior direct exposure to training or examples, particularly with the emergence of recent models with tens or hundreds of billions of parameters like ChatGPT and GPT-4. The recent introduction of agent-based APR represents a paradigm shift toward autonomous repair, where LLMs can plan, execute, and refine repair actions dynamically using tool-assisted feedback loops. Overall, these approaches illustrate a clear trend: as LLMs become more capable, adaptation moves from explicit training toward dynamic reasoning and autonomous interaction, broadening the applicability of LLMs in APR.

Summary of Results for RQ2

(1) We summarize 78 different LLMs already utilized to fix bugs, and these LLMs can be classified into three categories based on model architectures, i.e., encoder-only, encoder-decoder, and decoder-only. (2) Decoder-only LLMs are the most frequently utilized model architecture, and four of the top popular LLMs are decoder-only models. (3) ChatGPT, GPT-4, CodeT5, and CodeLLaMA are the most popular LLMs in existing LLM-based APR studies, utilized 72, 52, 31, and 28 times, respectively. (4) We summarize four typical ways of leveraging the vast knowledge encapsulated in LLMs for the specific program repair task, i.e., fine-tuning, few-shot, zero-shot, and agents.

7 RQ3: What repair scenarios have been facilitated by LLMs?

In this section, we conduct a comprehensive analysis of the utilization of LLMs in various repair scenarios. We categorize 20 existing repair scenarios addressed with LLMs, following the taxonomy from Monperrus et al. [164] and Zhang et al. [292], including semantic bugs, syntax bugs, and static warnings. Table 8 presents the distribution of repair studies across different repair scenarios to which LLMs are applied. We find that semantic bugs triggered by test suites have attracted the highest attention from the LLM-based APR research, constituting approximately 42.93% of the total research volume. This phenomenon is also observed in traditional learning-based APR [292], and is mainly driven by well-known benchmarks, such as Defects4J [103]. The second highest number of studies is observed in the programming problems domain, constituting approximately 12.57% of the total research volume. Security vulnerabilities account for about 10.99% of the research proportion, highlighting the significance of LLMs in addressing cybersecurity challenges. We also find a growing interest in rare bug types that are usually ignored by prior work, such as repository-level issues (9.42%), static warnings (4.71%), syntax errors (4.19%), and hardware bugs (2.62%). This underscores that, thanks to LLMs' general knowledge gleaned from vast amounts of data, researchers have begun to explore repair scenarios not previously addressed in prior works.

7.1 Semantic Bugs

Semantic bugs refer to logical errors in a syntactically correct program when the code does not do what the programmer intends, even though it may compile and run without syntax errors. Given that the majority of LLM-based APR studies focus on the semantic bug domain, we organize this rich body of work into two complementary parts to provide a clear, structured overview. To better capture their methodological diversity, Sections 7.1.1–7.1.3 summarize technical studies that propose new repair methods under fine-tuning, few-shot

Table 8. Distribution of LLM-based APR papers across different bug types

Bug Type	Year	Count	Studies
Semantic Bug	2020	1	[215]
	2021	4	[97] [219] [43] [159]
	2022	9	[128] [255] [281] [110] [117] [88] [191] [196] [208]
	2023	28	[90] [78] [102] [119] [247] [216] [236] [313] [321] [170] [165] [48] [252] [296] [62] [81]
	2024	18	[96] [114] [113] [187] [305] [241] [137] [146] [304] [218] [250] [254]
	2025	22	[29] [68] [80] [82] [133] [184] [317] [199] [205] [207] [256] [257] [265] [267] [278] [293]
Programming Problem	2023	4	[192] [266]
	2024	13	[19] [24] [46] [53] [84] [111] [112] [121] [123] [136] [148] [153] [183] [186] [200] [262]
	2025	7	[269] [277] [91] [286] [104] [206]
Security Vulnerability	2023	4	[51] [79] [145] [300]
	2024	13	[72] [94] [124] [138] [178] [212] [220] [228] [249] [268] [285] [310] [312]
	2025	7	[20] [33] [181] [245] [111] [121] [262]
	2022	1	[59]
Repository-level Issue	2023	7	[58] [188] [222] [90] [248] [247] [295]
	2024	7	[116] [118] [143] [174] [287] [316] [133]
	2025	6	[25] [61] [175] [209] [279] [91]
Static Warning	2024	7	[9] [26] [100] [270] [306] [144] [309]
	2025	11	[47] [73] [151] [168] [7] [182] [240] [251] [213] [237] [152]
Syntax Error	2021	1	[15]
	2022	1	[107]
	2023	5	[6] [101] [236] [321] [162]
	2024	2	[226] [227]
Hardware Bug	2022	1	[3]
	2023	4	[93] [90] [102] [313]
	2024	1	[199]
	2025	2	[35] [136]
Test Case	2023	1	[60]
	2024	4	[1] [142] [223] [273]
Type Error	2024	2	[52] [201]
	2025	2	[106] [263]
GUI Bug	2023	1	[197]
	2024	3	[31] [77] [189]
Code Review	2025	3	[92] [271] [280]
Performance Bug	2024	2	[74] [234]
Smart Contract	2022	1	[63]
	2025	1	[64]
API Misuse	2023	1	[169]
Crash Bug	2025	1	[289]
Error-handling Bug	2023	1	[302]
Formal Proof	2023	1	[44]
Formal Specification	2025	1	[139]
Motion Planner	2023	1	[55]
Translation Bug	2024	1	[5]
	2024	1	[135]
	2024	1	[185]

Table 9. A Summary of representative LLM-based technical studies on semantic bugs

Year	Technique	Venue	Paper Type	Model	Language
2021	CURE [97]	ICSE	GPT	Java	URL
2021	DeepDebug [43]	PLDI	BART	Java	N.A.
2022	DEAR [128]	ICSE	BERT	Java	URL
2022	AlphaRepair [255]	FSE/ESEC	CodeBERT	Java,Python	URL
2022	CIRCLE [281]	ISSTA	T5	Java,C,JS,Python	URL
2022	NSEdit [88]	ICLR	CodeBERT,CodeGPT	Java	N.A.
2023	APRFiT [78]	ICSME	GraphCodeBERT,CodeT5	Java	N.A.
2023	RING [102]	AAAI	Codex	C,JS,Python,Excel, Powershell,Power Fx	N.A.
2023	RAP-Gen [236]	FSE/ESEC	CodeT5	Java	URL
2023	CEDAR [170]	ICSE	Codex	JS	URL
2023	FitRepair [252]	ASE	CodeT5	Java	URL
2023	Gamma [296]	ASE	UniXcoder,CodeBERT,GPT-3.5	Java	URL
2023	Repilot [241]	FSE/ESEC	CodeT5,InCoder	Java	URL
2023	SarGaM [137]	TSE	PLBART,CoditT5,NatGen	Java	URL
2023	STEAM [304]	TOSEM	GPT-3.5	Java	N.A.
2024	T5APR [68]	JSS	CodeT5	Java,Python	URL
2024	CigaR [80]	arxiv	GPT-3.5	Java	URL
2024	Toggle [82]	FSE/ESEC	CodeT5,CodeGPT,CodeParrot, CodeGen,PolyCoder	Java	N.A.
2024	Mulpor [133]	ISSTA	CodeT5	Java	URL
2024	ThinkRepair [278]	ISSTA	CodeLLaMA,StarCoder, DeepseekCoder,GPT-3.5	Java,Python	URL
2024	ChatRepair [256]	ISSTA	GPT-3.5	Java, Python	N.A.
2025	RepairAgent [19]	ICSE	GPT-3.5	Java	N.A.
2025	Ehsani et al. [46]	ASE	GPT-4o-mini,Llama-3.3	Python	URL
2025	DEVLoRe [53]	TOSEM	GPT-4o-mini	Java	URL
2025	APRMCTS [84]	ASE	GPT-4o-mini,GPT-3.5	Java,Python	URL
2025	ContrastRepair [111]	TOSEM	GPT-3.5	Java	N.A.
2025	FixAgent [121]	arxiv	CodeLLaMA,Llama-2,DeepseekCoder, GPT-4o,Gemini-1.5,GPT-3.5,Claude-3.5	Java,Python,C	URL
2025	GIANTREPAIR [123]	TOSEM	StarCoder,CodeLLaMA,Llama-2,GPT-3.5	Java	URL
2025	HapRepair [136]	FSE/ESEC	GPT-4o	ArkTS	URL
2025	Ouyang et al. [183]	ICSE	GPT-3.5,GPT-4o-mini, DeepSeekCoder-V2,Codestral	Python	URL
2025	D4C [262]	ICSE	GPT-4,Mixtral	Java,C++,Python	URL
2025	MORepair [269]	TOSEM	CodeLLaMA	Java,C++	N.A.
2025	AdverIntent-Agent [277]	ISSTA	GPT-4o	Java	URL
2025	NTR [91]	ICSE	CodeT5,StarCoder,CodeLLaMA	Java	URL
2025	ReinFix [286]	ICSE	GPT-3.5,GPT-4o,GPT-4	Java	URL
2025	AutoSD [104]	EMSE	Codex,CodeGen,GPT-3.5	Java,Python	N.A.
2025	RepairLLaMA [206]	TSE	CodeLLaMA	Java	URL

prompting, or zero-shot prompting paradigms (as listed in Table 9), whereas Section 7.1.4 reviews empirical studies that focus on evaluating, benchmarking, and analyzing existing LLMs for APR (as listed in Table 10).

Table 10. A summary of representative LLM-based empirical studies on semantic bugs

Year	Technique	Venue	Model	Language	URL
2021	Mashhadi [159]	MSR	CodeBERT	Java	URL
2022	Kolak et al. [110]	ICLR	Codex, PolyCoder	Java, Python	N.A.
2023	Huang et al. [90]	ASE	CodeBERT, GraphCodeBERT, PLBART, CodeT5, UniXcoder	Java, C, C++, JS	URL
2023	Wu et al. [247]	arxiv	GPT-4, GPT-3.5	Java, C, Python	N.A.
2023	Zirak et al. [321]	TOSEM	CodeBERT, TFix	Java	URL
2023	Gao et al. [62]	ASE	Codex, GPT-3.5, text-davinci-003	Java	URL
2023	Jiang et al. [96]	ICSE	PLBART, CodeT5, InCoder, CodeGen	Java	URL
2023	Tian et al. [218]	arxiv	GPT-3.5	Python	URL
2023	Xia et al. [254]	ICSE	GPT-Neo, GPT-J, GPT-NeoX, Codex, CodeT5, InCoder	Java, C, Python	URL
2024	RTT [199]	TOSEM	PLBART, InCoder, GPT-4, TransCoder, SantaCoder, StarCoder, CodeT5, GPT-3.5	Java	URL
2024	MGDebugger [205]	arxiv	DeepSeekCoder-V2, CodeQwen1.5, GPT-4	Python	URL
2024	Prenner et al. [192]	ICSE	CodeT5	Java, Python	URL
2025	Cao et al. [24]	AUSE	GPT-3.5	Python	N.A.
2025	Kong et al. [112]	FSE/ESEC	GPT-3.5, CodeLLaMA	Java	URL
2025	Luo et al. [148]	TOSEM	CodeLLaMA, DeepseekCoder, WizardCoder, Mistral, CodeQwen1.5	Java	URL
2025	DSrepair [153]	ICSME	CodeT5, CodeGen, StarCoder, DeepseekCoder, BLOOM, CodeLlama-2	Java	URL
2025	MANIPLE [186]	ICSE	LLaMA3, GPT-3.5	Python	URL
2023	Defects4J-NI2fix [48]	ICSE	Codex, GPT-3.5	Java	N.A.

7.1.1 Repair with Fine-tuning. As early as 2021, Drain et al. [43] introduce DeepDebug, which learns to fix bugs in Java methods mined from real-world GitHub repositories through fine-tuning BART on the BFP [224] dataset. In 2022, Yuan et al. [281] propose CIRCLE, an LLM-based multi-lingual APR framework by fine-tuning a pre-trained T5 with continual learning. Particularly, CIRCLE utilizes a difficulty-based rehearsal strategy to achieve lifelong learning without access to all historical data and elastic regularization to resolve catastrophic forgetting. CIRCLE is the first approach to repair multiple programming language bugs with a single repair model in the continual learning setting. Hao et al. [76, 78] propose APRFiT by fine-tuning CodeT5 and GraphCodeBERT with a curricular learning framework. In 2023, RAP-Gen [236] further improves the repair performance of fine-tuning LLMs via a retrieval-augmented generation framework. RAP-Gen first utilizes a hybrid patch retriever to search for a relevant fix pattern from an external codebase, and utilizes Codet5 as the foundation model to synthesize candidate patches with the augmented inputs, i.e., the retrieved fix pattern and original buggy code. Recently, Silva [206] escalate the parameters of fine-tuned LLMs to the billion level, and propose RepairLLaMA based on CodeLlama-7B. RepairLLaMA utilizes a parameter-efficient fine-tuning technique called LoRA to train repair adapters with appropriate code representations.

7.1.2 Repair with Few-shot Prompting. In few-shot learning, LLMs take a prompt as input, which contains natural language instructions, a few examples of task demonstration, and a query, and generate an output. In

2023, Nashid et al. [170] propose a CEDAR, retrieval-based prompt selection approach for several code generation tasks, including program repair. CEDAR first automatically retrieves bug-fixing demonstrations relevant to the buggy code based on embedding or frequency similarity, and creates prompts to query Codex to generate correct patches in a few-shot learning manner. Meanwhile, Do et al. [41] conduct a preliminary study with ChatGPT and Galpaca-30b (a variant of LLaMA) by using a few-shot example generation pipeline.

7.1.3 Repair with Zero-shot Prompting. Unlike the above fine-tuning-based APR work, which relies on high-quality bug-fixing code pairs, some approaches are proposed to directly leverage off-the-shelf LLMs without any training. Such approaches attempt to generate patches in a zero-shot setting and can be categorized into two types, namely cloze-style APR and conversation-style APR.

Cloze-style APR leverages the pre-training objective (e.g., model language modeling) of LLMs to predict correct code tokens with the help of repair patterns. In 2022, Xia et al. [255] introduce AlphaRepair, a cloze-style APR tool to directly query LLMs to generate patches in a zero-shot learning setting. AlphaRepair first replaces the buggy line with a mask line and queries CodeBERT to fill the mask line to generate candidate patches. Furthermore, FitRepair [252] is a more advanced cloze-style APR approach based on CodeT5 and the plastic surgery hypothesis [13, 14]. FitRepair utilizes knowledge-intensified fine-tuning and repair-oriented fine-tuning to help CodeT5 learn the buggy project-specific knowledge and the cloze-style task knowledge, respectively. FitRepair then retrieves relevant identifiers with static analysis, which are fed into fine-tuned CodeT5 to generate candidate patches. Similarly, Repilot [241] improves the cloze-style APR AlphaRepair with a completion engine. Repilot builds an interaction between LLMs and a completion engine to generate more valid patches by first pruning away infeasible tokens suggested by LLMs and then completing the token based on the suggestions provided by the completion engine. GAMMA [296] further explores the potential of using LLMs to generate patches in a zero-shot learning scenario with a list of well-summarized fix templates. Particularly, GAMMA attempts to address the donor code retrieval issue of traditional template-based APR (e.g., TBar [141]) and regards patch generation as a fill-in-the-blank task by querying LLMs to predict the correct code for masked tokens in a pre-defined fix pattern. Unlike the cloze-style APR, Ribeiro et al. [196] frame the APR problem as a code completion task and apply CodeGPT to fix bugs from ManyStuBs4J [105].

Conversation-style APR leverages the powerful natural language and programming language understanding capabilities of LLMs to generate patches iteratively using test failure information. In 2023, Xia et al. [256] propose ChatRepair, the first conversation-driven APR approach that interleaves patch generation with instant feedback to perform APR in a dialogue manner. ChatRepair constructs prompts with test failure information and queries ChatGPT to generate correct patches from previously incorrect and plausible patches. However, ChatRepair mainly relies on negative feedback (i.e., failure information derived from failing tests) to guide the conversations, which may not always offer specific and adequate prompts for an effective repair. Thus, Kong et al. [111] introduce ContrastRepair, which includes positive feedback from passing tests to supplement the negative feedback. Given a buggy program and a failing test case, ContrastRepair generates a similar passing test case by making minimal modifications to the failing test case. ContrastRepair then constructs a contrastive pair to LLMs, allowing them to better pinpoint the root cause of the bug and generate accurate patches. In the above conversation-style APR scenarios, LLMs take a prompt containing some tokens about buggy code as the input and infer the following tokens about patches as the output. During the conversations, all tokens in the input prompt and output answer consume the computational and financial costs, such as \$0.06 per 1k input tokens and \$0.03 per 1k generated tokens for GPT-4. To reduce the computational cost of ChatRepair, Hidvegi et al. [80] propose CigaR, a token-efficiency LLM-based APR approach that concentrates on token cost minimization of ChatGPT. CigaR designs three prompts to help ChatGPT minimize the overall token cost with previous responses, including (1) an initiation prompt to initialize the repair process, (2) an improvement prompt to refine partial patches, avoiding discarding potentially valuable patches, and (3) a multiplication prompt builds upon the already

generated plausible patches to synthesize more plausible patches with diversity maximization. Unlike previous work relying on a fixed prompt template, RepairAgent [19] treats ChatGPT as an agent capable of autonomously planning and executing actions to generate patches by using dynamic prompts and a state machine to select suitable tools.

7.1.4 Empirical Study. In addition to the above novel techniques, researchers have conducted numerous empirical studies to explore the capabilities of LLMs in fixing semantic bugs. As early as 2021, Mashhadi et al. [159] preliminarily evaluate the performance of fine-tuning CodeBERT for fixing Java bugs from ManySStuBs4J. Lajko et al. [117] empirically fine-tune GPT-2 to automatically generate candidate patches for JavaScript bugs. Prenner et al. [191] investigate CodeX in fixing QuixBugs with a zero-shot setting and Sobania et al. [208] utilizes a more powerful LLM ChatGPT with prompt engineering to generate patches for QuixBugs. In 2023, Horvath et al. [81] explore the impact of model architectures and program representations, involving two popular programming languages, i.e., Java and JavaScript, three different code representations, i.e., raw text, command sequences, and ASTs, and four LLMs, i.e., T5, CodeT5, RoBERTa, and GPTNeo. Meanwhile, Zhao et al. [313] conduct a comprehensive investigation into the effective utilization of LLMs for repairing code review bugs, involving two LLMs (i.e., ChatGPT and GPT-4) in a zero-shot learning manner, and six LLMs (InCoder, CodeT5+, CodeFuse, LLaMA, CodeGen-2, CodeLLaMA) in a fine-tuning manner. Recently, inspired by the phenomenon that grammatical errors in natural language can be fixed by round-trip translation, i.e., translating sentences to another intermediate language and then back, Ruiz et al. [199] investigate to what extent the RTT pipeline can fix bugs in code in a zero-shot fashion. The comprehensive study involves eight LLMs: PLBART, CodeT5, TransCoder, SantaCoder, InCoder, StarCoderBase, GPT-3.5, and GPT-4, and four APR benchmarks: Defects4J-v1.2, Defects4J-v2.0, QuixBugs, and HumanEval-Java.

Furthermore, more comprehensive empirical studies are published in top-tier SE conferences. Xia et al. [254] conduct an comprehensive study to explore the performance of LLMs in program repair, involving nine LLMs from two categories (i.e., infilling and generative models), and five datasets across three programming languages. They explore three ways to use LLMs for patch generation: complete function generation, correct code infilling, and single-line generation. Meanwhile, Jiang et al. [96] empirically explore the fixing capabilities of ten variants from four LLMs under zero-shot and fine-tuning settings, involving four Java benchmarks. They also construct a new benchmark, HumanEval-Java, which none of the LLMs has seen during training to address the data leakage issue. Huang et al. [90] conduct an empirical study on fixing capabilities of LLMs in the fine-tuning paradigm, involving five LLMs, three programming languages, and three repair scenarios.

7.2 Programming Problems

This type of bug refers to incorrect solutions to programming problems from competition platforms, such as LeetCode. In 2022, Zhang et al. [284] propose MMAPR to repair both syntactic and semantic mistakes from introductory Python programming through Codex and few-shot learning. Fan et al. [51] systematically evaluate whether repair techniques can fix the incorrect solutions produced by LLMs in LeetCode contests. Similarly, Liu et al. [145] analyze the correctness of ChatGPT-generated code in Java and Python for 2,033 programming tasks from LeetCode and query ChatGPT to mitigate incorrect code with different prompts. Recently, Zhang et al. [300] empirically evaluate the performance of ChatGPT in fixing incorrect submissions from programming problems. Similar studies are conducted by Haque et al. [79] and Tian et al. [220]. Ishizue et al. [94] combine two LLMs (i.e., ChatGPT and GPT-4) with Refactory [87] to fix programming assignment problems. They leverage LLMs to repair programs with syntax errors and those that Refactory fails to repair. Recently, Zhao et al. [310] explore how LLMs, including ChatGPT, StarCoder, and CodeLlama, perform in repairing student assignments from higher-level programming courses. They construct a student assignment dataset named Defects4DS, which

Table 11. A summary of representative LLM-based studies on security vulnerabilities

Year	Technique	Venue	Paper Type	Model	Language	Link
2022	VulRepair [59]	FSE/ESEC	Approach	CodeT5	C	URL
2023	VQM [58]	TOSEM	Approach	CodeT5	C,C++	URL
2023	Huang et al. [90]	ASE	Empirical	CodeBERT, GraphCodeBERT, PLBART, CodeT5, UniXcoder	Java,C,C++,JS	URL
2023	Pearce et al. [188]	S&P	Empirical	Codex, PolyCoder, Jurassic-1	C,C++	URL
2023	ZeroLeak [222]	ESORICS	Empirical	GPT-4, PaLM, Llama-2, GPT-3.5	C,JS	N.A.
2023	Wu et al. [248]	ISSTA	Empirical	CodeT5, PLBART, InCoder, CodeGen, Codex	Java	URL
2023	Wu et al. [247]	arxiv	Empirical	GPT-4, GPT-3.5	Java,C,Python	N.A.
2023	Zhang et al. [295]	TDSC	Empirical	CodeT5, CodeBERT, UniXcoder, GraphCodeBERT, CodeGPT	C	URL
2024	VRpilot [116]	AIware	Approach	GPT-3.5	C,Java	URL
2024	Le et al. [118]	WWW	Empirical	GPT-4, Bard	JS	URL
2024	Mulpor [133]	ISSTA	Approach	CodeT5	Java	URL
2024	Liu et al. [143]	Security	Empirical	GPT-3.5, GPT-4	C,C++	URL
2024	Nong et al. [174]	arxiv	Approach	Llama-2, Falcon, GPT-3.5	C,C++	N.A.
2024	VulAdvisor [287]	ASE	Approach	CodeT5, Codex, GPT-3.5	C,C++	URL
2024	VulMaster [316]	ICSE	Approach	CodeT5	C,C++	URL
2025	ESBMC-AI [25]	AST	Approach	GPT-3.5	C,C++	N.A.
2025	VulnTrace [61]	FSE/ESEC	Approach	UniXcoder, GraphCodeBERT, CodeBERT, RoBERTa	C,C++	URL
2025	NTR [91]	ICSE	Approach	CodeT5, StarCoder, CodeLLaMA	Java	URL
2025	APPATCH [175]	Security	Approach	GPT-4, Gemini-1.5, Claude-3.5, LLaMA-3.1	C,C++	URL
2025	Steenhoek et al. [209]	ICSE	Human Study	GPT-4	Java	URL
2025	PATCHAGENT [279]	Security	Approach	GPT-4, GPT-4o, Claude-3	C,C++	URL

contains 682 submissions from 4 programming assignments, and introduce a repair framework named PaR, which generates patches by selecting peer solutions to create prompts.

7.3 Security Vulnerabilities

A software vulnerability refers to a flaw, bug, or weakness in the design, implementation, operation, or management of software. When exploited, vulnerabilities can lead to unauthorized access, manipulation, or disruption of the software's intended functionality. Table 11 summarizes some representative LLM-based APR studies on security vulnerabilities. Particularly, in 2022, Fu et al. [59] propose VulRepair, an LLM-based automated program repair approach by fine-tuning CodeT5 with vulnerability datasets. Furthermore, in 2023, considering that a vulnerable function only contains a few core elements that need to be fixed, Fu et al. [58] propose VQM, an LLM-based automated vulnerability repair approach that leverages VIT-based approaches for object detection to help CodeT5 focus more on vulnerable code areas during patch generation. Inspired by the relationship between semantic bugs and security vulnerabilities, Zhang et al. [295] propose an enhanced LLM-based vulnerability framework based on transfer learning, demonstrating superior performance against VulRepair. In 2024, Zhou et al. [316] propose VulMaster, a CodeT5-based approach to address several limitations of VulRepair, i.e., (1) handling lengthy vulnerable code by the Fusion-in-Decoder framework; (2) capturing the structure of the vulnerable code by AST; and (3) understanding the expert vulnerability knowledge by the CWE system. De et al. [34] leverage Quantized Low-Rank Adaptation (QLoRA) [129] to fine-tune two advanced 7-billion-parameter LLMs, i.e., CodeLlama [198] and Mistral [161], to fix C vulnerabilities.

Table 12. A Summary of representative LLM-based studies on repository-level issues.

Year	Tool name	Venue	Type	Model	Language	Link
2024	MASAI [9]	NeurIPS	Approach	GPT-4o	Python	N.A.
2024	CodeR [26]	arxiv	Approach	GPT-4	Python	URL
2024	SWE-bench [100]	ICLR	Benchmark	CodeLLaMA	Python	URL
2024	SWE-agent [270]	NeurIPS	Approach	GPT-4, Claude-3	Python	URL
2024	AutoCodeRover [306]	ISSTA	Approach	GPT-4	Python	URL
2024	MarsCode [144]	arxiv	Approach	GPT-4, Claude-2	Python	N.A.
2024	DRCodePilot [309]	ASE	Approach	GPT-4	Java	URL
2025	Ehsani et al. [47]	MSR	Approach	GPT-3.5	Python	URL
2025	OmniGIRL [73]	ISSTA	Benchmark	GPT-4o, Claude-3.5, Deepseek-v2.5	Python, JS, TS, Java	URL
2025	SWE-GPT [151]	ISSTA	Approach	Qwen-2.5, Qwen2.5-Coder	Python	URL
2025	SWT-Bench [168]	NeurIPS	Benchmark	GPT-4, GPT-4o-mini, Claude-3, Claude-3.5, Mistral	Python	URL
2025	Moatless [7]	ICLR	Approach	GPT-4o, GPT-4o-mini, Qwen-2.5, Deepseek-v2.5, LLaMA 3.1	Python	URL
2025	RepoGraph [182]	ICLR	Approach	GPT-4o, GPT-4	Python	URL
2025	SWE-RL [240]	NeurIPS	Approach	Llama-3.3	Python	URL
2025	Agentless [251]	FSE/ESEC	Approach	GPT-4o	Python	URL
2025	MAGIS [213]	NeurIPS	Approach	GPT-4	Python	N.A.
2025	SWE-bench M [270]	ICLR	Benchmark	GPT-4o-mini, Claude-3.5	JS, TS	URL
2025	OpenHands [237]	ICLR	Approach	GPT-4o, GPT-4o-mini, Claude-3.5	Python	URL

Empirical Study. Pearce et al. [188] evaluate the performance of LLMs in repair vulnerabilities in a zero-shot manner, involving five commercially available black-box and two open-source LLMs. They particularly investigate the potential of designing prompts that query LLMs to patch vulnerable code. Furthermore, Zhang et al. [295] conduct an empirical study investigating the performance of fine-tuning LLMs for vulnerability repair, involving five LLMs from three categories, two C vulnerability datasets and more than 100 trained LLMs, which is the largest set to date. They also explore the potential of ChatGPT in repairing security vulnerabilities in a zero-shot manner. Similarly, Wu et al. [248] explore the fixing capabilities of five LLMs in a zero-shot manner and four LLMs in a fine-tuning manner on two real-world Java vulnerability datasets. Recently, Le et al. [118] conduct a preliminary study of ChatGPT and Bard in detecting and fixing security vulnerabilities in JavaScript programs.

7.4 Repository-level Issues

Repository-level issues refer to real-world issues (such as bug reports or feature requests) in large-scale repositories. Recent LLM-based APR research increasingly targets repository-level issues, whose resolution requires long-context understanding over a large codebase, accurate fault localization from informal reports, dependency, and code edits across multiple modules, and robust validation of the resulting pull request. Table 12 summarizes some representative LLM-based APR studies on repository-level issues. A prominent benchmark for this scenario is SWE-bench [100], which comprises 2294 real GitHub issues with paired ground-truth pull requests from 12 popular Python repositories. In addition, several other datasets, such as SWT-bench [168], OmniGIRL [73], and SWE-bench M [271], are used to evaluate LLMs under varying APR settings. To address repository-level repair challenges, researchers have explored two main categories of approaches: procedural [182, 251] and agentic [7, 26, 237, 306]. In particular, similar to few-shot or zero-shot prompting approaches, procedural approaches [182, 251] rely on expert-designed workflows in which LLMs run within a fixed pipeline, with every choice hard-coded. In contrast,

agentic approaches [7, 26, 237, 306] query LLMs to autonomously plan and execute each choice by equipping LLMs with various external tools.

7.5 Static Warnings

Static warnings refer to automated alerts generated by static tools that examine code to identify potential errors, inefficiencies, or security vulnerabilities without executing the program. In 2021, Berabi et al. [15] introduce TFix, which is considered a pioneer in incorporating LLMs into APR. Inspired by T5, TFix formulates the program repair problem as a text-to-text task on code sequence, i.e., given a coding error as text, predicting a new text as the patch. TFix is particularly fine-tuned from T5 with a high-quality dataset consisting of 100k bug-fixing pairs across 52 error types detected by a static analyzer, ESLint. Similar to RAP-Gen [236], InferFix [101] employs a retrieval-augmented generation manner to fine-tune LLMs for program repair, i.e., (1) utilizing a retriever to search for semantically equivalent bugs and corresponding fixes from an external codebase, and (2) fine-tuning LLMs on supervised bug-fixing datasets with prompts augmented via adding retrieved similar fixes. However, RAP-Gen targets semantic bugs while InferFix repairs static errors detected by the static analyzer Infer; and RAP-Gen utilizes CodeT5 while InferFix has a significantly larger model Codex with 12 billion parameters. Recently, Wadhwa et al. [226] present CORE to resolve code quality issues flagged by static analysis tools. CORE utilizes GPT-3.5 as the proposer LLM to generate candidate code revisions and GPT-4 as the ranker to rank the candidate revisions before presenting them to the developer. Alrashedy et al. [6] introduce FDSP to resolve repair security vulnerabilities detected by a static code analysis tool, Bandit [12], in a self-debugging manner.

In addition to the above technical papers, Kim et al. [107] empirically investigate the performance of TFix in fixing errors from industrial Samsung Kotlin projects detected by a static analysis tool SonarQube. Mohajer et al. [162] conduct a more comprehensive study of LLMs in the static code analysis domain, and propose SkipAnalyzer, an LLM-based powered tool to perform three related tasks: detecting bugs, filtering false positive warnings, and patching the detected bugs.

7.6 Syntax Errors

Syntax errors refer to parsing mistakes that occur when code does not follow the rules or grammar of the programming language, such as invalid statements and expressions. These errors are detected by the compiler or interpreter during the parsing phase before the program is executed. Ahmed et al. [3] propose SYNSHINE to fix syntax errors by pre-training and fine-tuning RoBERTa with compiler diagnostics. RustAssistant [35] utilizes GPT-3.5 and GPT-4 to fix Rust compilation errors with prompt engineering. RING [102] is a multilingual syntax error repair approach powered by Codex and few-shot learning and is proven promising in six different languages. PCR-Chain [93] attempts to resolve fully-qualified names and fix last-mile syntax errors based on ChatGPT and Chain-of-Thought Prompting.

7.7 Hardware Bugs

APR is well-explored in the software domain, while the research is less mature for hardware description languages, such as Verilog. Ahmad et al. [1] empirically explore the potential of LLMs to repair security-relevant bugs in hardware designs. They leverage prompt engineering to query Codex, ChatGPT, GPT-4, and CodeGen for 15 security-related bugs from 10 CWEs. Inspired by the retrieval-augmented generation, Tsai et al. [223] introduce RTLFixer, a framework to fix Verilog syntax errors with LLMs and iterative prompting. RTLFixer formulates an input prompt to query GPT-4 to generate the correct code. RTLFixer then utilizes error logs from the compiler and human guidance from a retrieval database as feedback to perform an interactive debugging loop until all errors are resolved. Yao et al. [273] focus on the domain of chip design and propose HDLdebugger, an LLM-assisted hardware description language debugging framework. This framework encompasses data generation

through reverse engineering, a search engine to enhance the retrieval-augmented generation, and a fine-tuning approach to train retrieval-augmented LLMs. Similarly, Fu et al. [60] focus on the hardware design iteration process and introduce LLM4SECHW, an LLM-based hardware debugging framework. LLM4SECHW constructs a hardware debugging-oriented dataset from open-source hardware projects and fine-tunes a suite of hardware domain-specific LLMs capable of automatically reading hardware designs and fixing bugs.

7.8 Test Repair

Test repair focuses on fixing broken or outdated test cases to ensure they remain valid and meaningful throughout software evolution, whereas traditional APR studies modify production code to pass existing tests. Recent research has increasingly explored the potential of LLMs to automate test repair across diverse scenarios, including outdated tests, flaky tests, non-idempotent outcomes, and UI tests. For example, to address outdated test cases, Yaraghi et al. [201] introduce TARGET, an LLM-based framework that treats test repair as a language translation task. They construct TARBENCH, a comprehensive benchmark containing 45,373 broken test repairs across 59 open-source projects, and fine-tune off-the-shelf LLMs (i.e., PLBART, CodeGen, and CodeT5+) to generate correct test code with essential code context. Fatima et al. [52] introduce FlakyFix, a framework that leverages LLMs to predict fix categories for flaky tests and synthesize corresponding repairs. Ke et al. [106] propose NIODebugger, an agentic LLM-based framework for repairing non-idempotent-outcome tests that exhibit inconsistent results across repeated executions. NIODebugger adopts a three-stage agentic loop: (1) analyzing the root cause of test non-determinism (e.g., environment or concurrency issues), (2) generating repair hypotheses and corresponding patches, and (3) verifying them through re-execution and iterative feedback. Focusing on test script repair, Xu et al. [263] conduct the first feasibility study that integrates traditional heuristics with ChatGPT to repair Web UI tests (e.g., Selenium tests). They first use traditional repair approaches to generate a preliminary list of candidate-matched elements and then use ChatGPT to perform global matching to select the best-matched element.

7.9 Type Errors

Type errors occur when an operation or function is applied to an object of an inappropriate type. This indicates that the operation expects a certain data type but receives another, thus leading to unhandled exceptions that cause program crashes. These errors are prevalent in dynamically typed languages such as Python and JavaScript, which do not allow operations that are inherently impossible with the given data types. In 2024, Chow et al. [31] construct PyTyDefects, a dataset consisting of 2,766 type error-fix pairs from 176 GitHub repositories. They also introduce PyTy, a T5-based repair technique that fine-tunes the off-the-shelf LLM TFix with PyTyDefects for fixing type errors in Python. Unlike the fine-tuning-based approach PyTy, TypeFix [189] is a prompt-based repair approach for Python type errors in a zero-shot manner. Similar to GAMMA, TypeFix queries LLMs to generate patches by filling the masks in code prompts using fix templates. However, TypeFix distinguishes itself by automatically mining these templates through a hierarchical clustering algorithm, rather than relying on predefined ones in GAMMA. Although type errors are more prevalent in dynamically typed languages, they can also arise in statically typed languages (e.g., OCaml) due to incorrect type inference, mismatched annotations, and violations of declared type constraints during compilation. For example, Ribeiro et al. [197] present Mentat, a type error repair technique for OCaml programs powered by GPT-3. Mentat first analyzes the source code to generate contextually relevant prompts, then exploits GPT-3's advanced language understanding and generation capabilities to produce potential patches.

7.10 GUI Bugs

GUI (Graphical User Interface) bugs refer to defects in the visual or interactive layer of software systems, including layout misalignment, incorrect rendering of UI components, inconsistent visual states, and broken interactions under different environments. Compared with traditional APR, which focuses on source code-level bugs, GUI bugs involve multimodal contexts, including source code, visual layouts, and user interactions, which demand perception of visual cues and reasoning across the image-code domain. Building upon SWE-bench [99], Yang et al. [271] introduce SWE-bench M, an extension targeting visual and GUI-related issues in JavaScript projects. SWE-bench M comprises 617 visual tasks from 17 open-source repositories, each associated with real-world issues containing screenshots or rendered UI components. Huang et al. [92] propose GUIRepair, a cross-modal reasoning framework that incorporates visual context into GUI bug fixing for SWE-bench M. GUIRepair consists of two core modules: (1) Image2Code, which localizes the source code regions responsible for visual symptoms based on UI screenshots, and (2) Code2Image, which re-renders candidate patches to verify whether the visual defects are eliminated. Unlike GUIRepair, which focuses on general GUI functionality and rendering issues, Yuan et al. [280] present DesignRepair, a dual-stream framework for repairing UI design-quality issues. DesignRepair leverages both a low-level component representation and a high-level design-guideline knowledge base, integrating retrieval-augmented LLMs to correct design inconsistencies such as misalignment, color misuse, and accessibility violations.

7.11 Code Review

Code review aims to improve code quality and maintainability by reviewing submitted changes and providing actionable comments. Guo et al. [74] conduct the first empirical study investigating the potential of ChatGPT for automated code refinement based on existing review comments. They leverage prompt engineering to compare ChatGPT with CodeReviewer [130] using two datasets: the established CodeReview [130] dataset and a newly curated CodeReview-New dataset. To further improve ChatGPT's performance, they design several enhancement strategies, including leveraging more advanced models and optimized prompting. Furthermore, Wang et al. [234] propose CodeReviser, a two-phase "localization-and-revision" framework that follows the divide-and-conquer principle for automated code revision. Instead of asking a single model to both find and fix issues, CodeReviser first localizes the code entities to modify via a generative localizer and then revises them using a fine-tuned CodeT5-based reviser.

7.12 Performance Bugs

Performance bugs refer to non-functional defects that degrade efficiency while preserving functional correctness [63]. Such bugs, although not causing system failure, still represent undesired behavior because they violate performance expectations and can negatively impact user experience. Garg et al. [63] propose DeepDev-PERF, the first LLM-based approach to repair performance bugs for C# applications by fine-tuning the BART model. Building upon DeepDev-PERF, RAPGen [64] further enhances the repair process through prompt engineering, which demonstrates superior efficiency over traditional intensive fine-tuning methods. RAPGen retrieves relevant instructions from a knowledge base, constructs an input prompt tailored to the specific bug, and then uses Codex to generate the optimized code. This approach is similar to CEDAR [170] yet uniquely leverages prompt engineering to achieve more effective and resource-efficient solutions.

7.13 Smart Contracts

Smart contracts are self-executing contracts on a blockchain network with the terms of the agreement directly written into code. In 2023, Napoli et al. [169] conduct a preliminary study to explore the capabilities of ChatGPT in fixing vulnerable smart contracts with 143 vulnerable Solidity codes. In 2024, Zhang et al. [289] introduce ACFIX, a GPT-based approach to repair access control vulnerabilities in smart contracts based on static code slicing and

Chain-of-Thought Prompting. ACFIX mines common AC practices for major categories of code functionality. These practices form a knowledge base that guides LLMs in repairing code with analogous functionalities.

7.14 Misc Repair Types

We summarize other repair scenarios that are covered in fewer than two papers as follows.

API Misuse. Zhang et al. [302] conduct an empirical study to explore the performance of learning-based APR techniques on API misuse repair, involving nine LLMs with millions of parameters under a fine-tuning setting, i.e., CodeBERT, GraphCodeBERT, CodeGPT, PolyCoder-160M, PolyCoder-0.4B, CodeTrans, PLBART, CodeT5 and UniXcoder.

Crash Bugs. Crash bugs are considered critical issues as they might cause unexpected program behaviors and termination. Du et al. [44] conduct the first investigation into ChatGPT’s capability to resolve real-world crash bugs, particularly code-related and environment-related ones. They design different prompts and propose IntDiagSolver, an interaction approach that involves continuous interaction with LLMs.

Error-Handling Bugs. Error-handling bugs arise when erroneous conditions are not correctly detected, propagated, or managed by the software, leading to silent failures, data corruption, system crashes, or security vulnerabilities. Liu et al. [139] conduct the first in-depth empirical analysis of propagated error-handling bugs in mature software systems. They also introduce EH-Fixer, a retrieval-augmented generation approach to repair error-handling bugs by constructing the error-propagation path, retrieving similar historical cases, and prompting LLMs to generate patches.

Software Formal Proof. Formal software verification aims to validate the correctness of software properties. First et al. [55] introduce Baldur, an LLM-based approach to automate the generation and repair of whole formal proofs. Baldur leverages two versions of Minerva [122], which is pre-trained on a mathematics corpus based on the PaLM [32]: one with eight billion parameters and another with 62 billion parameters. Baldur first fine-tunes a generation model to synthesize whole proofs for theorems on a proof dataset, and then fine-tunes a repair model based on the proof assistant’s error messages to repair incorrectly generated proofs.

Formal Specification Such bugs often arise in specification languages (e.g., Alloy) where unintended logical constraints, missing relations, or erroneous propagation lead to incorrect or incomplete system models. Alhanah et al. [5] present a first systematic empirical investigation into the capability of LLMs for repairing faulty Alloy specifications. They design a novel dual-agent pipeline comprising a repair agent and a prompt agent, iteratively refining prompt-generation and repair proposals.

Motion Planning Algorithm. In the automated vehicles scenario, motion planners refer to algorithms that are responsible for computing a viable path for the vehicle to travel from an initial state to a designated goal region within a predefined time frame. Lin et al. [135] introduce DrPlanner, the first automated framework that utilizes GPT-4 to diagnose and repair motion planners. DrPlanner first designs a structured prompt to describe the planners using both natural and programming languages. DrPlanner then queries GPT-4 to repair algorithms with continuous diagnostic feedback in a closed-loop manner.

Translation Bugs. Code translation refers to the process of converting source code from one language into another without changing the original program’s functionality or logic. Pan et al. [185] present a large-scale empirical study to investigate the ability of LLMs for code translation across five different languages, including C, C++, Go, Java, and Python. They provide a bug taxonomy of unsuccessful translations across 15 categories and 5 groups, and design an iterative translation bug repair approach to fix such failures from LLMs, using a set of heuristics for prompt crafting.

7.15 Trend Analysis

Table 8 summarizes the distribution of LLM-based APR studies across various bug categories from 2020 to 2025. We discuss the research trend of LLM-based APR across different bug types.

Dominant Focus on Semantic Bug Repair. Semantic bug repair remains the core and most mature research direction in LLM-based APR. From 2020 to 2025, the number of studies addressing semantic bugs increased dramatically, from 4 papers in 2021 to more than 20 papers annually since 2023. This sharp growth reflects two parallel trends: (1) model evolution, from fine-tuned models (e.g., CodeT5 [281]) to general-purpose LLMs such as GPT-4 and Claude-3 [84]; and (2) methodological diversification, shifting from cloze-style styles [296] to conversational and agentic paradigms (e.g., ChatRepair [256], ContrastRepair [111], RepairAgent [19]). These successful explorations in semantic bug repair have laid the foundation and inspired subsequent studies on other bug types.

Rising Interest in Domain-Specific Bug Types. Several domain-oriented bug types have emerged since 2023, marking a broadening of APR’s application scope beyond classical semantic repair. For example, starting in 2023, programming problems form the second-largest research stream, with the number of related studies surging to 13 in 2024. Security vulnerability repair has become a persistent and practically important theme since 2022, maintaining 6–7 papers per year. Most studies fine-tune domain-specific code models (e.g., CodeT5, UniXcoder [295]) or use GPT-4 for zero-shot [1] repair on vulnerability datasets. Such diversification demonstrates the transferability of LLM reasoning to heterogeneous domains and suggests that APR research is progressively converging with broader software applications.

Emerging and Niche Bug Types. Until 2022, LLM-based APR research is limited to about five major bug types. However, since 2023, the field has expanded rapidly, with researchers starting to explore a wide spectrum of specialized or cross-domain bugs, each addressed by only one or two studies so far. For example, repository-level issue repair has represented a new frontier since 2024. These works (e.g., SWE-agent [270], RepoGraph [182], OpenHands [237]) extend APR to realistic GitHub repositories requiring long-context reasoning, multi-file edits, and tool orchestration. The rapid expansion from 7 to 11 papers between 2024 and 2025 demonstrates the growing emphasis on scalability and autonomous agent behavior.

Overall, the evolution of LLM-based APR research exhibits a clear trajectory, from foundational studies centered on semantic bug repair to increasingly diversified efforts tackling domain-specific, large-scale, and cross-modal bug types. This transition reflects not only the rapid advancement of model capabilities but also the community’s growing ambition to address real-world software engineering challenges. As LLMs continue to improve in reasoning, memory, and tool integration, future APR research is expected to move toward unified frameworks that can adaptively diagnose and repair heterogeneous bugs across different abstraction levels and programming environments.

Summary of Results for RQ3

(1) LLMs have been applied in a wide array of repair scenarios in the literature, involving 20 bug types. (2) In some common scenarios dominated by traditional APR, such as semantic bugs, researchers continue to invest substantial efforts in investigating the application of LLMs. (3) Thanks to LLMs’ general knowledge learned from all possible Internet data, LLM-based APR has been extended to some rare scenarios that are previously unexplored, such as hardware bugs and Web UI.

8 RQ4: What key factors contribute to the integration of LLMs for APR?

8.1 What sources of datasets are used to evaluate LLM APR studies?

Benchmarking is crucial for evaluating the performance of APR techniques, fundamentally shaping the direction of development within the research community. For example, Defects4J [103] has been the common practice

Table 13. Distribution of publications across datasets

Model	2020	2021	2022	2023	2024	2025	Total	Model	2020	2021	2022	2023	2024	2025	Total
Defects4J	1	1	2	12	19	16	51	Codeflaws	0	0	0	0	2	1	3
QuixBugs	0	1	5	9	10	4	29	ManySSuBs4J	0	1	1	0	1	0	3
HumanEval-Java	0	0	0	1	3	8	12	DebugBench	0	0	0	0	1	2	3
BFP	0	1	1	8	1	0	11	DeepDev-PERF	0	0	1	0	0	1	2
SWE-bench lite	0	0	0	0	4	6	10	DB	0	0	0	2	0	0	2
HumanEval	0	0	0	4	4	2	10	SWE-bench Verified	0	0	0	0	0	2	2
CVEfixes	0	0	1	3	3	1	8	Refactory	0	0	0	1	1	0	2
TFix	0	1	0	5	0	0	6	RWB	0	0	0	0	1	1	2
MBPP	0	0	0	1	5	0	6	MITRE	0	0	0	1	1	0	2
Big-Vul	0	0	0	3	3	0	6	BIFI	0	0	0	2	0	0	2
SWE-Bench	0	0	0	0	3	2	5	TypeBugs	0	0	0	0	2	0	2
BugsInPy	0	0	0	0	2	2	4	HumanEvalFix	0	0	0	1	1	0	2
Vul4J	0	0	0	1	1	2	4	VJBench	0	0	0	1	1	0	2
N.A.	0	0	0	3	1	0	4	Pearce et al.	0	0	0	1	1	0	2
ManyBugs	0	0	1	1	2	0	4	BugAID	0	0	1	0	1	0	2
APPS	0	0	0	2	2	0	4	NC	0	0	0	2	0	0	2
ExtractFix	0	0	0	0	2	1	3	ConDefect	0	0	0	0	1	1	2
Bears	0	0	0	1	2	0	3	CPatMiner	0	0	1	1	0	0	2

over the last decade, facilitating a standard comparison scenario to understand the strengths and weaknesses of proposed approaches and guiding researchers towards addressing the most pressing challenges in the field. We identify a total of 111 benchmarks from all collected studies and present the benchmarks utilized more than once in Table 13.

We find that Defects4J [103] (51 papers) is the most popular benchmark, followed by QuixBugs [134] (29 papers), HumanEval-Java [96] (12 papers), both of which are existing benchmarks from previous APR studies. This phenomenon is reasonable, as existing benchmarks are usually well-constructed and have been inspected and utilized by the community, making them the preferred choice for evaluating new APR techniques. For example, a mass of LLM-based APR techniques are evaluated with existing popular benchmarks, such as CIRCLE [281], AlphaRepair [255], and GAMMA [296].

Besides, we notice some newly constructed benchmarks in LLM-based APR, such as HumanEval-Java [96] (12 papers) and SWE-bench lite (10 papers). We summarize these new benchmarks into three categories. The first category dataset is tailored for rare repair scenarios. As discussed in Section 7, LLMs have been used in a variety of scenarios, some of which have not been considered by previous work, leading to a gap in relevant benchmarks. As a result, with the advent of LLM-based APR techniques, researchers have also developed corresponding new datasets, such as TFix [15] for static warnings, DeepDev-PERF [63] for performance bugs, Du et al. [44] for crash bugs, and Zhang et al. [302] for API misuse. The second category dataset attempts to address the limitations of previous benchmarks. For example, considering BFP [224] lacking test suites, FixEval [79] offers a collection of unit tests for a large scale of competitive programming problems and is evaluated with PLBART and CodeT5. The third category of datasets is designed to address the issues unique to LLMs, particularly the data leakage problem. For example, Jiang et al. [96] create a new evaluation benchmark, HumanEval-Java, that has not been seen by LLMs during pre-training. Zhang et al. [300] extensively explore the data leakage issue of ChatGPT in the APR domain and introduce EvalGPTFix, a new benchmark from competitive programming problems after the training cutoff point of ChatGPT. DebugBench [220] is a follow-up of EvalGPTFix with a larger scale and more diverse types of bugs. DebugBench contains 4,253 buggy programs from the LeetCode community, covering four major bug categories and 18 minor types in C++, Java, and Python. Similarly, ConDefects [249] contains 1,254 Java faulty programs and 1,625 Python faulty programs from the online competition platform AtCoder. These collected programs are produced between October 2021 and September 2023 to address the data leakage issue

for LLM-based APR approaches. Different from the aforementioned benchmarks derived from programming problems[72], Silva et al. [207] introduce *GitBug-Java*, a reproducible benchmark comprising 199 recent Java bugs. These bugs are extracted from the 2023 commit history of 55 notable open-source repositories to mitigate the risk of data leakage.

8.1.1 Trend Analysis. From Table 13, a clear temporal trend is evident in the evolution of benchmarks used to evaluate LLM-based APR techniques. Before 2022, studies mainly rely on traditional benchmarks such as *Defects4J*, *BFP*, and *QuixBugs*, which are originally designed for traditional APR approaches. These benchmarks primarily target Java or Python bugs and offer limited coverage of diverse programming languages and bug categories. With the emergence of LLM-based repair around 2022, researchers begin to explore more complex and large-scale datasets. The number of datasets increases substantially after 2023, reflecting both the expansion of APR scenarios and the community’s growing emphasis on comprehensive and standardized evaluation.

In particular, *Defects4J* remains the most widely used benchmark across all years (51 studies), underscoring its established role as a standard testbed in the APR community. However, since 2023, there is a clear shift toward more diversified benchmarks, such as *HumanEval-Java* (12 studies), indicating a move toward benchmarks that better align with the scale and reasoning capabilities of modern LLMs. These newer benchmarks are often multi-language, task-diverse, and intentionally designed to mitigate issues such as data leakage (e.g., *HumanEval-Java* and *DebugBench*). Furthermore, the introduction of *SWE-bench* in 2024–2025 reflects a methodological shift toward verifiable, reproducible, and large-scale benchmarks tailored to LLM evaluation.

Overall, the trend indicates a transition from traditional, manually curated datasets to large-scale and repository-aware benchmarks. This evolution marks a maturation of the field, from proof-of-concept experiments on small datasets to robust and reproducible evaluations grounded in realistic software repair scenarios. It also highlights a growing consensus within the community to establish standardized evaluation protocols for LLM-based APR, paving the way for fairer and more interpretable comparisons among future approaches.

8.2 What input forms are software bugs transformed into when utilizing LLMs?

Thanks to the powerful natural language understanding and code reasoning capabilities of LLMs, the inputs used in LLM-based APR are much richer and more diverse than those in traditional APR techniques [292]. We categorize these input representations into six major types according to their data modalities and structural forms. As shown in Table 14, prompt-based inputs remain the dominant form, adopted by 47.78% (86) of all collected studies, where LLMs are guided through carefully engineered textual instructions or templates describing the bug-fixing task. Meanwhile, chat-based inputs (18.33%) have gained popularity with the emergence of conversational LLMs, enabling iterative and context-aware repair dialogues. Only 13.89% of studies still rely on raw code inputs similar to those used in traditional DL/ML repair models. In addition, a small but growing proportion of works explore mask-based (7%) and repository-level (5%), indicating a trend toward richer contextualization and multi-source reasoning in future APR systems. We will discuss the five input representations utilized by LLMs as follows.

Raw Bug-fixing Input. Similar to most traditional learning-based APR, this type of input regards APR as an NMT task, which translates a sentence from one source language (i.e., buggy code) to another target language (i.e., fixed code). Such representation directly feeds LLMs with the buggy code snippet and has been typically employed to train LLMs with supervised learning in semantic bugs [42, 159, 321] security vulnerabilities [59, 295], and static warnings [107]. For example, Zhang et al. [295] investigate the performance of three bug-fixing representations (i.e., context, abstraction, and tokenization) to fine-tune five LLMs for vulnerability repair.

Prompt Input. This type of input incorporates more information into the buggy code. The prompt concatenates different input components with a prefixed prompt, thus effectively bridging the gap between pre-trained tasks and the APR downstream task. For example, *CIRCLE* [281] utilizes a manually-designed prompt template to convert buggy code and corresponding context into a unified fill-in-the-blank format. Particularly, they utilize

Table 14. Distribution of publications across input forms

Input	Year	Count	Citations
Raw	2021	3	[97] [43] [159]
	2022	6	[3] [59] [128] [63] [117] [88]
	2023	8	[58] [90] [78] [295] [321] [81] [187] [302]
	2024	4	[201] [68] [312] [192]
	2025	4	[245] [61] [148] [153]
Prompt	2021	1	[15]
	2022	5	[281] [110] [191] [208] [107]
	2023	25	[6] [44] [51] [60] [93] [188] [222] [101] [102] [247] [236] [313] [321] [145] [170] [162] [169] [62] [197] [113] [187] [146] [218] [254] [300]
	2024	33	[1] [31] [52] [74] [270] [77] [144] [124] [116] [142] [138] [118] [143] [178] [185] [174] [189] [287] [223] [226] [227] [234] [29] [273] [80] [82] [285] [310] [317] [199] [205] [257] [265]
	2025	22	[33] [64] [47] [151] [240] [263] [251] [280] [289] [175] [24] [53] [84] [112] [136] [152] [186] [262] [269] [91] [286] [206]
Mask	2022	2	[255] [196]
	2023	8	[55] [248] [252] [296] [96] [114] [241] [254]
	2024	4	[189] [228] [133] [267]
Conversation	2022	1	[208]
	2023	10	[155] [89] [98] [165] [177] [137] [304] [250] [300] [288]
	2024	12	[5] [28] [40] [85] [309] [135] [212] [268] [314] [315] [256] [278]
	2025	10	[20] [35] [139] [181] [25] [19] [46] [111] [200] [104]
Repository	2024	3	[9] [26] [306]
	2025	11	[92] [73] [168] [106] [7] [182] [213] [237] [279] [121] [277]
Conversation	2022	1	[128]
	2023	2	[81] [305]
	2024	2	[306] [316]
	2025	2	[123] [183]
Multimodal	2025	1	[92]

“Buggy line:” and “Context:” to denote the buggy and contextual code, and they utilize “The fixed code is:” to query a T5-based model to generate candidate patches according to the previous input. Besides, TFix [15], Zirak et al. [321] and Kim et al. [107] represent all valuable information about the bug as a single piece of text, including bug type, bug message, bug line, and bug context. Furthermore, InferFix [101] and RAP-Gen [236] construct prompts by retrieving relevant repair examples from an external codebase.

Mask Input. This type of input masks the buggy code and queries LLMs to fill the masks with the correct code tokens. Unlike the above input forms, the mask input reformulates the APR problem as a cloze-style task and directly leverages LLMs’ pre-training objectives in a zero-shot setting. AlphaRepair [255] is considered the first work to demonstrate the potential of mask inputs, and researchers have proposed various follow-ups to better perform mask prediction for patch generation, such as GAMMA [296] with well-summarized repair patterns, FitRepair [252] with the plastic surgery hypothesis, Repilot [241] with a completion engine, as well as empirical studies [96, 254].

Conversation-Style Input This type of input further extends the prompt input with feedback-driven chats like humans. Conversation-style representation contains more complex information, such as dynamic execution results, while iteratively improving generated patches through multiple rounds of dialogue. For example,

Sobania et al. [208] conduct an early exploration into the feasibility of leveraging ChatGPT’s conversational capabilities for program repair, motivating some follow-ups [256, 300].

⑤**Structure-Aware Input.** This type of input represents source code as syntactic structures, such as Abstract Syntax Trees (ASTs). For example, Horvath et al. [81] utilizes RoBERTa and GPTNeo to encode ASTs for program repair. Besides, VulMaster [316] utilizes the AST as part of its input to capture the structural aspects of the vulnerable code.

⑥**Repository-level Input.** This type of input extends beyond single-file or localized code fragments to encompass the entire software repository. Such inputs enable LLMs to access multi-file dependencies and repository-level relationships, thereby capturing the holistic semantics required for realistic bug fixing. Unlike earlier APR paradigms that rely on static context windows, repository-level inputs are dynamically retrieved, updated, and reasoned about by autonomous LLM agents capable of tool invocation and code search. For example, approaches such as SWE-agent [270], AutoCodeRover [306], and RepoGraph [182] leverage agents equipped with retrieval and execution tools to navigate the repository, localize faults, edit multiple files, and validate candidate patches.

8.2.1 Trend Analysis. Table 14 presents a clear evolution in how software bugs are represented as inputs for LLM-based APR. First, in the early years (2021–2022), most studies use raw inputs similar to those of traditional learning-based APR models. At this stage, researchers mainly adapt models such as CodeBERT and CodeT5 to program repair tasks, where the input consists primarily of buggy code and its immediate context. There are also some studies that adopt mask-based inputs to simulate the pre-training objectives of LLMs, allowing models to directly predict the correct code tokens without additional fine-tuning. Second, starting from 2023, prompt-based inputs rapidly become dominant, accounting for nearly half of all studies (47.78%). This trend coincides with the widespread availability of instruction-tuned models such as GPT-3.5 and with the increasing use of prompt engineering to align model behavior with repair objectives. Prompt-based inputs integrate natural language instructions, examples, and bug context, marking a significant shift from pure code transformation toward task-oriented reasoning. At the same time, conversation-style inputs emerge as an important paradigm from 2023 onward, extending prompt-based formats with dynamic, feedback-driven interactions between the LLM and its environment (e.g., test execution results). The continuous increase in conversation-style studies through 2025 reflects a growing interest in interactive and human-in-the-loop repair processes. Very recently, researchers have attempted to explore repository-level and multimodal inputs (first appearing in 2024–2025). These approaches move beyond single-file contexts to capture multi-file dependencies, project histories, and even visual information, supporting real-world repair scenarios that require long-context reasoning and cross-file understanding.

Overall, the evolution of input forms reflects a broader shift in the field, from static, isolated code snippets to dynamic, context-aware, and semantically rich representations. This trend reflects how LLM-based APR increasingly emphasizes reasoning, interactivity, and holistic program understanding, rather than treating bug fixing as a simple sequence-to-sequence translation problem in learning-based APR [292].

8.3 How are LLMs used to support patch correctness?

Test-overfitting has recently been the focus of APR research due to the mainstream test-driven generate-and-validate repair workflow in the community. Particularly, after candidate patches are generated, the developer-written test suites are utilized as the oracle to validate the correctness of patches, and the passing patches are returned to developers. However, as an incomplete specification, test suites only describe a part of the program’s behavioral space, resulting in plausible yet overfitting patches without fixing bugs. Thus, developers need to spend enormous effort filtering out overfitting patches manually, even resulting in a negative debugging performance [214, 303]. Researchers have proposed various automated **patch correctness** assessment (APCA)

techniques to identify whether a plausible patch is overfitting or not, so as to improve the quality of returned patches. For example, PATCH-SIM [260] assesses the correctness of patches by calculating the similarity of dynamic execution traces. PATCH-SIM is acknowledged as a foundational work in the APCA field, providing crucial guidance for the development of follow-up works, particularly learning-based or LLM-based ones [292].

Table 15. A summary of existing APR studies using LLMs to predict patch correctness.

Year	Study	LLMs	Repository
2020	Tian et al. [215]	BERT	https://github.com/TruX-DTF/DL4PatchCorrectness
2022	Quatrain [219]	BERT	https://github.com/Trustworthy-Software/Quatrain
2023	Tian et al. [217]	BERT	https://github.com/HaoyeTianCoder/Panther
2023	Invalidator [119]	CodeBERT	https://github.com/thanhlecong/Invalidator
2023	PatchZero [317]	StarCoder	N.A.
2024	APPT [293]	BERT, CodeBERT, GraphCodeBERT	https://github.com/iSEngLab/APPT

Table 15 presents existing APCA studies involving LLMs. We summarize them into three stages.

①LLMs as Feature Extractor. In 2020, Tian et al. [215, 217] empirically explore the performance of code embeddings via representation learning models in reasoning about patch correctness. Following the similarity-based pipeline from PATCH-SIM, they first calculate the similarities of patched and buggy code snippets based on code embeddings, and then predict patch correctness with a binary classifier. They consider four embedding models, including re-trained (i.e., Doc2vec, code2vec and CC2vec) and pre-trained models (i.e., BERT), which is the first APCA study empowered with LLMs. Recently, Le et al. [119] propose Invalidator, to assess the correctness of patches via semantic and syntactic reasoning. Similar to Tian et al. [215], they utilize CodeBERT to extract code features and train a classifier for prediction. Unlike the above studies calculating similarities of patches, Tian et al. [219] formulate APCA as a question-answering (QA) problem and propose Quatrain. Quatrain first utilizes CodeBERT to encode bug reports and patch descriptions and trains a QA model for prediction.

②Fine-tuning LLM-based APCA. In 2024, Zhang et al. [293] propose APPT, equipped with BERT as the encoder stack, followed by an LSTM stack and a deep learning classifier. Unlike previous studies [215, 217] limiting BERT to extract features without benefiting from training, APPT further fine-tunes LLMs in conjunction with other components as a whole pipeline to fully adapt it specifically for reasoning about patch correctness. APPT is implemented with BERT by default and is also proven generalizable to other advanced LLMs, such as CodeBERT and GraphCodeBERT.

③Zero-shot LLM-based APCA. In 2023, Zhou et al. [317] propose PatchZero to explore the feasibility of LLMs in predicting patch correctness with a zero-shot setting. PatchZero directly queries LLMs to generate the next token about patch correctness (i.e., a token either “correct” or “overfitting”) based on previous tokens, which is similar to LLMs’ original pre-training objective.

8.4 How are LLMs utilized to facilitate both code generation and program repair?

Compared with traditional APR approaches [292] that usually employ heuristics or neural networks to generate a multitude of patches in one go, LLMs can iteratively refine the generated patches based on the outcomes of dynamic execution. As mentioned in Section 6.2, the iterative patch generation capabilities brought by LLMs have facilitated the emergence of conversational-based repair techniques [35, 169, 188, 189, 191, 208, 300]. In addition to conversational-based repair, we have identified some efforts that improve code generation performance by repairing LLM-generated code with feedback, referred to as self-repair. Different from conversational-based repair, which utilizes feedback for patch generation, self-repair approaches leverage LLMs to identify code errors implemented by themselves via investigating execution results and explaining the generated code. Such

self-repair approaches integrate code generation and program repair, which can be considered a further step toward automated programming.

Table 16. A summary of existing studies that use APR to boost code generation.

Year	Study	LLMs	Repository
2023	AgentCoder [89]	GPT-3.5, GPT-4, Claude, PaLM	N.A.
2023	Self-Edit [288]	InCoder, CodeGen, Codex	https://github.com/zkcpku/Self-Edit
2024	Self-Debugging [28]	GPT-3.5, GPT-4, Codex, StarCoder	N.A.
2024	Zheng et al. [314]	CodeLLaMA, DeepseekCoder	https://github.com/OpenCodeInterpreter/OpenCodeInterpreter
2024	CYCLE [40]	CodeGen, StarCoder	https://github.com/ARISE-Lab/CYCLE_OOPSLA_24
2024	LDB [315]	StarCoder, CodeLLaMA, GPT-3.5	https://github.com/FloridSleeves/LLMDebugger
2024	Olausson et al. [177]	GPT-3.5, GPT-4, CodeLLaMA	https://github.com/theoxo/self-repair
2024	Hu et al. [85]	GPT-4	N.A.

Table 16 presents some LLM-based studies that leverage program repair to boost code generation. Self-Edit [288] represents the first attempt to adopt a neural code editor that takes both the generated code and error messages as inputs to improve the code quality on the competitive programming task. Self-Edit is evaluated with both fine-tuned models (i.e., PyCodeGPT, GPT-Neo, CodeGen, GPT-Neo, InCoder, GPT-J) and prompt-based LLMs (i.e., InCoder, CodeGen, Codex). Chen et al. [28] from DeepMind propose Self-Debugging to teach LLMs to debug their own predicted code via few-shot prompting, including Codex, ChatGPT, GPT-4 and StarCoder. There also exist some similar self-repair studies, including OpenCodeInterpreter [314], Cycle[40], LDB [315], SelfEvolve [98], Self-Refine [155], Hu et al. [85], AgentCoder [89]. Recently, Olausson et al. [177] conduct an empirical study to investigate the ability of CodeLlama, GPT-3.5, and GPT-4 to perform self-repair in code generation. They find that self-repair is not a panacea for code generation challenges, as existing LLMs often fail to provide reliable, accurate, and valuable feedback on why the code is incorrect.

8.5 How often do the collected LLM-based APR papers provide publicly available artifacts?

Open science plays a crucial role in advancing scientific progress through principles of transparency, reproducibility, and applicability. Given the benefits, the SE community has been actively promoting open science principles and encouraging all researchers to share their artifacts, thereby bolstering the reliability of research findings. In this section, we investigate the extent to which the analyzed papers make their artifacts publicly accessible.

We find that 80 studies provide the replication packages in their papers, accounting for 62.99% (80/189) of all collected studies. Among 78 studies that propose novel LLM-based APR approaches, which is the largest contribution type in Table 5, we find that 53.85% (42/78) of them fail to make their artifacts publicly available. This makes it difficult for researchers to validate experimental findings, conduct quantitative comparisons with existing studies, and build follow-ups instead of reinventing the wheel. Considering that some papers have not been published, we then focus on top-tier SE venues, i.e., ICSE, ASE, FSE, ISSTA, TSE and TOSEM, and identify 86.84% of papers (33/38) make related artifacts publicly open, indicating a high-quality commitment to reproducibility in high-quality papers. Besides, some studies only provide datasets or trained models without source code or essential instructions. Overall, open science is a critical challenge in advancing LLM-based APR research development because many factors, such as datasets, data pre-processing methods, source code, hyper-parameters, and documents, may lead to the reproducibility of studies. Therefore, we hope that researchers in the LLM-based APR community can provide high-quality open-source artifacts for convenient reproduction. For transparency and reproducibility, we provide the detailed artifact availability statistics and corresponding open-source links for all collected papers in our replication package repository.

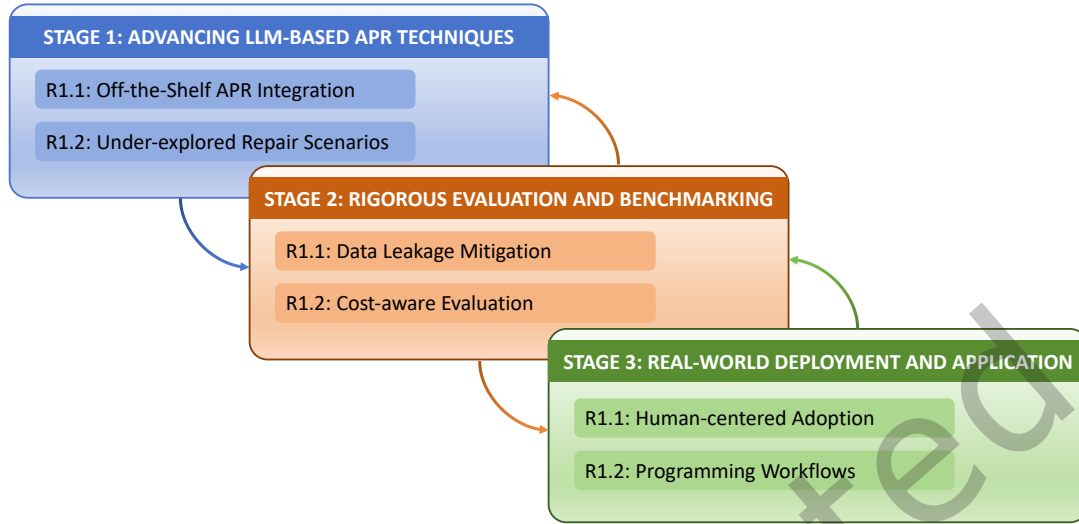


Fig. 5. A three-stage future research roadmap for LLM-based APR.

Summary of Results for RQ4

(1) We summarize 111 different datasets that are utilized to benchmark LLMs in fixing bugs. (2) Defects4J, QuixBugs, BFP, CVEfixes, and Big-Vul are most frequently adopted in the LLM-based APR. (3) We categorize the input forms within all collected papers into five groups: raw bug-fixing input, prompt input, mask input, conversation-style input, and structure-aware input. (4) Prompt input is the most frequently used form in applying LLMs to program repair, indicating that designing effective prompts is particularly important for leveraging LLMs' natural language processing capabilities. (5) We summarize some studies that leverage LLMs to predict patch correctness. (6) 62.99% of all collected papers have made their artifacts open source, and the ratio increases to 86.84% for top-tier SE publications.

9 Research Agenda for LLM-based APR

Based on our systematic analysis, this section presents a structured research agenda that outlines concrete and actionable directions for future studies on LLM-based APR. As illustrated in Figure 5, we organize future research directions into three progressive stages: (1) advancing LLM-based APR techniques, (2) rigorous evaluation and benchmarking, and (3) real-world deployment and adoption.

9.1 Stage 1: Advancing LLM-based APR Techniques

9.1.1 Off-the-Shelf APR Integration. Current state. As mentioned in Section 7, researchers typically utilize LLMs as core backbones to design novel repair approaches in an end-to-end setting, such as sequence-to-sequence learning [15, 281]. Parallely, the community has seen some explorations treating LLMs as components integrated into existing repair workflow. These studies attempt to boost the capabilities of off-the-shelf APR approaches instead of proposing new techniques. For example, CURE [97] combines GPT and CoCoNut [149] to capture code syntax for the APR task. Built on top of DLFix [127], DEAR [128] attempts to fix multi-hunk, multi-statement bugs by fine-tuning BERT to learn fixing-together relationships among statements, i.e., identifying if two statements are

needed to be fixed together or not. Recently, GAMMA [296] integrates LLMs into the traditional template-based APR TBar by querying LLMs to generate masked code tokens instead of retrieving donor code from local files. These efforts demonstrate the potential of integrating LLMs with off-the-shelf APR techniques, yet there is currently a lack of more in-depth work in this area.

Opportunities. In the future, researchers could attempt to combine LLMs with more traditional APR techniques. For example, it is promising to utilize LLMs to help SMT solvers generate patches for constraint-based APR, or feed search algorithms with LLM-generated potential patches to build search space for heuristic-based APR. Besides, domain-specific repair techniques can benefit from the powerful code-understanding capabilities of LLMs, thus extending to a broader range of repair scenarios. For example, we can design fix templates for specific scenarios, such as static warnings, and then utilize the general knowledge contained in LLMs to generate correct patches.

9.1.2 Under-explored Repair Scenarios. Current state. As summarized in Section 7, we observe that most existing LLM-based APR studies are concentrated on a limited number of bug types, particularly semantic bugs. However, there exist some rare repair scenarios that benefit less from LLMs, such as hardware bugs (one paper) and concurrency bugs (zero paper). A key challenge lies in the scarcity of labeled training data and the increased complexity associated with these scenarios.

Opportunities. We suggest that future work concentrate on three possible directions to broaden the scope of LLM applications for more repair scenarios, such as software requirement [259] and fuzzing [307].

- Transfer learning is an effective training approach for rare scenarios. We can first fine-tune LLMs with abundant data in a source scenario and then utilize a small amount of data to transfer the acquired knowledge to a target scenario. The source and target scenarios should have similar data distributions. For example, Zhang et al. [295] demonstrate that transferring learning from bug-fixing can improve the vulnerability repair performance of five LLMs by 9.40% on average.
- It is promising to utilize the cloze-style APR with repair patterns to generate patches for rare scenarios. Unlike fine-tuning, which requires substantial labeled data, cloze-style APR effectively leverages expert domain knowledge to guide LLMs in performing APR based on their existing pre-trained knowledge without additional retraining. For example, researchers design repair patterns and exploit LLMs' pre-training objective (i.e., predicting masked tokens) to directly generate correct patches by filling masked tokens under these patterns. This makes the cloze-style APR particularly effective for scenarios with limited training data or domain-specific bug types. Besides, for scenarios not previously encountered by LLMs, researchers can employ unlabeled data in the target scenario to learn the data distribution of the project or language under test in an unsupervised learning setting.
- In-context learning and prompt engineering are feasible solutions for directly querying billion-parameter LLMs to generate correct code in a target scenario, given that these models are immensely large and encompass virtually all data available on the Internet.

9.2 Stage 2: Rigorous Evaluation and Benchmarking

9.2.1 Data Leakage Mitigation. Current state. As highlighted in Section 8.1, Zhang et al. [300] identify that existing repair benchmarks have been inadvertently included in the pre-training data of popular LLMs, such as ChatGPT, through web scraping and other methods. For example, ChatGPT is able to enumerate all projects within Defects4J [103], one of the most popular APR benchmarks. Researchers [296] can ascertain the exposure for open-source LLMs by inspecting pre-training data against benchmarks. It is significantly more challenging with more powerful black-box LLMs due to a lack of training details. However, as preliminary explorations, they are mainly limited in the quantity of involved bugs and the variety of bug types. For example, all of them are created from programming problems with only small-scale or medium-scale buggy solutions, without delving

into large-scale, real-world projects that encompass complex API calls, such as Defects4J [103]. More importantly, data leakage issues in other repair scenarios, such as security vulnerabilities and API misuse, continue to be overlooked. There may be overlaps among datasets across different repair scenarios, such as Defects4J, which also serves as a source for a subset of the API misuse dataset [302]. Overall, the risk of data leakage introduces bias into the benchmarking of existing work, necessitating urgent efforts from researchers to mitigate it.

Opportunities. We recommend that future work be conducted in the following two directions. First, it is crucial to construct a large-scale benchmark free from data leakage that contains real-world projects so as to evaluate the actual fix capabilities of LLMs in a more practical debugging scenario. Commercial closed-source software, real-time updated programming websites, or manually written programs may serve as potential data sources. Second, considering the variety of bug types that LLMs have been applied to, researchers need to consider and attempt to address the data leakage risk when conducting related studies.

9.2.2 Cost-aware Evaluation. Current state. As mentioned in Section 6.1, the literature tends to employ the growing size of LLMs to achieve better performance, such as from T5-250M in CIRCLE to Codex-12B in InferFix. This trend is reasonable as Xia et al. [254] demonstrate that larger models typically generate more correct patches for software bugs. However, APR, being a task highly relevant to humans, the millions, billions and even trillions of parameters pose significant challenges in the development workflow. First, training LLMs with billions (or even more) of parameters is a highly time-consuming and resource-intensive process. The GPU resources required for such training are often prohibitively expensive, making them inaccessible for many researchers in both academic and industrial settings. For example, in Section 6.2, we observe that most fine-tuning-LLM-based APR studies utilize CodeT5/T5 or similar-sized models, except for InferFix from Microsoft, which fine-tunes Codex. However, InferFix is proposed by the world-leading technology company Microsoft and trained with industrial-grade hardware, i.e., 64 32-GB-V100-GPUs. Second, despite the increased likelihood of generating correct patches with larger models, the patch generation time cost also increases. For example, Jiang et al. [96] demonstrate PLBART takes 0.70–0.89 seconds on average to generate a correct patch, CodeGen, although fixing more bugs than PLBART with more parameters, requires 3.64–13.88 seconds on average to generate a correct patch. Third, the increase in patch generation time further compresses the time available for patch validation, as developers need to spend more time waiting for the model to complete its inference. For example, Shi et al. [204] demonstrate that the vast storage and runtime memory consumption, coupled with high inference latency, make these LLMs prohibitive for integration in modern IDEs, especially on resource-constrained or real-time terminal devices.

Opportunities. In the future, to address the first challenge, it is promising to explore the potential of parameter-efficient fine-tuning approaches on APR, such as prefix-tuning and low-rank adaptation [39]. In addition, recent sparsity-aware optimization frameworks such as SparseGPT [56], SparseML [156], and DeepSparse [172] demonstrate promising directions for improving inference efficiency of LLMs, which could also benefit LLM-based APR in future work. To address the second challenge, researchers can optimize the size of LLMs without significantly compromising their performance, such as model pruning, quantization, and knowledge distillation [204]. To address the third challenge, we recommend boosting patch validation with advanced strategies, such as mutation testing [36, 258], or utilizing LLMs to rank candidate patches before validation.

9.3 Stage 3: Real-world Deployment

9.3.1 Human-centered Adoption. Current state. As summarized in Section 7, with the introduction of LLMs, APR has achieved groundbreaking progress in terms of the number of correctly fixed bugs in popular benchmarks, exemplified by the advancements on Defects4J-1.2 from 64 bugs by CIRCLE, to 74 bugs by AlphaRepair, and 82 bugs by GAMMA. The progress prompts us to consider whether LLMs have indeed facilitated improvements in real-world debugging and how LLM-based APR has broader implications for developers' daily activities. Previous research [244, 301] highlights the potential pitfalls of developing APR tools without adequate feedback from

developers, which could compromise their effectiveness in real-world deployment. However, there remains a significant gap in our understanding of how software engineers tackle software problems in practical settings, including their use of dedicated debugging tools and their expertise in debugging techniques.

Opportunities. Researchers should conduct human studies to gain deeper insights into the maturity and reliability of LLM-based APR tools in terms of human factors. Possible directions are to investigate whether LLMs can assist developers in reducing the debugging cost, such as fixing more bugs, accelerating the bug-fixing process, and handling more complex bugs. Besides, it would be valuable to investigate developers' perceptions and interactions with LLMs based on their practical experiences and established debugging practices.

9.3.2 Programming Workflows. Current state. As mentioned in Section 2, existing APR approaches operate under a “near-correct assumption” [36, 176], which posits that experienced programmers are capable of writing programs that are almost correct, requiring only minimal modifications to rectify bugs and ensure compliance with all test cases. This assumption has long served as the foundation for APR research development. However, the evolution of LLMs and their application in programming suggests a future where APR can transcend its traditional boundaries, moving towards a more holistic approach in conjunction with fully autonomous programming. In this context, APR can be reimagined not only just as a tool for correcting minor coding errors, but as an integral component of a self-correcting, self-improving system that iteratively enhances the quality of automatically generated code. Fortunately, we have observed initial explorations into combining repair with programming, yet these efforts are far from sufficient. For example, Fan et al. [51] employ LLMs to fix buggy solutions generated by themselves, and recent studies [256, 300] iteratively refine auto-generated code with dynamic execution.

Opportunities. In the future, the opportunities presented by integrating APR with fully autonomous programming are vast [150]. First, it is flexible to implement collaborative Human-AI Programming tools [131], where the initial code is written by developers and continuously optimized and repaired by LLMs. Particularly for complex problem-solving, LLMs suggest innovative solutions that had not been considered by human programmers. This partnership could accelerate development cycles, reduce the burden of debugging on human developers, and lead to more creative and effective solutions. Second, the general knowledge inherent in LLMs endows them with the capability to support multiple downstream tasks, which opens up the possibility of bridging the gap between code generation, testing, debugging, and fixing. For example, fault localization is a precondition of patch generation, while patch validation can reflect the accuracy of fault localization, making these tasks interconnected. Thus, it is promising to explore the capabilities of LLMs in these connected tasks using real-time feedback within a unified framework.

10 Threats to Validity

In this section, we discuss several potential threats to validity and the corresponding mitigation strategies below.

10.1 Construct Validity

Construct validity concerns whether our RQ formulation and categorization processes accurately reflect the intended objectives. In designing RQs, as the first literature review in the LLM-based APR field, we followed prior LLM4SE [231] and APR [292] survey studies, focusing on three main dimensions (i.e., APR in Section 5, LLMs in Section 6 and integration in Section 7) to systematically classify the collected papers. However, one threat arises from the possible ambiguity of paper categorization, as some studies could reasonably belong to multiple categories (e.g., Mashhadi et al. [159] involving both fine-tuning and empirical evaluation). To mitigate this, we assign each paper to the category that best represented its , following consistent classification rules.

10.2 Internal Validity

Internal validity threats are related to potential bias in the process of paper selection, data extraction, and quality assessment. To reduce such risks, we followed a multi-stage procedure combining both manual and automated search. First, we performed manual searches in top venues to identify seed papers, which are then used to design the search keywords for automatic retrieval from digital libraries. Second, we applied inclusion and exclusion criteria consistently across all sources and conducted a forward and backward snowballing process to ensure completeness. Finally, each included paper underwent a quality assessment using a ten-question checklist. Two authors independently performed selection and evaluation, and any disagreement was resolved through discussion.

10.3 External Validity

External validity refers to the generalizability of our findings beyond the collected dataset. Although we carefully collected studies published between 2020 and September 2025 from major digital libraries and high-quality preprints, some relevant works may have been missed due to limitations in indexing or keyword variations. To mitigate this, we employed iterative search refinement, snowballing, and inclusion of diverse publication venues across SE, AI, and PL communities. Nevertheless, we acknowledge that the rapidly evolving nature of LLM-based APR research means that future studies may extend or refine our findings.

11 Conclusion

Automated Program Repair (APR) tackles the long-standing challenge of fixing software bugs automatically, thus facilitating software testing, validation, and debugging practices. Very recently, Large Language Models (LLMs) have brought significant changes to the APR domain, already yielding impressive progress and further demonstrating a promising future in follow-up research. In this paper, we provide a systematic literature review of existing LLM-based APR techniques from LLMs, APR and their integration perspectives. We summarize popular LLMs, typical utilization strategies, and repair scenarios. We also discuss some crucial factors, such as input forms and self-debug metrics, within the LLM-based APR community. Finally, we outline several challenges, such as data leakage issues, and suggest potential directions for future research. In the

Overall, LLM-based APR has rapidly emerged over the past few years, reflecting growing interest across diverse research communities (e.g., software engineering, artificial intelligence, and programming languages) and suggesting that many promising directions remain largely unexplored. We conclude by discussing three complementary perspectives that not only appear particularly promising for future research but also shaped the way we structured this survey. From the repair perspective, future work should explore broader repair scenarios and further evaluate how LLMs can effectively assist developers in real-world repair environments. From the integration perspective, future research could design more advanced strategies to address emerging challenges (e.g., enabling long-term memory and repository-level reasoning), and establish comprehensive benchmark suites to systematically evaluate diverse repair capabilities across different settings.

Acknowledgments

This research was supported in part by the Australian Research Council Discovery Projects under Grant No. DP200102491 and DP230101790, the National Natural Science Foundation of China (61932012, 62141215, 62372228), Natural Science Foundation of Jiangsu Province (BK20251458), and Fundamental Research Funds for the Central Universities (AE89991/463).

References

- [1] Baleegh Ahmad, Shailja Thakur, Benjamin Tan, Ramesh Karri, and Hammond Pearce. 2024. On hardware security bug code fixes by prompting large language models. *IEEE Transactions on Information Forensics and Security* 19 (2024), 4043–4057.
- [2] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified Pre-Training for Program Understanding and Generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2655–2668.
- [3] Toufique Ahmed, Noah Rose Ledesma, and Premkumar Devanbu. 2022. SynShine: Improved Fixing of Syntax Errors. *IEEE Transactions on Software Engineering* 49, 4 (2022), 2169–2181.
- [4] Saranya Alagarsamy, Chakkrit Tantithamthavorn, and Aldeida Aleti. 2023. A3test: Assertion-Augmented Automated Test Case Generation. *arXiv preprint arXiv:2302.10352* (2023).
- [5] Mohannad Alhanahnah, Md Rashedul Hasan, Lisong Xu, and Hamid Bagheri. 2025. An empirical evaluation of pre-trained large language models for repairing declarative formal specifications. *Empirical Software Engineering* 30, 5 (2025), 1–38.
- [6] Kamel Alrashedy and Abdullah Aljasser. 2023. Can LLMs Patch Security Issues? *arXiv preprint arXiv:2312.00024* (2023).
- [7] Antonis Antoniadis, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. 2024. SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement. *arXiv preprint arXiv:2410.20285* (2024).
- [8] John Anvik, Lyndon Hiew, and Gail C Murphy. 2005. Coping with an Open Bug Repository. In *Proceedings of the 2005 OOPSLA workshop on Eclipse Technology eXchange*. 35–39.
- [9] Daman Arora, Atharv Sonwane, Nalin Wadhwa, Abhav Mehrotra, Saiteja Utpala, Ramakrishna Bairi, Aditya Kanade, and Nagarajan Natarajan. 2024. Masai: Modular architecture for software-engineering ai agents. *arXiv preprint arXiv:2406.11638* (2024).
- [10] Charles Arthur and Phillip Inman. 2013. The Error That Could Subvert George Osborne's Austerity Programme. <https://www.theguardian.com/politics/2013/apr/18/uncovered-error-george-osborne-austerity/>. Lasted accessed: 2025-10-01.
- [11] Anonymous Author(s). 2024. Replication Package. <https://anonymous.4open.science/r/RetriGen/>. Lasted accessed: 2024-04-01.
- [12] Bandit. 2024. A Static tool to Find Common Security Issues in Python Code. URL: <https://github.com/PyCQA/bandit>. Lasted accessed: 2024-04-01.
- [13] Earl T Barr, Yuriy Brun, Premkumar Devanbu, Mark Harman, and Federica Sarro. 2014. The plastic surgery hypothesis. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 306–317.
- [14] Earl T Barr, Mark Harman, Yue Jia, Alexandru Marginean, and Justyna Petke. 2015. Automated software transplantation. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis*. 257–269.
- [15] Berkay Berabi, Jingxuan He, Veselin Raychev, and Martin Vechev. 2021. TFix: Learning to Fix Coding Errors with a Text-to-Text Transformer. In *International Conference on Machine Learning*. PMLR, 780–791.
- [16] Guru Bhandari, Amara Naseer, and Leon Moonen. 2021. CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-source Software. In *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*. 30–39.
- [17] Sidney Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, et al. 2022. GPT-Neox-20b: An Open-Source Autoregressive Language Model. In *Proceedings of BigScience Episode# 5-Workshop on Challenges & Perspectives in Creating Large Language Models*. 95–136.
- [18] Sid Black, Leo Gao, Phil Wang, Connor Leahy, and Stella Biderman. 2021. GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow. URL: <https://doi.org/10.5281/zenodo.5297715>. Lasted accessed: 2024-04-01.
- [19] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. *arXiv preprint arXiv:2403.17134* (2024).
- [20] Ricardo Brancas, Vasco Manquinho, and Ruben Martins. 2025. Combining Logic and Large Language Models for Assisted Debugging and Repair of ASP Programs. In *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 646–657.
- [21] Tom Britton, Lisa Jeng, Graham Carver, Paul Cheak, and Tomer Katzenellenbogen. 2013. Reversible Debugging Software: Quantify the Time and Cost Saved Using Reversible Debuggers. *Judge Bus. School, Univ. Cambridge, Cambridge, UK, Tech. Rep* 229 (2013).
- [22] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, Vol. 33. 1877–1901.
- [23] Cambridge Judge Business School. 2013. Research by Cambridge MBAs for Tech Firm Undo Finds Software Bugs Cost the Industry \$316 Billion a Year. <https://www.jbs.cam.ac.uk/2013/research-by-cambridge-mbas-for-tech-firm-undo-finds-software-bugs-cost-the-industry-316-billion-a-year/>. Lasted accessed: 2025-10-01.
- [24] Jialun Cao, Meiziniu Li, Ming Wen, and Shing-chi Cheung. 2023. A Study on Prompt Design, Advantages and Limitations of ChatGPT for Deep Learning Program Repair. *arXiv preprint arXiv:2304.08191* (2023).

- [25] Yiannis Charalambous, Norbert Tihanyi, Ridhi Jain, Youcheng Sun, Mohamed Amine Ferrag, and Lucas C Cordeiro. 2023. A New Era in Software Security: Towards Self-Healing Software Via Large Language Models and Formal Verification. *arXiv preprint arXiv:2305.14752* (2023).
- [26] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, et al. 2024. Coder: Issue resolving with multi-agent and task graphs. *arXiv preprint arXiv:2406.01304* (2024).
- [27] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
- [28] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching Large Language Models to Self-Debug. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=KuPixIqPiq>
- [29] Yuxiao Chen, Jingzheng Wu, Xiang Ling, Changjiang Li, Zhiqing Rui, Tianyue Luo, and Yanjun Wu. 2024. When Large Language Models Confront Repository-Level Automatic Program Repair: How Well They Done?. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 459–471.
- [30] Zimin Chen, Steve Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2019. Sequencer: Sequence-To-Sequence Learning for End-To-End Program Repair. *IEEE Transactions on Software Engineering* 47, 9 (2019), 1943–1959.
- [31] Yiu Wai Chow, Luca Di Grazia, and Michael Pradel. 2024. PyTy: Repairing Static Type Errors in Python. In *Proceedings of the 46th International Conference on Software Engineering*. 871–871.
- [32] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2023. PaLM: Scaling Language Modeling with Pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.
- [33] Zhenlong Dai, Bingrui Chen, Zhuoluo Zhao, Xiu Tang, Sai Wu, Chang Yao, Zhipeng Gao, and Jingyuan Chen. 2025. Less is More: Adaptive Program Repair with Bug Localization and Preference Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 128–136.
- [34] David de Fitero-Dominguez, Eva Garcia-Lopez, Antonio Garcia-Cabot, and Jose-Javier Martinez-Herraz. 2024. Enhanced Automated Code Vulnerability Repair Using Large Language Models. *arXiv preprint arXiv:2401.03741* (2024).
- [35] Pantazis Deligiannis, Akash Lal, Nikita Mehrotra, and Aseem Rastogi. 2023. Fixing Rust Compilation Errors Using Llms. *arXiv preprint arXiv:2308.05177* (2023).
- [36] Richard A DeMillo, Richard J Lipton, and Frederick G Sayward. 2006. Hints on Test Data Selection: Help for the Practicing Programmer. *Computer* 11, 4 (2006), 34–41.
- [37] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT’19)*. Association for Computational Linguistics, 4171–4186.
- [38] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K Lahiri. 2022. TOGA: A Neural Method for Test Oracle Generation. In *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2130–2141.
- [39] Ning Ding, Yufia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. 2023. Parameter-Efficient Fine-Tuning of Large-Scale Pre-Trained Language Models. *Nature Machine Intelligence* 5, 3 (2023), 220–235.
- [40] Yangruibo Ding, Marcus J Min, Gail Kaiser, and Baishakhi Ray. 2024. CYCLE: Learning to Self-Refine the Code Generation. *arXiv preprint arXiv:2403.18746* (2024).
- [41] Tung Do Viet and Konstantin Markov. 2023. Using Large Language Models for Bug Localization and Fixing. In *2023 12th International Conference on Awareness Science and Technology*. IEEE, 192–197.
- [42] Dawn Drain, Colin B Clement, Guillermo Serrato, and Neel Sundaresan. 2021. DeepDebug: Fixing Python Bugs Using Stack Traces, Backtranslation, and Code Skeletons. *arXiv preprint arXiv:2105.09352* (2021).

- [43] Dawn Drain, Chen Wu, Alexey Svyatkovskiy, and Neel Sundaresan. 2021. Generating Bug-Fixes Using Pretrained Transformers. In *Proceedings of the 5th ACM SIGPLAN International Symposium on Machine Programming*. 1–8.
- [44] Xueying Du, Mingwei Liu, Juntao Li, Hanlin Wang, Xin Peng, and Yiling Lou. 2023. Resolving Crash Bugs Via Large Language Models: An Empirical Study. *arXiv preprint arXiv:2312.10448* (2023).
- [45] Thomas Durieux and Martin Monperrus. 2016. DynaMoth: Dynamic Code Synthesis for Automatic Program Repair. In *Proceedings of the 11th International Workshop on Automation of Software Test*. 85–91.
- [46] Ramtin Ehsani, Esteban Parra, Sonia Haiduc, and Preetha Chatterjee. 2025. Bug Fixing with Broader Context: Enhancing LLM-Based Program Repair via Layered Knowledge Injection. *arXiv preprint arXiv:2506.24015* (2025).
- [47] Ramtin Ehsani, Sakshi Pathak, and Preetha Chatterjee. 2025. Towards detecting prompt knowledge gaps for improved llm-guided issue resolution. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 699–711.
- [48] Sarah Fakhoury, Saikat Chakraborty, Madan Musuvathi, and Shuvendu K Lahiri. 2023. Towards Generating Functionally Correct Code Edits from Natural Language Issue Descriptions. *arXiv preprint arXiv:2304.03816* (2023).
- [49] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. *arXiv preprint arXiv:2310.03533* (2023).
- [50] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories*. 508–512.
- [51] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated Repair of Programs from Large Language Models. In *2023 IEEE/ACM 45th International Conference on Software Engineering*. IEEE, 1469–1481.
- [52] Sakina Fatima, Hadi Hemmati, and Lionel C Briand. 2024. FlakyFix: Using large language models for predicting flaky test fix categories and test code repair. *IEEE Transactions on Software Engineering* 50, 12 (2024), 3146–3171.
- [53] Qiong Feng, Xiaotian Ma, Jiayi Sheng, Ziyuan Feng, Wei Song, and Peng Liang. 2025. Integrating Various Software Artifacts for Better LLM-based Bug Localization and Program Repair. *ACM Transactions on Software Engineering and Methodology* (2025).
- [54] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics*. 1536–1547.
- [55] Emily First, Markus Rabe, Talia Ringer, and Yuriy Brun. 2023. Baldur: Whole-proof generation and repair with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1229–1241.
- [56] Elias Frantar and Dan-Adrian Alistarh. 2023. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 10323–10337.
- [57] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=hQwb-lbM6EL>
- [58] Michael Fu, Van Nguyen, Chakkrit Tantithamthavorn, Dinh Phung, and Trung Le. 2024. Vision Transformer-Inspired Automated Vulnerability Repair. *ACM Transactions on Software Engineering and Methodology* 33, 3 (2024), 1–29.
- [59] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Phung Dinh. 2022. VulRepair: A T5-Based Automated Software Vulnerability Repair. In *the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 935–947.
- [60] Weimin Fu, Kaichen Yang, Raj Gautam Dutta, Xiaolong Guo, and Gang Qu. 2023. LLM4SecHW: Leveraging domain-specific large language model for hardware debugging. In *2023 Asian Hardware Oriented Security and Trust Symposium (AsianHOST)*. IEEE, 1–6.
- [61] Amiao Gao, Zenong Zhang, Simin Wang, Liguang Huang, Shiyi Wei, and Vincent Ng. 2025. Teaching AI the ‘Why’ and ‘How’ of Software Vulnerability Fixes. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2006–2029.
- [62] Shuzheng Gao, Xin-Cheng Wen, Cuiyun Gao, Wenxuan Wang, Hongyu Zhang, and Michael R Lyu. 2023. What Makes Good In-Context Demonstrations for Code Intelligence Tasks with LLMs?. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 761–773.
- [63] Spandan Garg, Roshanak Zilouchian Moghaddam, Colin B Clement, Neel Sundaresan, and Chen Wu. 2022. DeepDev-PERF: A Deep Learning-Based Approach for Improving Software Performance. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 948–958.
- [64] Spandan Garg, Roshanak Zilouchian Moghaddam, and Neel Sundaresan. 2023. Rapgen: An Approach for Fixing Code Inefficiencies in Zero-Shot. *arXiv preprint arXiv:2306.17077* (2023).
- [65] Vahid Garousi, Michael Felderer, and Mika V Mäntylä. 2019. Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and software technology* 106 (2019), 101–121.
- [66] Luca Gazzola, Daniela Micucci, and Leonardo Mariani. 2019. Automatic Software Repair: A Survey. *IEEE Transactions on Software Engineering* 45, 1 (2019), 34–67.

- [67] Haotong Ge and Yuemeng Wu. 2023. An Empirical Study of Adoption of ChatGPT for Bug Fixing among Professional Developers. *Innovation & Technology Advances* 1, 1 (Jun. 2023), 21–29. <https://doi.org/10.61187/ita.v1i1.19>
- [68] Reza Gharibi, Mohammad Hadi Sadreddini, and Seyed Mostafa Fakhrahmad. 2024. T5APR: Empowering automated program repair across languages through checkpoint ensemble. *Journal of Systems and Software* 214 (2024), 112083.
- [69] Google. 2025. Google Bug Hunters. <https://bughunters.google.com/>. Lasted accessed: 2025-10-01.
- [70] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7212–7225.
- [71] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin B. Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *Proceedings of the 9th International Conference on Learning Representations*. 1–18.
- [72] Jiawei Guo, Ziming Li, Xueling Liu, Kaijing Ma, Tianyu Zheng, Zhouliang Yu, Ding Pan, Yizhi Li, Ruibo Liu, Yue Wang, et al. 2024. CodeEditorBench: Evaluating Code Editing Capability of Large Language Models. *arXiv preprint arXiv:2404.03543* (2024).
- [73] Lianghong Guo, Wei Tao, Runhan Jiang, Yanlin Wang, Jiachi Chen, Xilin Liu, Yuchi Ma, Mingzhi Mao, Hongyu Zhang, and Zibin Zheng. 2025. OmniGIRL: A Multilingual and Multimodal Benchmark for GitHub Issue Resolution. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 24–46.
- [74] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2023. Exploring the Potential of ChatGPT in Automated Code Refinement: An Empirical Study. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 379–391.
- [75] C. Hait and G. Tasse. 2002. *The Economic Impacts of Inadequate Infrastructure for Software Testing*. DIANE Publishing Company. <https://books.google.com/books?id=juSgPAAACAAJ>
- [76] Sichong Hao, Xianjun Shi, and Hongwei Liu. 2024. Exploring the Potential of Pre-Trained Language Models of Code for Automated Program Repair. *Electronics* 13, 7 (2024), 1200.
- [77] Sichong Hao, Xianjun Shi, and Hongwei Liu. 2024. RetypeR: Integrated Retrieval-based Automatic Program Repair for Python Type Errors. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 199–210.
- [78] Sichong Hao, Xianjun Shi, Hongwei Liu, and Yanjun Shu. 2023. Enhancing Code Language Models for Program Repair by Curricular Fine-tuning Framework. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 136–146.
- [79] Md Mahim Anjum Haque, Wasi Uddin Ahmad, Ismini Lourentzou, and Chris Brown. 2023. FixEval: Execution-Based Evaluation of Program Fixes for Programming Problems. In *2023 IEEE/ACM International Workshop on Automated Program Repair*. IEEE, 11–18.
- [80] Dávid Hidvégi, Khashayar Etemadi, Sofia Bobadilla, and Martin Monperrus. 2024. CigaR: Cost-efficient Program Repair with LLMs. *arXiv preprint arXiv:2402.06598* (2024).
- [81] Dániel Horváth, Viktor Csúvik, Tibor Gyimóthy, and László Vidács. 2023. An Extensive Study on Model Architecture and Program Representation in the Domain of Learning-Based Automated Program Repair. In *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*. 31–38.
- [82] Soneya Binta Hossain, Nan Jiang, Qiang Zhou, Xiaopeng Li, Wen-Hao Chiang, Yingjun Lyu, Hoan Nguyen, and Omer Tripp. 2024. A Deep Dive into Large Language Models for Automated Bug Localization and Repair. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1471–1493.
- [83] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2023. Large Language Models for Software Engineering: A Systematic Literature Review. *arXiv preprint arXiv:2308.10620* (2023).
- [84] Haichuan Hu, Congqing He, Hao Zhang, Xiaochen Xie, and Qunjun Zhang. 2025. APRMCTS: Improving LLM-based Automated Program Repair with Iterative Tree Search. *arXiv preprint arXiv:2507.01827* (2025).
- [85] Xueyu Hu, Kun Kuang, Jiankai Sun, Hongxia Yang, and Fei Wu. 2024. Leveraging Print Debugging to Improve Code Generation in Large Language Models. *arXiv preprint arXiv:2401.05319* (2024).
- [86] Xing Hu, Zhuang Liu, Xin Xia, Zhongxin Liu, Tongtong Xu, and Xiaohu Yang. 2023. Identify and Update Test Cases When Production Code Changes: A Transformer-Based Approach. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1111–1122.
- [87] Yang Hu, Umair Z Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 388–398.
- [88] Yaojie Hu, Xingjian Shi, Qiang Zhou, and Lee Pike. 2022. Fix Bugs with Transformer through a Neural-Symbolic Edit Grammar. In *Deep Learning for Code Workshop*. https://openreview.net/forum?id=SBgE6i_WkZq
- [89] Dong Huang, Qingwen Bu, Jie M Zhang, Michael Luck, and Heming Cui. 2023. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. *arXiv preprint arXiv:2312.13010* (2023).
- [90] Kai Huang, Jian Zhang, Xinlei Bao, Xu Wang, and Yang Liu. 2025. Comprehensive Fine-Tuning Large Language Models of Code for Automated Program Repair. *IEEE Transactions on Software Engineering* 51, 4 (2025), 904–928.

- [91] Kai Huang, Jian Zhang, Xiangxin Meng, and Yang Liu. 2025. Template-Guided Program Repair in the Era of Large Language Models.. In *ICSE*. 1895–1907.
- [92] Kai Huang, Jian Zhang, Xiaofei Xie, and Chunyang Chen. 2025. Seeing is Fixing: Cross-Modal Reasoning with Multimodal LLMs for Visual Software Issue Fixing. *arXiv preprint arXiv:2506.16136* (2025).
- [93] Qing Huang, Jiahui Zhu, Zhenchang Xing, Huan Jin, Changjing Wang, and Xiwei Xu. 2023. A Chain of AI-Based Solutions for Resolving Fqns and Fixing Syntax Errors in Partial Code. *arXiv preprint arXiv:2306.11981* (2023).
- [94] Ryosuke Ishizue, Kazunori Sakamoto, Hironori Washizaki, and Yoshiaki Fukazawa. 2024. Improved Program Repair Methods using Refactoring with GPT Models. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. 569–575.
- [95] Jiajun Jiang, Yingfei Xiong, Hongyu Zhang, Qing Gao, and Xiangqun Chen. 2018. Shaping Program Repair Space with Existing Patches and Similar Code. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 298–309.
- [96] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *Proceedings of the 45th International Conference on Software Engineering*. 1430–1442.
- [97] Nan Jiang, Thibaud Lutellier, and Lin Tan. 2021. CURE: Code-Aware Neural Machine Translation for Automatic Program Repair. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering*. 1161–1173.
- [98] Shuyang Jiang, Yuhao Wang, and Yu Wang. 2023. SelfEvolve: A Code Evolution Framework via Large Language Models. *arXiv preprint arXiv:2306.02907* (2023).
- [99] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).
- [100] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQm66>
- [101] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. InferFix: End-to-End Program Repair with LLMs. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 1646–1656.
- [102] Harshit Joshi, José Cambronero Sanchez, Sumit Gulwani, Vu Le, Gust Verbruggen, and Ivan Radiček. 2023. Repair Is Nearly Generation: Multilingual Program Repair with LLMs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 5131–5140.
- [103] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. In *Proceedings of the 23rd International Symposium on Software Testing and Analysis*. 437–440.
- [104] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. 2023. Explainable Automated Debugging Via Large Language Model-Driven Scientific Debugging. *arXiv preprint arXiv:2304.02195* (2023).
- [105] Rafael-Michael Karampatsis and Charles Sutton. 2020. How Often Do Single-Statement Bugs Occur? The Manysstubs4j Dataset. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR’20)*. 573–577.
- [106] Kaiyao Ke. 2025. NIODebugger: A Novel Approach to Repair Non-Idempotent-Outcome Tests with LLM-Based Agent. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 762–762.
- [107] Misoo Kim, Youngkyoung Kim, Hohyeon Jeong, Jinseok Heo, Sungoh Kim, Hyunhee Chung, and Eunseok Lee. 2022. An Empirical Study of Deep Transfer Learning-Based Program Repair for Kotlin Projects. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1441–1452.
- [108] Barbara Kitchenham, Lech Madeyski, and David Budgen. 2023. SEGRESS: Software Engineering Guidelines for REporting Secondary Studies. *IEEE Transactions on Software Engineering* 49, 3 (2023), 1273–1298.
- [109] Barbara Ann Kitchenham and Stuart Charters. 2007. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. Technical Report EBSE 2007-001. Keele University and Durham University Joint Report. 1–65 pages.
- [110] Sophia D Kolak, Ruben Martins, Claire Le Goues, and Vincent Josua Hellendoorn. 2022. Patch Generation with Language Models: Feasibility and Scaling Behavior. In *International Conference on Learning Representations Deep Learning for Code Workshop*.
- [111] Jiaolong Kong, Mingfei Cheng, Xiaofei Xie, Shangqing Liu, Xiaoning Du, and Qi Guo. 2024. ContrastRepair: Enhancing Conversation-Based Automated Program Repair Via Contrastive Test Case Pairs. *arXiv preprint arXiv:2403.01971* (2024).
- [112] Jiaolong Kong, Xiaofei Xie, and Shangqing Liu. 2025. Demystifying Memorization in LLM-Based Program Repair via a General Hypothesis Testing Framework. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2712–2734.
- [113] Charles Koutchme. 2023. Training Language Models for Programming Feedback Using Automated Repair Tools. In *International Conference on Artificial Intelligence in Education*. Springer, 830–835.
- [114] Charles Koutchme, Sami Sarsa, Juho Leinonen, Arto Hellas, and Paul Denny. 2023. Automated Program Repair Using Generative Models for Code Infilling. In *International Conference on Artificial Intelligence in Education*. Springer, 798–803.
- [115] Anil Koyuncu, Kui Liu, Tegawendé F Bissyandé, Dongsun Kim, Jacques Klein, Martin Monperrus, and Yves Le Traon. 2020. FixMiner: Mining Relevant Fix Patterns for Automated Program Repair. *Empirical Software Engineering* 25, 3 (2020), 1980–2024.
- [116] Ummay Kulsum, Haotian Zhu, Bowen Xu, and Marcelo d’Amorim. 2024. A Case Study of LLM for Automated Vulnerability Repair: Assessing Impact of Reasoning and Patch Validation Feedback. In *Proceedings of the 1st ACM International Conference on AI-Powered*

- Software*. 103–111.
- [117] Márk Lajkó, Viktor Csuik, and László Vidács. 2022. Towards Javascript Program Repair with Generative Pre-trained Transformer. In *2022 IEEE/ACM International Workshop on Automated Program Repair*. IEEE, 61–68.
 - [118] Tan Khang Le, Saba Alimadadi, and Steven Y Ko. 2024. A Study of Vulnerability Repair in JavaScript Programs with Large Language Models. *arXiv preprint arXiv:2403.13193* (2024).
 - [119] Thanh Le-Cong, Duc-Minh Luong, Xuan Bach D Le, David Lo, Nhat-Hoa Tran, Bui Quang-Huy, and Quyet-Thang Huynh. 2023. Invalidator: Automated Patch Correctness Assessment Via Semantic and Syntactic Reasoning. *IEEE Transactions on Software Engineering* 49, 06 (2023), 3411–3429.
 - [120] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Transactions on Software Engineering* 38, 01 (2012), 54–72.
 - [121] Cheryl Lee, Chunqiu Steven Xia, Longji Yang, Jen-tse Huang, Zhourixin Zhu, Lingming Zhang, and Michael R Lyu. 2024. A unified debugging approach via llm-based multi-agent synergy. *arXiv preprint arXiv:2404.17153* (2024).
 - [122] Aitor Lewkowycz, Anders Johan Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Venkatesh Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. 2022. Solving Quantitative Reasoning Problems with Language Models. In *Advances in Neural Information Processing Systems*. <https://openreview.net/forum?id=IFXTZERXDM7>
 - [123] Fengjie Li, Jiajun Jiang, Jiajun Sun, and Hongyu Zhang. 2025. Hybrid automated program repair by combining large language models and program analysis. *ACM Transactions on Software Engineering and Methodology* 34, 7 (2025), 1–28.
 - [124] Guochang Li, Chen Zhi, Jialiang Chen, Junxiao Han, and Shuiguang Deng. 2024. Exploring Parameter-Efficient Fine-Tuning of Large Language Model on Automated Program Repair. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 719–731.
 - [125] Jia Li, Yongmin Li, Ge Li, Zhi Jin, Yiyang Hao, and Xing Hu. 2023. SkCoder: A Sketch-based Approach for Automatic Code Generation. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2124–2135.
 - [126] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy V, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Moustafa-Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2023. StarCoder: May the Source be With You! *arXiv preprint arXiv:2305.06161* (2023).
 - [127] Yi Li, Shaohua Wang, and Tien N Nguyen. 2020. DLFix: Context-based Code Transformation Learning for Automated Program Repair. In *Proceedings of the 42nd ACM/IEEE International Conference on Software Engineering*. 602–614.
 - [128] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2022. DEAR: A Novel Deep Learning-based Approach for Automated Program Repair. In *Proceedings of the 44th International Conference on Software Engineering*. 511–523.
 - [129] Yixiao Li, Yifan Yu, Chen Liang, Nikos Karampatziakis, Pengcheng He, Weizhu Chen, and Tuo Zhao. 2024. LoftQ: LoRA-Fine-Tuning-aware Quantization for Large Language Models. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=LzPWWPAdY4>
 - [130] Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu, and Neel Sundaresan. 2022. Automating Code Review Activities by Large-Scale Pre-Training. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1035–1047.
 - [131] Jingjing Liang, Ruyi Ji, Jiajun Jiang, Shurui Zhou, Yiling Lou, Yingfei Xiong, and Gang Huang. 2021. Interactive Patch Filtering as Debugging Aid. In *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 239–250.
 - [132] Ben Liblit, Mayur Naik, Alice X Zheng, Alex Aiken, and Michael I Jordan. 2005. Scalable Statistical Bug Isolation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*. 15–26.
 - [133] Bo Lin, Shangwen Wang, Ming Wen, Liqian Chen, and Xiaoguang Mao. 2024. One Size Does Not Fit All: Multi-granularity Patch Generation for Better Automated Program Repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1554–1566.
 - [134] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. QuixBugs: A Multi-Lingual Program Repair Benchmark Set Based on the Quixey Challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion’17)*. 55–56.
 - [135] Yuanfei Lin, Chenran Li, Mingyu Ding, Masayoshi Tomizuka, Wei Zhan, and Matthias Althoff. 2024. DrPlanner: Diagnosis and Repair of Motion Planners Using Large Language Models. *arXiv preprint arXiv:2403.07470* (2024).

- [136] Zhihao Lin, Mingyi Zhou, Wei Ma, Chi Chen, Yun Yang, Jun Wang, Chunming Hu, and Li Li. 2025. HapRepair: Learn to Repair OpenHarmony Apps. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 319–330.
- [137] Changshu Liu, Pelin Cetin, Yogesh Patodia, Saikat Chakraborty, Yangruibo Ding, and Baishakhi Ray. 2023. Automated Code Editing with Search-Generate-Modify. *arXiv preprint arXiv:2306.06490* (2023).
- [138] Fang Liu, Zhenwei Liu, Qianhui Zhao, Jing Jiang, Li Zhang, Zian Sun, Ge Li, Zhongqi Li, and Yuchi Ma. 2024. FastFixer: an efficient and effective approach for repairing programming assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 669–680.
- [139] Haoran Liu, Shanshan Li, Zhouyang Jia, Yuanliang Zhang, Linxiao Bai, Si Zheng, Xiaoguang Mao, and Xiangke Liao. 2025. Error Delayed Is Not Error Handled: Understanding and Fixing Propagated Error-Handling Bugs. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2571–2593.
- [140] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F Bissyandé. 2019. Avatar: Fixing Semantic Bugs with Fix Patterns of Static Analysis Violations. In *Proceedings of the 26th IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, 1–12.
- [141] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F Bissyandé. 2019. TBar: Revisiting Template-based Automated Program Repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 31–42.
- [142] Mingjie Liu, Yun-Da Tsai, Wenfei Zhou, and Haoxing Ren. 2024. CraftRTL: High-quality Synthetic Data Generation for Verilog Code Models with Correct-by-Construction Non-Textual Representations and Targeted Code Repair. *arXiv preprint arXiv:2409.12993* (2024).
- [143] Peiyu Liu, Junming Liu, Lirong Fu, Kangjie Lu, Yifan Xia, Xuhong Zhang, Wenzhi Chen, Haiqin Weng, Shouling Ji, and Wenhai Wang. 2023. How ChatGPT Is Solving Vulnerability Management Problem. *arXiv preprint arXiv:2311.06530* (2023).
- [144] Yizhou Liu, Pengfei Gao, Xincheng Wang, Jie Liu, Yexuan Shi, Zhao Zhang, and Chao Peng. 2024. MarsCode Agent: AI-native Automated Bug Fixing. *arXiv preprint arXiv:2409.00899* (2024).
- [145] Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D. Le, and David Lo. 2024. Refining ChatGPT-Generated Code: Characterizing and Mitigating Code Quality Issues. *ACM Transactions on Software Engineering and Methodology* (2024). <https://doi.org/10.1145/3643674> Just Accepted.
- [146] Vadim Liventsev, Anastasiia Grishina, Aki Härmä, and Leon Moonen. 2023. Fully Autonomous Programming with Large Language Models. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '23)*. 1146–1155.
- [147] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1*.
- [148] Wenqiang Luo, Jacky Keung, Boyang Yang, He Ye, Claire Le Goues, Tegawende F Bissyande, Haoye Tian, and Xuan Bach D Le. 2024. When Fine-Tuning LLMs Meets Data Privacy: An Empirical Study of Federated Learning in LLM-Based Program Repair. *ACM Transactions on Software Engineering and Methodology* (2024).
- [149] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. CoCoNut: Combining Context-Aware Neural Translation Models Using Ensemble for Program Repair. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 101–114.
- [150] Michael R Lyu, Baishakhi Ray, Abhik Roychoudhury, Shin Hwei Tan, and Patanamorn Thongtanunam. 2024. Automatic Programming: Large Language Models and Beyond. *arXiv preprint arXiv:2405.02213* (2024).
- [151] Yingwei Ma, Rongyu Cao, Yongchang Cao, Yue Zhang, Jue Chen, Yibo Liu, Yuchen Liu, Binhua Li, Fei Huang, and Yongbin Li. 2025. SWE-GPT: A Process-Centric Language Model for Automated Software Improvement. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 2362–2383.
- [152] Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. 2025. Alibaba lingmaagent: Improving automated issue resolution via comprehensive repository exploration. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 238–249.
- [153] Roman Macháček, Anastasiia Grishina, Max Hort, and Leon Moonen. 2025. The Impact of Fine-tuning Large Language Models on Automated Program Repair. *arXiv preprint arXiv:2507.19909* (2025).
- [154] Callum MacNeil. 2023. *A Systematic Review of Automated Program Repair using Large Language Models*. Bachelor of Computer Science Honours Thesis. Dalhousie University, Halifax, Nova Scotia, Canada. <https://raise.cs.dal.ca/thesis/CallumBCS2023.pdf>
- [155] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=S37hOerQLB>
- [156] Neural Magic. 2023. SparseML: Libraries for Applying Sparsification Recipes to Neural Networks. <https://github.com/neuralmagic/sparseml>. Accessed: 2025-10-01.

- [157] Matias Martinez and Martin Monperrus. 2016. ASTOR: A Program Repair Library for Java. In *Proceedings of the 25th International Symposium on Software Testing and Analysis*. 441–444.
- [158] Matias Martinez and Martin Monperrus. 2018. Ultra-Large Repair Search Space with Automatically Mined Templates: The Cardumen Mode of Astor. In *Proceedings of the International Symposium on Search Based Software Engineering (SSBSE'18)*. Springer, 65–86.
- [159] Ehsan Mashhadi and Hadi Hemmati. 2021. Applying Codebert for Automated Program Repair of Java Simple Bugs. In *Proceedings Companion of the 18th IEEE/ACM International Conference on Mining Software Repositories (MSR'21)*. 505–509.
- [160] Fairuz Nawer Meem, Justin Smith, and Brittany Johnson. 2024. Exploring Experiences with Automated Program Repair in Practice. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 870–870.
- [161] Mistral. 2024. Mistral-7B. URL: <https://mistral.ai/news/announcing-mistral-7b/>. Lasted accessed: 2024-04-01.
- [162] Mohammad Mahdi Mohajer, Reem Aleithan, Nima Shiri Harzevili, Moshi Wei, Alvine Boaye Belle, Hung Viet Pham, and Song Wang. 2023. Skipanalyzer: An Embodied Agent for Code Analysis with Large Language Models. *arXiv preprint arXiv:2310.18532* (2023).
- [163] Martin Monperrus. 2018. Automatic Software Repair: A Bibliography. *Comput. Surveys* 51, 1 (2018), 1–24.
- [164] Martin Monperrus. 2022. *The Living Review on Automated Program Repair*. Technical Report hal-01956501. HAL/archives-ouvertes.fr.
- [165] Seungjun Moon, Yongho Song, Hyungjoo Chae, Dongjin Kang, Taeyoon Kwon, Kai Tzu-iunn Ong, Seung-won Hwang, and Jinyoung Yeo. 2023. Coffee: Boost Your Code Llm's by Fixing Bugs with Feedback. *arXiv preprint arXiv:2311.07215* (2023).
- [166] Mozilla. 2025. Mozilla Security Bug Bounty Program. <https://www.mozilla.org/en-US/security/bug-bounty/>. Lasted accessed: 2025-10-01.
- [167] Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. 2025. EXPEREPAIR: Dual-Memory Enhanced LLM-based Repository-Level Program Repair. *arXiv preprint arXiv:2506.10484* (2025).
- [168] Niels Münder, Mark Müller, Jingxuan He, and Martin Vechev. 2024. SWT-bench: Testing and validating real-world bug-fixes with code agents. *Advances in Neural Information Processing Systems* 37 (2024), 81857–81887.
- [169] Emanuele Antonio Napoli and Valentina Gatteschi. 2023. Evaluating ChatGPT for Smart Contracts Vulnerability Correction. In *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 1828–1833.
- [170] Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning. In *Proceedings of the 45th International Conference on Software Engineering*. IEEE, 2450–2462.
- [171] Roberto Natella, Domenico Cotroneo, and Henrique S Madeira. 2016. Assessing Dependability with Software Fault Injection: A Survey. *ACM Computing Surveys (CSUR)* 48, 3 (2016), 1–55.
- [172] Neural Magic. 2024. DeepSparse: A Sparsity-Aware Neural Network Inference Engine for CPUs. <https://github.com/neuralmagic/deepsparse>. Accessed: 2025-10-01.
- [173] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=iaYcJKpY2B>.
- [174] Yu Nong, Mohammed Aldeen, Long Cheng, Hongxin Hu, Feng Chen, and Haipeng Cai. 2024. Chain-of-Thought Prompting of Large Language Models for Discovering and Fixing Software Vulnerabilities. *arXiv preprint arXiv:2402.17230* (2024).
- [175] Yu Nong, Haoran Yang, Long Cheng, Hongxin Hu, and Haipeng Cai. 2025. {APPATCH}: Automated Adaptive Prompting Large Language Models for {Real-World} Software Vulnerability Patching. In *34th USENIX Security Symposium (USENIX Security 25)*. 4481–4500.
- [176] A Jefferson Offutt and Stephen D Lee. 1994. An Empirical Evaluation of Weak Mutation. *IEEE Transactions on Software Engineering* 20, 5 (1994), 337–344.
- [177] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2024. Is Self-Repair a Silver Bullet for Code Generation?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=y0GJXRungR>.
- [178] Safwan Omari, Kshitiz Basnet, and Mohammad Wardat. 2024. Investigating large language models capabilities for automatic code repair in python. *Cluster Computing* 27, 8 (2024), 10717–10731.
- [179] OpenAI. 2022. GPT-3.5. URL: <https://platform.openai.com/docs/models/gpt-3-5>. Lasted accessed: 2024-04-01.
- [180] OpenAI. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).
- [181] Pedro Orvalho, Mikoláš Janota, and Vasco M Manquinho. 2025. Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 649–657.
- [182] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2024. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. *arXiv preprint arXiv:2410.14684* (2024).
- [183] Shuyin Ouyang, Jie M Zhang, Zeyu Sun, and Albert Merono Penuela. 2025. Knowledge-Enhanced Program Repair for Data Science Code. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 898–910.
- [184] Yicheng Ouyang, Jun Yang, and Lingming Zhang. 2024. Benchmarking automated program repair: An extensive study on both real-world and artificial bugs. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 440–452.
- [185] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in Translation: A Study of Bugs Introduced by Large Language Models

- While Translating Code. In *2024 IEEE/ACM 46th International Conference on Software Engineering*. IEEE Computer Society, 866–866.
- [186] Nikhil Parasaram, Huijie Yan, Boyu Yang, Zineb Flahy, Abriele Qudsi, Damian Ziaber, Earl T Barr, and Sergey Mechtaev. 2025. The Fact Selection Problem in LLM-Based Program Repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2574–2586.
 - [187] Rishov Paul, Md Mohib Hossain, Mohammed Latif Siddiq, Masum Hasan, Anindya Iqbal, and Joanna CS Santos. 2023. Enhancing Automated Program Repair through Fine-Tuning and Prompt Engineering. *arXiv preprint arXiv:2304.07840* (2023).
 - [188] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. 2023. Examining Zero-Shot Vulnerability Repair with Large Language Models. In *2023 IEEE Symposium on Security and Privacy*. IEEE, 2339–2356.
 - [189] Yun Peng, Shuzheng Gao, Cuiyun Gao, Yintong Huo, and Michael Lyu. 2024. Domain Knowledge Matters: Improving Prompts with Fix Templates for Repairing Python Type Errors. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–13.
 - [190] Kai Petersen, Sairam Vakkalanka, and Ludwik Kuzniarz. 2015. Guidelines for Conducting Systematic Mapping Studies in Software Engineering: An Update. *Information and Software Technology* 64 (2015), 1–18.
 - [191] Julian Aron Prenner, Hlib Babii, and Romain Robbes. 2022. Can OpenAI’s Codex Fix Bugs? An Evaluation on QuixBugs. In *Proceedings of the Third International Workshop on Automated Program Repair*. 69–75.
 - [192] Julian Aron Prenner and Romain Robbes. 2023. Out of Context: How Important Is Local Context in Neural Program Repair? *arXiv preprint arXiv:2312.04986* (2023).
 - [193] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. GPT-1: Improving Language Understanding by Generative Pre-Training. URL: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf. Lasted accessed: 2024-04-01.
 - [194] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language Models Are Unsupervised Multitask Learners. *OpenAI blog* 1, 8 (2019), 9.
 - [195] Noam Raffel, Colinand Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *The Journal of Machine Learning Research* 21, 1 (2020), 5485–5551.
 - [196] Francisco Ribeiro, Rui Abreu, and João Saraiva. 2022. Framing Program Repair as Code Completion. In *Proceedings of the Third International Workshop on Automated Program Repair*. IEEE, 38–45.
 - [197] Francisco Ribeiro, José Nuno Castro de Macedo, Kanae Tsushima, Rui Abreu, and João Saraiva. 2023. GPT-3-Powered Type Error Debugging: Investigating the Use of Large Language Models for Code Repair. In *Proceedings of the 16th ACM SIGPLAN International Conference on Software Language Engineering*. 111–124.
 - [198] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023).
 - [199] Fernando Vallecillos Ruiz, Anastasiia Grishina, Max Hort, and Leon Moonen. 2024. A Novel Approach for Automatic Program Repair Using Round-Trip Translation with Large Language Models. *arXiv preprint arXiv:2401.07994* (2024).
 - [200] Fernando Vallecillos Ruiz, Max Hort, and Leon Moonen. 2025. The Art of Repair: Optimizing Iterative Program Repair with Instruction-Tuned Models. *arXiv preprint arXiv:2505.02931* (2025).
 - [201] Ahmadreza Saboor Yaraghi, Darren Holden, Nafiseh Kahani, and Lionel Briand. 2024. Automated Test Case Repair Using Language Models. *arXiv e-prints* (2024), arXiv–2401.
 - [202] Max Schafer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105.
 - [203] Ye Shang, Quanjun Zhang, Chunrong Fang, Siqi Gu, Jianyi Zhou, and Zhenyu Chen. 2025. A large-scale empirical study on fine-tuning large language models for unit testing. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1678–1700.
 - [204] Jieke Shi, Zhou Yang, Bowen Xu, Hong Jin Kang, and David Lo. 2022. Compressing Pre-Trained Models of Code into 3 Mb. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.
 - [205] Yuling Shi, Songsong Wang, Chengcheng Wan, Min Wang, and Xiaodong Gu. 2024. From code to correctness: Closing the last mile of code generation with hierarchical debugging. *arXiv preprint arXiv:2410.01215* (2024).
 - [206] André Silva, Sen Fang, and Martin Monperrus. 2023. RepairLLaMA: Efficient Representations and Fine-Tuned Adapters for Program Repair. *arXiv preprint arXiv:2312.15698* (2023).
 - [207] André Silva, Nuno Saavedra, and Martin Monperrus. 2024. GitBug-Java: A Reproducible Benchmark of Recent Java Bugs. *arXiv preprint arXiv:2402.02961* (2024).
 - [208] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. 2023. An Analysis of the Automatic Bug Fixing Performance of ChatGPT. In *2023 IEEE/ACM International Workshop on Automated Program Repair*. 23–30.

- [209] Benjamin Steenhoeck, Kalpathy Sivaraman, Renata Saldívar Gonzalez, Yevhen Mohylevskyy, Roshanak Zilouchian Moghaddam, and Wei Le. 2025. Closing the Gap: A User Study on the Real-world Usefulness of AI-powered Vulnerability Detection & Repair in the IDE. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 663–663.
- [210] Chia-Yi Su and Collin McMillan. 2024. Distilled GPT for Source Code Summarization. *Automated Software Engineering* 31, 1 (2024), 22.
- [211] Weisong Sun, Chunrong Fang, Yudu You, Yun Miao, Yi Liu, Yuekang Li, Gelei Deng, Shenghan Huang, Yuchen Chen, Quanjun Zhang, et al. 2023. Automatic Code Summarization via ChatGPT: How Far Are We? *arXiv preprint arXiv:2305.12865* (2023).
- [212] Hao Tang, Keya Hu, Jin Zhou, Si Cheng Zhong, Wei-Long Zheng, Xujie Si, and Kevin Ellis. 2024. Code Repair with LLMs gives an Exploration-Exploitation Tradeoff. *Advances in Neural Information Processing Systems* 37 (2024), 117954–117996.
- [213] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. 2024. MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue Resolution. *Advances in Neural Information Processing Systems* 37 (2024), 51963–51993.
- [214] Yida Tao, Jindae Kim, Sunghun Kim, and Chang Xu. 2014. Automatically Generated Patches As Debugging Aids: A Human Study. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 64–74.
- [215] Haoye Tian, Kui Liu, Abdoul Kader Kaboré, Anil Koyuncu, Li Li, Jacques Klein, and Tegawendé F Bissyandé. 2020. Evaluating Representation Learning of Code Changes for Predicting Patch Correctness in Program Repair. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 981–992.
- [216] Haoye Tian, Kui Liu, Yinghua Li, Abdoul Kader Kaboré, Anil Koyuncu, Andrew Habib, Li Li, Junhao Wen, Jacques Klein, and Tegawendé F Bissyandé. 2022. The Best of Both Worlds: Combining Learned Embeddings with Engineered Features for Accurate Prediction of Correct Patches. *ACM Transactions on Software Engineering and Methodology* 1, 1 (2022), 1–1.
- [217] Haoye Tian, Kui Liu, Yinghua Li, Abdoul Kader Kaboré, Anil Koyuncu, Andrew Habib, Li Li, Junhao Wen, Jacques Klein, and Tegawendé F Bissyandé. 2023. The Best of Both Worlds: Combining Learned Embeddings with Engineered Features for Accurate Prediction of Correct Patches. *ACM Transactions on Software Engineering and Methodology* 32, 4 (2023), 1–34.
- [218] Haoye Tian, Weiqi Lu, Tsz On Li, Xunzhu Tang, Shing-Chi Cheung, Jacques Klein, and Tegawendé F Bissyandé. 2023. Is ChatGPT the Ultimate Programming Assistant—How Far Is It? *arXiv preprint arXiv:2304.11938* (2023).
- [219] Haoye Tian, Xunzhu Tang, Andrew Habib, Shangwen Wang, Kui Liu, Xin Xia, Jacques Klein, and Tegawendé F Bissyandé. 2022. Is This Change the Answer to That Problem? Correlating Descriptions of Bug and Code Changes for Evaluating Patch Correctness. In *37th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 1–13.
- [220] Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2024. DebugBench: Evaluating Debugging Capability of Large Language Models. *arXiv preprint arXiv:2401.04621* (2024).
- [221] Yuan Tian, David Lo, and Chengnian Sun. 2013. DRONE: Predicting Priority of Reported Bugs by Multi-factor Analysis. In *Proceedings of the 2013 IEEE International Conference on Software Maintenance*. 200–209.
- [222] M Caner Tol and Berk Sunar. 2023. Zeroleak: Using LLMs for Scalable and Cost Effective Side-Channel Patching. *arXiv preprint arXiv:2308.13062* (2023).
- [223] YunDa Tsai, Mingjie Liu, and Haoxing Ren. 2023. RTLFixer: Automatically Fixing RTL Syntax Errors with Large Language Models. *arXiv preprint arXiv:2311.16543* (2023).
- [224] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An Empirical Study on Learning Bug-Fixing Patches in the Wild Via Neural Machine Translation. *ACM Transactions on Software Engineering and Methodology* 28, 4 (2019), 1–29.
- [225] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*. 5998–6008.
- [226] Nalin Wadhwa, Jui Pradhan, Atharv Sonwane, Surya Prakash Sahu, Nagarajan Natarajan, Aditya Kanade, Suresh Parthasarathy, and Sriram Rajamani. 2023. Frustrated with Code Quality Issues? LLMs Can Help! *arXiv preprint arXiv:2309.12938* (2023).
- [227] Nalin Wadhwa, Jui Pradhan, Atharv Sonwane, Surya Prakash Sahu, Nagarajan Natarajan, Aditya Kanade, Suresh Parthasarathy, and Sriram Rajamani. 2024. CORE: Resolving Code Quality Issues using LLMs. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 789–811.
- [228] Han Wan, Hongzhen Luo, Mengying Li, and Xiaoyan Luo. 2024. Automated program repair for introductory programming assignments. *IEEE Transactions on Learning Technologies* 17 (2024), 1705–1720.
- [229] Ben Wang and Aran Komatsuzaki. 2021. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. URL: <https://github.com/kingoflolz/mesh-transformer-jax>. Lasted accessed: 2024-04-01.
- [230] Deze Wang, Boxing Chen, Shanshan Li, Wei Luo, Shaoliang Peng, Wei Dong, and Xiangke Liao. 2023. One Adapter for All Programming Languages? Adapter Tuning for Code Search and Summarization. *arXiv preprint arXiv:2303.15822* (2023).
- [231] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing with Large Language Models: Survey, Landscape, and Vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936.
- [232] Shangwen Wang, Mingyang Geng, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Li Li, Tegawendé F Bissyandé, and Xiaoguang Mao. 2023. Natural Language to Code: How Far Are We?. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 375–387.

- [233] Simin Wang, Liguang Huang, Amiao Gao, Jidong Ge, Tengfei Zhang, Haitao Feng, Ishna Satyarth, Ming Li, He Zhang, and Vincent Ng. 2022. Machine/Deep Learning for Software Engineering: A Systematic Literature Review. *IEEE Transactions on Software Engineering* (2022).
- [234] Shangwen Wang, Bo Lin, Liqian Chen, and Xiaoguang Mao. 2025. Divide-and-conquer: Automating code revisions via localization-and-revision. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–26.
- [235] Shangwen Wang, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Yan Lei, and Xiaoguang Mao. 2023. Two Birds with One Stone: Boosting Code Generation and Code Search via a Generative Adversarial Network. *Proceedings of the ACM on Programming Languages* 7, OOPSLA2 (2023), 486–515.
- [236] Weishi Wang, Yue Wang, Shafiq Joty, and Steven CH Hoi. 2023. RAP-Gen: Retrieval-Augmented Patch Generation with CodeT5 for Automatic Program Repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 146–158.
- [237] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *arXiv preprint arXiv:2407.16741* (2024).
- [238] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 8696–8708.
- [239] Cody Watson, Nathan Cooper, David Nader Palacio, Kevin Moran, and Denys Poshyvanyk. 2022. A Systematic Literature Review on the Use of Deep Learning in Software Engineering Research. *ACM Transactions on Software Engineering and Methodology* 31, 2 (2022), 1–58.
- [240] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I Wang. 2025. Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449* (2025).
- [241] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the Copilots: Fusing Large Language Models with Completion Engines for Automated Program Repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 172–184.
- [242] Westley Weimer, ThanhVu Nguyen, Claire Le Goues, and Stephanie Forrest. 2009. Automatically Finding Patches Using Genetic Programming. In *2009 IEEE 31st International Conference on Software Engineering*. IEEE, 364–374.
- [243] Cathrin Weiss, Rahul Premraj, Thomas Zimmermann, and Andreas Zeller. 2007. How Long Will It Take to Fix This Bug?. In *Fourth International Workshop on Mining Software Repositories*. IEEE, 1–1.
- [244] Emily Winter, Vesna Nowack, David Bowes, Steve Counsell, Tracy Hall, Sæmundur Haraldsson, and John Woodward. 2022. Let’s Talk with Developers, Not about Developers: A Review of Automatic Program Repair Research. *IEEE Transactions on Software Engineering* 49, 1 (2022), 419–436.
- [245] Kyle Wong, Alfonso Amayuelas, Liangming Pan, and William Yang Wang. 2024. Investigating the transferability of code repair for low-resource programming languages. *arXiv preprint arXiv:2406.14867* (2024).
- [246] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa. 2016. A Survey on Software Fault Localization. *IEEE Transactions on Software Engineering* 42, 8 (2016), 707–740.
- [247] Fangzhou Wu, Qingzhao Zhang, Ati Priya Bajaj, Tiffany Bao, Ning Zhang, Ruoyu Wang, Chaowei Xiao, et al. 2023. Exploring the Limits of ChatGPT in Software Security Applications. *arXiv preprint arXiv:2312.05275* (2023).
- [248] Yi Wu, Nan Jiang, Hung Viet Pham, Thibaud Lutellier, Jordan Davis, Lin Tan, Petr Babkin, and Sameena Shah. 2023. How Effective Are Neural Networks for Fixing Security Vulnerabilities. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2023)*. Association for Computing Machinery, 1282–1294.
- [249] Yonghao Wu, Zheng Li, Jie M Zhang, and Yong Liu. 2023. ConDefects: A New Dataset to Address the Data Leakage Concern for LLM-based Fault Localization and Program Repair. *arXiv preprint arXiv:2310.16253* (2023).
- [250] Marchel Christoper Wuisang, Marcel Kurniawan, Komang Andika Wira Santosa, Alexander Agung Santoso Gunawan, and Karen Etania Saputra. 2023. An Evaluation of the Effectiveness of OpenAI’s ChatGPT for Automated Python Program Bug Fixing Using QuixBugs. In *2023 International Seminar on Application for Technology of Information and Communication*. IEEE, 295–300.
- [251] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying llm-based software engineering agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824.
- [252] Chunqiu Steven Xia, Yifeng Ding, and Lingming Zhang. 2023. The Plastic Surgery Hypothesis in the Era of Large Language Models. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 522–534.
- [253] Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. 2025. Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly? *arXiv preprint arXiv:2511.13646* (2025).
- [254] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated Program Repair in the Era of Large Pre-trained Language Models. In *2023 IEEE/ACM 45th International Conference on Software Engineering*. IEEE Computer Society, 1482–1494.

- [255] Chunqiu Steven Xia and Lingming Zhang. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair Via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 959–971.
- [256] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 819–831.
- [257] Jiahong Xiang, Xiaoyang Xu, Fanchu Kong, Mingyuan Wu, Zizheng Zhang, Haotian Zhang, and Yuqun Zhang. 2024. How far can we go with practical function-level program repair? *arXiv preprint arXiv:2404.12833* (2024).
- [258] Yuan-An Xiao, Chenyang Yang, Bo Wang, and Yingfei Xiong. 2024. Accelerating Patch Validation for Program Repair with Interception-Based Execution Scheduling. *IEEE Transactions on Software Engineering* 01 (2024), 1–18.
- [259] Danning Xie, Byungwoo Yoo, Nan Jiang, Mijung Kim, Lin Tan, Xiangyu Zhang, and Judy S Lee. 2023. Impact of Large Language Models on Generating Software Specifications. *arXiv preprint arXiv:2306.03324* (2023).
- [260] Yingfei Xiong, Xinyuan Liu, Muhan Zeng, Lu Zhang, and Gang Huang. 2018. Identifying Patch Correctness in Test-Based Program Repair. In *Proceedings of the 40th IEEE/ACM International Conference on Software Engineering*. 789–799.
- [261] Yingfei Xiong, Jie Wang, Runfa Yan, Jiachen Zhang, Shi Han, Gang Huang, and Lu Zhang. 2017. Precise Condition Synthesis for Program Repair. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering*. IEEE, 416–426.
- [262] Junjielong Xu, Ying Fu, Shin Hwei Tan, and Pinjia He. 2025. Aligning the Objective of LLM-Based Program Repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2548–2560.
- [263] Zhuolin Xu, Yuanzhang Lin, Qiushi Li, and Shin Hwei Tan. 2023. Guiding ChatGPT to Fix Web Ui Tests Via Explanation-Consistency Checking. *arXiv preprint arXiv:2312.05778* (2023).
- [264] Jifeng Xuan, Matias Martinez, Favio Demarco, Maxime Clement, Sebastian Lamelas Marcote, Thomas Durieux, Daniel Le Berre, and Martin Monperrus. 2016. Nopol: Automatic Repair of Conditional Statement Bugs in Java Programs. *IEEE Transactions on Software Engineering* 43, 1 (2016), 34–55.
- [265] Pengyu Xue, Linhao Wu, Zhen Yang, Zhongxing Yu, Zhi Jin, Ge Li, Yan Xiao, Shuo Liu, Xinyi Li, Hongyi Lin, et al. 2024. Exploring and Lifting the Robustness of LLM-powered Automated Program Repair with Metamorphic Testing. *arXiv preprint arXiv:2410.07516* (2024).
- [266] Weixiang Yan, Haitian Liu, Yunkun Wang, Yunzhe Li, Qian Chen, Wen Wang, Tingyu Lin, Weishan Zhao, Li Zhu, Shuiguang Deng, et al. 2023. Codescope: An Execution-Based Multilingual Multitask Multidimensional Benchmark for Evaluating LLMs on Code Understanding and Generation. *arXiv preprint arXiv:2311.08588* (2023).
- [267] Aidan ZH Yang, Sophia Kolak, Vincent J Hellendoorn, Ruben Martins, and Claire Le Goues. 2024. Revisiting unnaturalness for automated program repair in the era of large language models. *arXiv preprint arXiv:2404.15236* (2024).
- [268] Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F Bissyandé, and Shunfu Jin. 2024. Cref: An llm-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 882–894.
- [269] Boyang Yang, Haoye Tian, Jiadong Ren, Hongyu Zhang, Jacques Klein, Tegawendé F Bissyandé, Claire Le Goues, and Shunfu Jin. 2024. MORepair: Teaching LLMs to Repair Code via Multi-Objective Fine-Tuning. *arXiv preprint arXiv:2404.12636* (2024).
- [270] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.
- [271] John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, et al. 2024. SWE-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859* (2024).
- [272] Yanming Yang, Xin Xia, David Lo, and John Grundy. 2022. A Survey on Deep Learning for Software Engineering. *Comput. Surveys* 54, 10s (2022), 1–73.
- [273] Xufeng Yao, Haoyang Li, Tsz Ho Chan, Wenyi Xiao, Mingxuan Yuan, Yu Huang, Lei Chen, and Bei Yu. 2024. HDLdebugger: Streamlining HDL debugging with Large Language Models. *arXiv preprint arXiv:2403.11671* (2024).
- [274] He Ye, Matias Martinez, Xiapu Luo, Tao Zhang, and Martin Monperrus. 2022. SelfAPR: Self-Supervised Program Repair with Test Execution Diagnostics. In *2022 37th IEEE/ACM International Conference on Automated Software Engineering*. IEEE.
- [275] He Ye, Matias Martinez, and Martin Monperrus. 2022. Neural Program Repair with Execution-Based Backpropagation. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering*. 1506–1518.
- [276] He Ye and Martin Monperrus. 2024. ITER: Iterative Neural Repair for Multi-Location Patches. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 79–91.
- [277] He Ye, Aidan ZH Yang, Chang Hu, Yanlin Wang, Tao Zhang, and Claire Le Goues. 2025. AdverIntent-Agent: Adversarial Reasoning for Repair Based on Inferred Program Intent. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1398–1420.
- [278] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. 2024. Thinkrepair: Self-directed automated program repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1274–1286.

- [279] Zheng Yu, Ziyi Guo, Yuhang Wu, Jiahao Yu, Meng Xu, Dongliang Mu, Yan Chen, and Xinyu Xing. 2025. PATCHAGENT: A Practical Program Repair Agent Mimicking Human Expertise. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security'25)*, Seattle, WA, USA.
- [280] Mingyue Yuan, Jieshan Chen, Zhenchang Xing, Aaron Quigley, Yuyu Luo, Tianqi Luo, Gelareh Mohammadi, Qinghua Lu, and Liming Zhu. 2024. Designrepair: Dual-stream design guideline-aware frontend repair with large language models. *arXiv preprint arXiv:2411.01606* (2024).
- [281] Wei Yuan, Quanjun Zhang, Tieke He, Chunrong Fang, Nguyen Quoc Viet Hung, Xiaodong Hao, and Hongzhi Yin. 2022. CIRCLE: Continual Repair across Programming Languages. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, 678–690.
- [282] Yuan Yuan and Wolfgang Banzhaf. 2018. ARJA: Automated Repair of Java Programs Via Multi-objective Genetic Programming. *IEEE Transactions on Software Engineering* 46, 10 (2018), 1040–1067.
- [283] He Zhang, Muhammad Ali Babar, and Paolo Tell. 2011. Identifying Relevant Studies in Software Engineering. *Information and Software Technology* 53, 6 (2011), 625–637.
- [284] Jialu Zhang, José Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2022. Repairing Bugs in Python Assignments Using Large Language Models. *arXiv preprint arXiv:2209.14876* (2022).
- [285] Jialu Zhang, José Pablo Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2024. PyDex: Repairing Bugs in Introductory Python Assignments using LLMs. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 1100–1124.
- [286] Jiayi Zhang, Kai Huang, Jian Zhang, Yang Liu, and Chunyang Chen. 2025. Repair Ingredients Are All You Need: Improving Large Language Model-Based Program Repair via Repair Ingredients Search. *arXiv preprint arXiv:2506.23100* (2025).
- [287] Jian Zhang, Chong Wang, Anran Li, Wenhan Wang, Tianlin Li, and Yang Liu. 2024. Vuladvisor: Natural language suggestion generation for software vulnerability repair. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1932–1944.
- [288] Kechi Zhang, Zhuo Li, Ge Li, and Zhi Jin. 2023. Self-Edit: Fault-Aware Code Editor for Code Generation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 769–787.
- [289] Lyuye Zhang, Kaixuan Li, Kairan Sun, Daoyuan Wu, Ye Liu, Haoye Tian, and Yang Liu. 2024. ACFIX: Guiding LLMs with Mined Common RBAC Practices for Context-Aware Repair of Access Control Vulnerabilities in Smart Contracts. *arXiv preprint arXiv:2403.06838* (2024).
- [290] Mengxiao Zhang, Yongqiang Tian, Zhenyang Xu, Yiwen Dong, Shin Hwei Tan, and Chengnian Sun. 2024. LPR: Large Language Models-Aided Program Reduction. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 261–273.
- [291] Quanjun Zhang, Chunrong Fang, Siqi Gu, Ye Shang, Zhenyu Chen, and Liang Xiao. 2025. Large Language Models for Unit Testing: A Systematic Literature Review. *arXiv preprint arXiv:2506.15227* (2025).
- [292] Quanjun Zhang, Chunrong Fang, Yuxiang Ma, Weisong Sun, and Zhenyu Chen. 2023. A Survey of Learning-Based Automated Program Repair. *ACM Transactions on Software Engineering and Methodology* 33, 2 (2023), 1–69.
- [293] Quanjun Zhang, Chunrong Fang, Weisong Sun, Yan Liu, Tieke He, Xiaodong Hao, and Zhenyu Chen. 2024. APPT: Boosting Automated Patch Correctness Prediction via Fine-Tuning Pre-Trained Models. *IEEE Transactions on Software Engineering* 50, 3 (2024), 474–494.
- [294] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Yun Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. 2023. A Survey on Large Language Models for Software Engineering. *arXiv preprint arXiv:2312.15223* (2023).
- [295] Quanjun Zhang, Chunrong Fang, Bowen Yu, Weisong Sun, Tongke Zhang, and Zhenyu Chen. 2023. Pre-Trained Model-Based Automated Software Vulnerability Repair: How Far are We? *IEEE Transactions on Dependable and Secure Computing* (2023). Early Access, DOI: 10.1109/TDSC.2023.3308897.
- [296] Quanjun Zhang, Chunrong Fang, Tongke Zhang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. GAMMA: Revisiting Template-based Automated Program Repair via Mask Prediction. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 535–547.
- [297] Quanjun Zhang, Chunrong Fang, Yi Zheng, Ruixiang Qian, Shengcheng Yu, Yuan Zhao, Jianyi Zhou, Yun Yang, Tao Zheng, and Zhenyu Chen. 2025. Improving Retrieval-Augmented Deep Assertion Generation via Joint Training. *IEEE Transactions on Software Engineering* 51, 4 (2025), 1232–1247.
- [298] Quanjun Zhang, Chunrong Fang, Yi Zheng, Yaxin Zhang, Yuan Zhao, Rubing Huang, Jianyi Zhou, Yun Yang, Tao Zheng, and Zhenyu Chen. 2025. Improving Deep Assertion Generation via Fine-Tuning Retrieval-Augmented Pre-trained Language Models. *ACM Transactions on Software Engineering and Methodology* (2025).
- [299] Quanjun Zhang, Weifeng Sun, Chunrong Fang, Bowen Yu, Hongyan Li, Meng Yan, Jianyi Zhou, and Zhenyu Chen. 2025. Exploring automated assertion generation via large language models. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–25.
- [300] Quanjun Zhang, Tongke Zhang, Juan Zhai, Chunrong Fang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. A Critical Review of Large Language Model on Software Engineering: An Example from ChatGPT and Automated Program Repair. *arXiv preprint*

- arXiv:2310.08879* (2023).
- [301] Quanjun Zhang, Yuan Zhao, Weisong Sun, Chunrong Fang, Ziyuan Wang, and Lingming Zhang. 2022. Program Repair: Automated vs. Manual. *arXiv preprint arXiv:2203.05166* (2022).
 - [302] Ting Zhang, Ivana Clairine Irsan, Ferdian Thung, David Lo, Asankhaya Sharma, and Lingxiao Jiang. 2023. Evaluating Pre-Trained Language Models for Repairing Api Misuses. *arXiv preprint arXiv:2310.16390* (2023).
 - [303] Yuntong Zhang, Xiang Gao, Gregory J. Duck, and Abhik Roychoudhury. 2022. Program Vulnerability Repair Via Inductive Inference. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 691–702.
 - [304] Yuwei Zhang, Zhi Jin, Ying Xing, and Ge Li. 2023. Steam: Simulating the Interactive Behavior of Programmers for Automatic Bug Fixing. *arXiv preprint arXiv:2308.14460* (2023).
 - [305] Yuwei Zhang, Ge Li, Zhi Jin, and Ying Xing. 2023. Neural Program Repair with Program Dependence Analysis and Effective Filter Mechanism. *arXiv preprint arXiv:2305.09315* (2023).
 - [306] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1592–1604.
 - [307] Yuntong Zhang, Ridwan Shariffdeen, Gregory J Duck, Jiaqi Tan, and Abhik Roychoudhury. 2023. Program Repair by Fuzzing over Patch and Input Space. *arXiv preprint arXiv:2308.00666* (2023).
 - [308] Ziyin Zhang, Chaoyu Chen, Bingchang Liu, Cong Liao, Zi Gong, Hang Yu, Jianguo Li, and Rui Wang. 2023. Unifying the Perspectives of NLP and Software Engineering: A Survey on Language Models for Code. *arXiv preprint arXiv:2311.07989* (2023).
 - [309] Jiuang Zhao, Donghao Yang, Li Zhang, Xiaoli Lian, Zitian Yang, and Fang Liu. 2024. Enhancing automated program repair with solution design. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1706–1718.
 - [310] Qianhui Zhao, Fang Liu, Li Zhang, Yang Liu, Zhen Yan, Zhenghao Chen, Yufei Zhou, Jing Jiang, and Ge Li. 2024. Peer-aided Repairer: Empowering Large Language Models to Repair Advanced Student Assignments. *arXiv preprint arXiv:2404.01754* (2024).
 - [311] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A Survey of Large Language Models. *arXiv preprint arXiv:2303.18223* (2023).
 - [312] Yuze Zhao, Zhenya Huang, Yixiao Ma, Rui Li, Kai Zhang, Hao Jiang, Qi Liu, Linbo Zhu, and Yu Su. 2024. RePair: Automated Program Repair with Process-based Feedback. In *Findings of the Association for Computational Linguistics: ACL 2024*. 16415–16429.
 - [313] Zelin Zhao, Zhaogui Xu, Jialong Zhu, Peng Di, Yuan Yao, and Xiaoxing Ma. 2023. The Right Prompts for the Job: Repair Code-Review Defects with Large Language Model. *arXiv preprint arXiv:2312.17485* (2023).
 - [314] Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhui Chen, and Xiang Yue. 2024. OpenCodeInterpreter: Integrating Code Generation with Execution and Refinement. *arXiv preprint arXiv:2402.14658* (2024).
 - [315] Li Zhong, Zilong Wang, and Jingbo Shang. 2024. LDB: A Large Language Model Debugger via Verifying Runtime Execution Step-by-step. *arXiv preprint arXiv:2402.16906* (2024).
 - [316] Xin Zhou, Kisub Kim, Bowen Xu, DongGyun Han, and David Lo. 2024. Out of Sight, Out of Mind: Better Automatic Vulnerability Repair by Broadening Input Ranges and Sources. In *2024 IEEE/ACM 46th International Conference on Software Engineering*. IEEE Computer Society, 872–872.
 - [317] Xin Zhou, Bowen Xu, Kisub Kim, Dong Gyun Han, Hung Huu Nguyen, Thanh Le-Cong, Junda He, Bach Le, and David Lo. 2024. Leveraging Large Language Model for Automatic Patch Correctness Assessment. *IEEE Transactions on Software Engineering* 50, 11 (2024), 2865–2883.
 - [318] Qihao Zhu, Zeyu Sun, Yuan-an Xiao, Wenjie Zhang, Kang Yuan, Yingfei Xiong, and Lu Zhang. 2021. A Syntax-Guided Edit Decoder for Neural Program Repair. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 341–353.
 - [319] Qihao Zhu, Zeyu Sun, Wenjie Zhang, Yingfei Xiong, and Lu Zhang. 2023. Tare: Type-Aware Neural Program Repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering*. IEEE, 1443–1455.
 - [320] Yuqi Zhu, Jia Li, Ge Li, YunFei Zhao, Zhi Jin, and Hong Mei. 2024. Hot or Cold? Adaptive Temperature Sampling for Code Generation with Large Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 437–445.
 - [321] Armin Zirak and Hadi Hemmati. 2024. Improving Automated Program Repair with Domain Adaptation. *ACM Transactions on Software Engineering and Methodology* 33, 3 (2024), 1–43.